



Tunisian Republic
Ministry of Higher Education
Higher Institute Of Technological Studies Rades

End-of-Studies Project Report

Higher Institute Of Technological Studies Rades

by

Mohamed Dhia SLEIMI

Mustapha BOUZIRI

MES System

Complete Project Report

made at

Mavision

Résumé

Français : [Résumé en français, 100–150 mots.]

Mots-clés : MES, Flutter, Microsoft Dynamics 365 Business Central, AL, production, traçabilité

English : [Abstract in English, 100–150 words.]

Keywords: MES, Flutter, Microsoft Dynamics 365 Business Central, AL, production, traceability

العربية: ملخص باللغة العربية، 100--150 كلمة .
الكلمات المفتاحية: نظام تنفيذ التصنيع، فلاتر، مايكروسوفت داينامكس 365

Dédicaces

[Dédicace de Mohamed Dhia SLEIMI]

[Dédicace de Mustapha BOUZIRI]

Acknowledgements

First and foremost, We would like to thank our academic supervisor, **Ms. Marwa CHAABANI**, for their expert guidance and unwavering commitment. Their in-depth knowledge, availability and unconditional support have been of paramount importance in the completion of this project.

We would also like to express our gratitude to our professional supervisor at **Mavision**, **Mr. Mourad YOUNSI**. Their contribution to this project, both on a technical and organisational level, has been invaluable. Their strategic vision, domain expertise and availability greatly facilitated our progress and allowed us to gain an enriching professional experience.

Contents

Dédicaces	2
Acknowledgements	3
List of Abbreviations	13
General Introduction	1
1 Global Project Framework	2
Introduction	3
1.1 Presentation of the Host Company	3
1.1.1 Overview of Mavision	3
1.1.2 Areas of Expertise	3
1.2 Project Framework	4
1.3 State of the Art	4
1.3.1 Subject Overview	4
1.3.2 Problem Statement	4
1.3.3 Study of the Existing Situation	5
1.3.4 Critical Analysis of the Existing Situation	5
1.3.5 Proposed Solution	7
1.3.6 Objectives	7
1.3.7 Adopted Methodology	7
Conclusion	8
2 Requirements Analysis and Specification	9
Introduction	10
2.1 Requirements Analysis	10
2.1.1 Identification of Actors	10
2.1.2 Identification of Requirements	10
2.2 Project Management with Scrum	12
2.2.1 Backlog Features	12
2.3 Sprint Plan	20
2.4 Global Application Architecture	21

2.4.1 Physical Architecture	21
2.4.2 Software Architecture	22
2.5 Work Environment	23
2.5.1 Software Environment	23
2.5.2 Development Environment	24
Conclusion	24
3 Sprint 1 Authentication & User Management	25
Introduction	26
3.1 Sprint Backlog	26
3.2 Design	30
3.2.1 Use Case Diagram	30
3.2.2 Textual Use Case Description	31
3.2.3 Sequence Diagrams	32
3.2.4 Class Diagram	39
3.3 Implementation	40
3.3.1 Backend Implementation	40
3.3.2 REST API and Web Services Implementation	44
3.3.3 Frontend Implementation	44
3.4 Testing and Validation	49
3.4.1 Integration testing	49
Conclusion	50
4 Sprint 2 Machine & Production Management	51
Introduction	52
4.1 Sprint Backlog	52
4.2 Business Rules	55
4.3 Design	57
4.3.1 Use Case Diagram	57
4.3.2 Textual Use Case Description	57
4.3.3 Sequence Diagrams	58
4.3.4 Class Diagram	63
4.4 Implementation	64
4.4.1 Backend Implementation	64
4.4.2 REST API and Web Services Implementation	66
4.4.3 Frontend Implementation	67
4.5 Testing and Validation	71
4.5.1 Integrations tests	71
Conclusion	72

5 Sprint 3 Production Execution & Traceability	73
Introduction	74
5.1 Sprint Backlog	74
5.2 Business Rules	80
5.3 Design	83
5.3.1 Use Case Diagram	83
5.3.2 Textual Use Case Description	86
5.3.3 Sequence Diagrams	87
5.3.4 Class Diagram	96
5.4 Implementation	97
5.4.1 Backend Implementation	97
5.4.2 Frontend Implementation	99
5.5 Test and Validation	102
5.5.1 Integration test	102
Conclusion	103
6 Sprint 4 Administration, Printing & Scanning	104
Introduction	105
6.1 Sprint Backlog	105
6.2 Business Rules	106
6.3 Design	107
6.3.1 Use Case Diagram	107
6.3.2 Textual Use Case Description	108
6.3.3 Sequence Diagrams	109
6.3.4 Class Diagram	111
6.4 Implementation	112
6.4.1 Backend Implementation	112
6.4.2 Frontend Implementation	112
6.5 Test and validation	115
6.5.1 Integration test	115
Conclusion	116
7 Sprint 5 AI Assistant & Analytical Intelligence.	118
Introduction	119
7.1 Sprint Backlog	119
7.2 Business Rules	122
7.3 Design	125
7.3.1 Use Case Diagram	125
7.3.2 Textual Use Case Description	126
7.3.3 Sequence Diagrams	128

7.3.4 Class Diagram 130

7.4 Implementation 131

7.4.1 Python AI Agent 131

7.4.2 Node.js Middleware 135

7.4.3 Flutter Frontend 135

Conclusion 136

General Conclusion 138

List of Tables

1.1	Comparison table with Delmia Apriso	6
2.1	List of MES application users	10
2.7	MES project sprint plan	21
2.8	MES project software environment	23
3.1	Story Backlog	26
3.2	Business Rules	29
3.3	Textual Use Case Description — Login	32
3.4	MES User (Table 50101) — Field Dictionary	42
3.5	MES Auth Token (Table 50106) — Field Dictionary	43
4.1	Sprint Backlog	52
4.2	Business Rules	55
4.3	Textual Use Case Description — Start Operation	58
4.4	MES Machine Status (Table 50107) — Field Dictionary	64
4.5	MES Operation State (Table 50108) — Field Dictionary	65
5.1	US-3.1 — Declare Production	74
5.2	Business Rules	80
5.3	Textual Use Case Description — Declare Production	86
5.4	Textual Use Case Description — Finish / Cancel Operation	87
5.5	MES Operation Execution (Table 50110) — Field Dictionary	97
5.6	MES Operation Progression (Table 50109) — Field Dictionary	98
5.7	MES Component Consumption (Table 50111) — Field Dictionary	98
5.8	New API Endpoints — Sprint 3	99
5.9	Provider Responsibilities — Sprint 3 Additions	100
6.1	Sprint 4 Story Backlog	105
6.2	Business Rules	107
6.3	Textual Use Case Description — View Machine Dashboard	108
6.4	New API Endpoints — Sprint 4	112
6.5	Provider Responsibilities — Sprint 4 Additions	113
7.1	US-5.1 — AI Chat Interface	119

7.2	Business Rules	122
7.3	Textual Use Case Description — Chat with AI Assistant	126
7.4	Textual Use Case Description — Query Supervisor Overview	127
7.5	Pydantic Models — agent/models.py	131
7.6	Intent Classes — agent/intent.py	132
7.7	MES Tool Catalogue — tools/mes_tools.py	133
7.8	Supported LLM Providers — agent/llm_client.py	135

List of Figures

1.1	Mavision Logo [Source: Mavision, https://www.mavision.tn , visited April 2025]	3
1.2	Mavision Headquarters	4
1.3	Logo of the Delmia Apriso solution [Source: Dassault Systèmes, https://www.3ds.com/products/delmia/apriso , visited April 2025]	6
1.4	Adopted Scrum development cycle	8
2.1	MES System Physical Architecture	22
2.2	MES System Software Architecture	23
3.1	UML Use Case Diagram.	31
3.2	SD-01: Login.	33
3.3	SD-02: Token Validation.	34
3.4	SD-03: Logout	35
3.5	SD-03: Change Password	36
3.6	SD-05: Admin - Create User.	37
3.7	SD-06: Admin - Set Password.	37
3.8	SD-07: Admin - Change Role / Work Centre.	38
3.9	SD-08: Admin - Toggle Account Active Status.	39
3.10	UML Class Diagram.	40
3.11	Backend project structure.	41
3.12	Flutter layered architecture.	45
3.13	<i>Login Page.</i>	46
3.14	<i>MES User List Page.</i>	46
3.15	<i>Assign MES Role Dialog.</i>	47
3.16	<i>Generate Password Dialog.</i>	47
3.17	<i>Password Generated Successfully Dialog.</i>	48
3.18	<i>Change Password Page.</i>	48
3.19	<i>Change Role Dialog.</i>	49
3.20	Postman endpoint test — fetchAllMesUsers.	50
4.1	UML Use Case Diagram — Machine & Production Management.	57
4.2	SD-09: Fetch Machines.	59
4.3	SD-10: Get Machine Orders.	60

4.4	SD-11: Start Operation.	61
4.5	SD-23: Fetch Operation progress.	62
4.6	SD-24: Fetch Operation History	63
4.7	UML Class Diagram — Machine & Production Management.	63
4.8	Flutter machines domain — layered architecture.	67
4.9	<i>Machine List Page</i> — desktop and mobile views.	68
4.10	<i>Machine Orders Page</i> — desktop and mobile views.	69
4.11	<i>Orders Progression Page</i> — desktop and mobile views.	70
4.12	<i>Machine History Page</i> — desktop and mobile views.	70
4.13	Postman endpoint test — fetchMachines.	71
4.14	Postman endpoint test — getMachinesOrders.	72
5.1	UML Use Case Diagram — Sprint 3	85
5.2	SD-12: Declare Production.	88
5.3	SD-21: Fetch Bill of Materials.	89
5.4	SD-22: Fetch Production Cycles.	90
5.5	SD-13: Pause Operation.	91
5.6	SD-14: Resume Operation.	92
5.7	SD-15: Finish/Cancel Operation.	93
5.8	SD-16: declare scrap.	94
5.9	SD-17: scan component.	95
5.10	SD-18: Fetch Operation Live Data.	96
5.11	UML Class Diagram — Production Execution & Traceability.	97
5.12	<i>Operation Detail Page</i> (Running state) — desktop and mobile views.	100
5.13	<i>Operation Detail Page</i> (Finished state) — desktop and mobile views.	101
5.14	<i>Operation Detail Page</i> (Paused state) — desktop and mobile views.	101
5.15	<i>Operation Detail Page</i> (Cancelled state) — desktop and mobile views.	102
5.16	Postman endpoint test — fetchBom.	103
6.1	UML Use Case Diagram — Administration & UX.	108
6.2	SD-19: Fetch Machine Dashboard.	110
6.3	SD-20: Fetch Activity Log.	111
6.4	UML Class Diagram.	111
6.5	<i>Admin Navigation Shell</i> — desktop view.	114
6.6	<i>Machine Dashboard Page</i> — desktop and mobile views.	115
6.7	<i>Activity Log Page</i> — desktop and mobile views.	115
6.8	Postman endpoint test — fetchActivityLog.	116
6.9	Postman endpoint test — fetchMachineDashboard.	116
7.1	UML Use Case Diagram — Sprint 5	126

7.2 SD-30: AI Chat — Main Flow. 129

7.3 SD-31: Intent Classification — LLM with Regex Fallback. 130

7.4 UML Class Diagram — AI Assistant & Analytical Intelligence. 131

7.5 *AI Chat* dialog — mobile view. 136

7.6 *AI Chat* dialog — mobile view. 136

List of Abbreviations

Abbreviation	Definition
AL	Application Language (Microsoft Dynamics 365 Business Central)
API	Application Programming Interface
BOM	Bill of Materials
ERP	Enterprise Resource Planning
GUID	Globally Unique Identifier
MES	Manufacturing Execution System
OData	Open Data Protocol
RTL	Right-to-Left
SCRUM	(no acronym — Scrum is a proper noun)
SHA-256	Secure Hash Algorithm 256-bit
UML	Unified Modelling Language

General Introduction

In an industrial context undergoing rapid digital transformation, manufacturing companies face the necessity of optimising their production processes, reducing downtime and ensuring complete traceability of workshop operations. Manufacturing Execution Systems, known by the acronym MES (*Manufacturing Execution System*), represent a direct response to these challenges by bridging the gap between decision-making layers (ERP) and the production floor.

It is within this context that the present end-of-studies project was carried out at **Mavision**. The project consists of the design and development of an integrated MES system, structured around two main components: a business back-end developed in AL language on the Microsoft Dynamics 365 Business Central platform, and a cross-platform mobile/web application developed with the Flutter framework. The system enables machine management, production order tracking, real-time production reporting, as well as user and role-based access management.

This report is structured into several chapters. The first chapter introduces the host organisation and presents the general framework of the project, highlighting the limitations of existing solutions and the relevance of the proposed solution. The second chapter focuses on requirements analysis and specification, as well as the technology choices made. The subsequent chapters trace the various development phases, exploring the conceptual aspects and illustrating the results obtained at the end of each sprint.

Through this approach, we aim to demonstrate how a well-designed digital solution can transform workshop production management by providing visibility, responsiveness and performance, serving both operators and management alike.

Chapter 1

Global Project Framework

Contents

Introduction	3
1.1 Presentation of the Host Company	3
1.1.1 Overview of Mavision	3
1.1.2 Areas of Expertise	3
1.2 Project Framework	4
1.3 State of the Art	4
1.3.1 Subject Overview	4
1.3.2 Problem Statement	4
1.3.3 Study of the Existing Situation	5
1.3.4 Critical Analysis of the Existing Situation	5
1.3.5 Proposed Solution	7
1.3.6 Objectives	7
1.3.7 Adopted Methodology	7
Selection of the Agile Methodology	8
Adoption of the Scrum Framework	8
Conclusion	8

Introduction

This first chapter serves to contextualise our project. It begins with a brief introduction to the host organisation, then describes the objectives to be achieved, provides an in-depth analysis of the existing situation, and finally describes the methodology used to implement this work.

1.1 Presentation of the Host Company

To properly organise this section, we begin with some general details about Mavision before addressing the activities directly relevant to us.

1.1.1 Overview of Mavision

Founded in 2014 and headquartered in Manar II, Tunis, Mavision [1] is a software development company specialising in consulting, auditing and implementing ERP information systems. Mavision supports companies in their IT development projects and helps its clients better organise themselves around their information systems. In addition to its ERP offerings, Mavision develops and integrates reporting solutions to help its clients manage their business activities more effectively. In order to build lasting trust, Mavision places emphasis on its professionalism, commitment and the technical and functional expertise of its team.



Figure 1.1. Mavision Logo [Source: Mavision, <https://www.mavision.tn>, visited April 2025]

1.1.2 Areas of Expertise

Mavision's activities are structured around three main areas:

ERP Integration: Consulting, implementation and integration of ERP solutions, with a strong focus on Microsoft Dynamics 365 Business Central, as well as project and platform management, architecture and custom business software design.

Business Intelligence: Design and deployment of BI solutions enabling clients to gain strategic visibility over their operations through dashboards, reporting tools and data analysis.

Web & Mobile Technology: Development and integration of mobile phone applications and web platforms that complement ERP systems, enabling clients to manage their business activities with greater agility and efficiency.

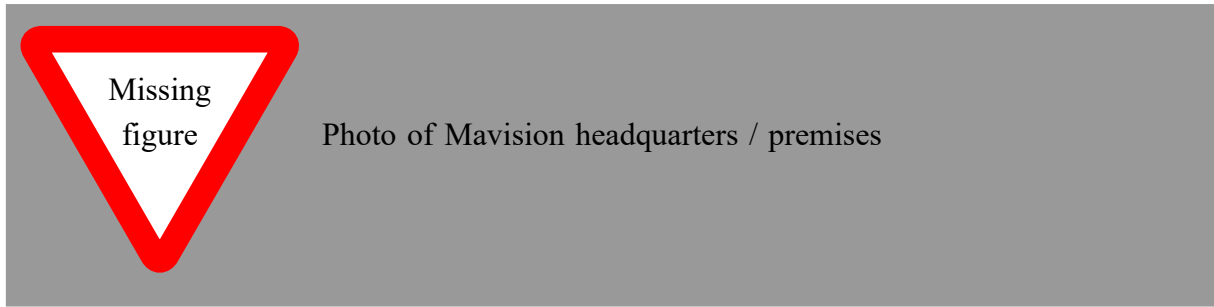


Figure 1.2. Mavision Headquarters

1.2 Project Framework

This project forms part of our end-of-studies internship, an essential step for obtaining the bachelor's degree at the Higher Institute of Technological Studies. This internship, lasting four months, took place from February 2nd to May 30th at Mavision. During this period, we were assigned to the development team.

This team is responsible for designing and maintaining software solutions that meet the company's business needs, leveraging modern technologies such as Microsoft Dynamics 365 Business Central and Flutter, and agile practices. Within this context, we worked on the development of the MES (*Manufacturing Execution System*) project, a mobile and web solution enabling intelligent management of machines and production operations on the workshop floor.

1.3 State of the Art

1.3.1 Subject Overview

With the rise of Industry 4.0 and the widespread adoption of ERP systems in manufacturing companies, the need to connect decision-making levels with floor operations has become critical. A MES (*Manufacturing Execution System*) is an information system that provides this link in real time: it supervises the execution of production orders, monitors machine status, records production declarations and provides operators and supervisors with a consolidated view of the workshop.

In this context, our project reflects a desire to provide Mavision and its industrial clients with a MES solution natively integrated with Microsoft Dynamics 365 Business Central. The primary objective is to allow operators to start and monitor their production operations from an intuitive mobile/web application, while ensuring data consistency with the ERP.

1.3.2 Problem Statement

The problem of optimising production floor operations arises with particular force in manufacturing companies that rely on an ERP system such as Microsoft Dynamics 365 Business Central. Indeed, while Business Central effectively handles production planning and order management

at the decision-making level, no unified, intelligent and dynamic solution exists to bridge the gap between the ERP and the shop floor in real time.

This gap leads to several consequences: inability to track the progress of production orders as they are being executed, manual entry of production declarations through paper routing sheets or spreadsheet files, lack of visibility for supervisors and managers on machine states and operation statuses, and insufficient traceability of actual shop-floor activity. Thus, a central question arises: how can Mavision provide its industrial clients with a solution that optimises the management of production operations, ensuring effective, accurate and real-time synchronisation between the shop floor and the ERP system?

1.3.3 Study of the Existing Situation

Following an in-depth analysis, it is clear that there is a genuine need within manufacturing companies for a centralised solution to efficiently manage production operations directly from the workshop floor.

Currently, production tracking is often carried out manually, via paper route sheets or spreadsheet files, leading to data entry errors, delays in information reporting and the absence of reliable traceability.

Supervisors and production managers have no real-time visibility into the progress of production orders, nor the current status of machines (running, paused, out of order). There is often no tool to automatically correlate production declarations with orders planned in the ERP.

1.3.4 Critical Analysis of the Existing Situation

In the context of this study, we briefly analysed the MES solution proposed by Delmia Apriso, one of the leading enterprise-grade Manufacturing Execution Systems on the market, whose interface is illustrated in the figure below.

Delmia Apriso is a comprehensive cloud-based MES platform developed by Dassault Systemes, designed for large-scale industrial enterprises. It covers the full spectrum of manufacturing operations including production execution, quality management, labour tracking and supply chain integration. While technically mature and feature-rich, it targets large multinational manufacturers with complex and highly customised deployments, resulting in significant licensing, implementation and maintenance costs that place it out of reach for small and medium-sized manufacturers.



Figure 1.3. Logo of the Delmia Apriso solution [Source: Dassault Systèmes, <https://www.3ds.com/products/delmia/apriso>, visited April 2025]

From Mavision’s perspective as a software provider, the goal is not to use an MES solution internally, but to deliver one to its industrial clients. The solution must therefore be cost-effective to sell, straightforward to deploy and maintain at client sites, and natively integrated into Microsoft Dynamics 365 Business Central, which is already Mavision’s core ERP offering. A heavyweight platform such as Delmia Apriso, which requires a separate infrastructure and lengthy implementation cycles, is incompatible with this commercial model. Table 1.1 summarises the key attributes of Delmia Apriso against the specific needs of the project.

Table 1.1. Comparison table with Delmia Apriso

Features	Project needs	Delmia Apriso
Real-time production order tracking	Yes	Yes
Machine status management	Yes	Yes
Production declaration from mobile	Yes	Partial
Native Business Central integration	Yes	No
Cross-platform mobile interface	Yes	Partial
Role management (Admin / Supervisor / Operator)	Yes	Yes
Secure token-based authentication	Yes	Yes
Adaptability to internal needs	High (custom solution)	Low (proprietary platform)
Ease of deployment at client sites	High	Low
Compatible with SME commercial model	Yes	No

1.3.5 Proposed Solution

The diagnosis of the current situation has highlighted several shortcomings and limitations, as detailed in the previous section. To address these, we propose to design and deploy an innovative MES solution natively integrated with Microsoft Dynamics 365 Business Central.

This solution consists of two complementary parts:

- **An AL back-end (Business Central):** developed in AL language, it exposes OData V4 endpoints enabling user management, token-based authentication, machine and operation tracking, as well as production declaration.
- **A Flutter application (front-end):** a cross-platform mobile and web application allowing operators and supervisors to view production orders assigned to their machine, start, pause or complete an operation, declare produced quantities, and visualise progress in real time.

The objective is to optimise every phase of production management by establishing a reliable, traceable and ergonomic digital framework, serving both operators and management.

1.3.6 Objectives

Our MES solution aims to:

- Provide real-time tracking of machine status and production operations directly from the Business Central ERP.
- Allow operators to start, pause and complete operations from an intuitive mobile interface.
- Record production declarations (produced quantities, scrap) and synchronise them with production orders in Business Central.
- Manage users by roles (Administrator, Supervisor, Operator) with secure token-based authentication.
- Offer supervisors a consolidated view of production progress across their work centres.
- Reduce manual data entry and the risk of errors associated with paper-based or non-integrated processes.
- Provide an extensible architecture, ready to accommodate new features (statistics, alerts, QR/label integrations).

1.3.7 Adopted Methodology

The development method corresponds to an ordered and structured sequence of project phases, for efficient and optimal management.

Selection of the Agile Methodology

Agile is an iterative and collaborative working method that takes into account the initial needs of the client and any changes that may arise. Its central ingredient is customer-centred iterative development, which allows a team to gather client feedback on a regular basis and make the necessary adjustments at the right time.

Thus, the client has a better overview of work progress, leading to regular sessions that guarantee the delivery of a functional application at all times throughout the improvement cycles. Consequently, the underlying key idea is to work faster in order to satisfy a client.

Adoption of the Scrum Framework

We adopted the Scrum framework, an agile approach based on short sprints and regular ceremonies. This choice allowed us to deliver functional increments at the end of each sprint, while maintaining smooth communication with the professional and our academic supervisors.

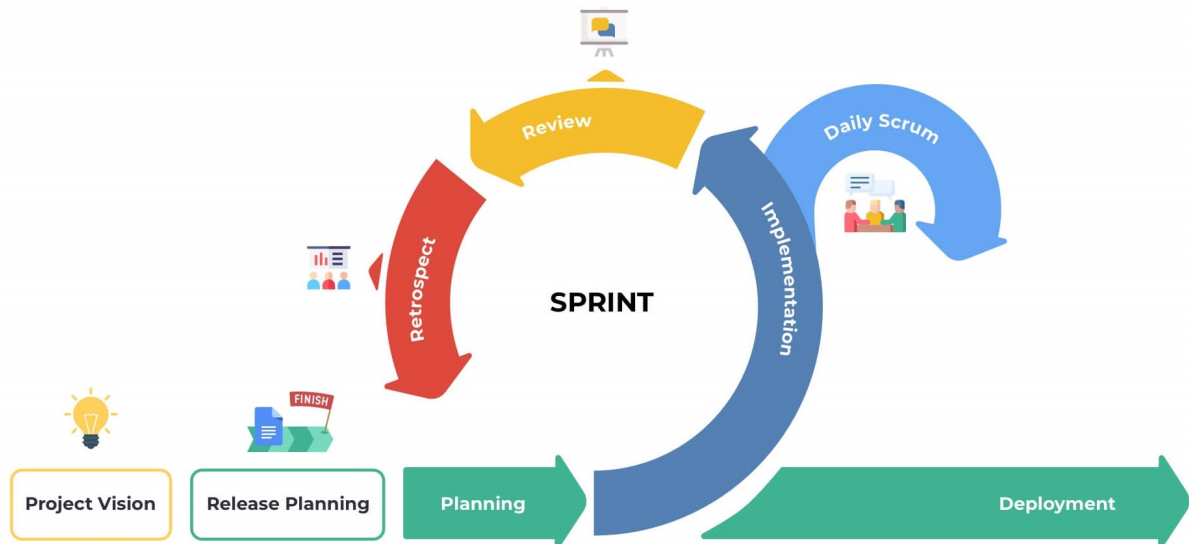


Figure 1.4. Adopted Scrum development cycle

Conclusion

In this chapter, we presented Mavision, the company where the project is being carried out. We then defined the subject context, as well as the objectives of our work and the chosen methodology. In order to ensure the report progresses logically, we will examine the existing solutions used in the company. We will then explain the functional and non-functional requirements of our solution. This will allow us to evaluate existing solutions and identify the weaknesses and strengths for which we should make improvements to our solution. Finally, we will formulate the specific requirements for our solution.

Chapter 2

Requirements Analysis and Specification

Contents

Introduction	10
2.1 Requirements Analysis	10
2.1.1 Identification of Actors	10
2.1.2 Identification of Requirements	10
Functional Requirements	10
Non-Functional Requirements	12
2.2 Project Management with Scrum	12
2.2.1 Backlog Features	12
Domain 1: Authentication & User Management	12
Domain 2: Machine Monitoring	14
Domain 3: Production Execution & Traceability	16
Domain 4: Administration & Monitoring	19
2.3 Sprint Plan	20
2.4 Global Application Architecture	21
2.4.1 Physical Architecture	21
2.4.2 Software Architecture	22
AL Back-end (Business Central)	22
Flutter Front-end	22
2.5 Work Environment	23
2.5.1 Software Environment	23
2.5.2 Development Environment	24
Conclusion	24

Introduction

Requirements analysis and specification is a crucial phase in the information system development process. It consists, first and foremost, of clarifying the functional and non-functional requirements of the system. This step then allows a complete understanding of the subject in order to position oneself clearly, precisely and without ambiguity.

2.1 Requirements Analysis

This step allows us to understand the overall context of our project, that is to say to identify the main features and determine the actors who will interact with the system.

2.1.1 Identification of Actors

Actors are entities external to the system (people, hardware, systems) that are likely to exchange information with it. The following is a list of the actors who will interact with our system:

Table 2.1. List of MES application users

Actor	Description
Administrator	This is the person responsible for the MES platform. They manage all users (supervisors, operators), passwords and account statuses. They have access to all application features as well as the administration pages in Business Central.
Supervisor	This is an intermediate user. They can monitor the progress of production operations on their work centre, view machine status and track the progress of production orders.
Operator	This is the end user of the MES application. Assigned to a work centre (<i>Work Center</i>), they can view the production orders assigned to their machine, start, pause or resume an operation, and declare produced quantities.

2.1.2 Identification of Requirements

At this stage, we will theoretically define the specific requirements of our application, clearly distinguishing functional requirements from non-functional requirements.

Functional Requirements

Each actor of the platform has specific expectations referred to as functional requirements. To define the functional scope, it is necessary that the conceptual solution meets these functional requirements.

- **Authentication:** This feature allows any user to log in to the platform using their Auth ID and password. Upon first login, the user is prompted to change their temporary password.

A secure session token (GUID, 12 h validity) is issued upon each successful login.

- **User Management:** This feature allows the administrator to:
 - View the complete list of MES users.
 - Create a new operator or supervisor account by linking it to a Business Central employee and assigning them a work centre.
 - Generate and assign a temporary password.
 - Activate or deactivate an account (automatic revocation of active tokens).
- **Machine Management:** This feature allows users to:
 - View the list of machines in a work centre with their current status (Idle, Starting, OutOfOrder).
 - View the production order currently running on each machine.
- **Production Order Management:** This feature allows the operator to:
 - View production orders (Planned, Firm, Released) assigned to their machine.
 - Filter and sort orders by status or date.
 - View the details of an order (item, quantity, planned dates, operation description).
- **Production Operation Management:** This feature is the core of the system. It allows the operator to:
 - Start an operation (validation of prerequisites: released order, free machine, previous operation completed if applicable).
 - Pause or resume an ongoing operation.
 - Declare produced quantities (validation: quantity > 0, no excess of the ordered quantity).
 - View progress in real time (produced quantity, completion percentage, operation status).
- **Progress Dashboard:** Allows the operator and supervisor to view all active operations (Running / Paused) on a given machine, with their respective progress.
- **Password Change:** Allows any logged-in user to change their password by entering their old password and the new one.

Non-Functional Requirements

To define the non-functional scope, it is necessary that the conceptual solution meets these non-functional requirements:

- **Performance:** The platform performs efficiently in terms of handling routine operations (login, order consultation, production declaration). Machine status updates are refreshed every 5 seconds via a streaming mechanism.
- **Security:** Implementation of a token-based authentication system (signed GUID, 12 h expiry) and role management (Admin, Supervisor, Operator) to control access to features. Passwords are stored as SHA-256 hashes with a salt.
- **Maintainability:** The AL back-end code is organised in layers (Tables, Enums, Codeunits, API Pages) and the Flutter code follows a layered architecture (Data, Domain, Presentation) with the Provider pattern, facilitating updates and the addition of new features.
- **Ergonomics:** The user interface is adaptive (mobile, tablet, PC) thanks to responsive layouts dedicated to each form factor, minimising the learning curve for workshop operators.
- **Portability:** The Flutter application is compiled for Android, iOS, Web, Linux, macOS and Windows from a single codebase. The Business Central back-end is deployable both on-premises and in SaaS (cloud) mode.

2.2 Project Management with Scrum

2.2.1 Backlog Features

Domain 1: Authentication & User Management

ID	User Story	Acceptance Criteria	Priority
US-1.1	As an Administrator, I need to assign access to each user so that users only access features appropriate to their role.	• Ability to assign a user as Operator or Supervisor or Administrator. • Unauthorized actions are blocked with clear error messages.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-1.2	As a User (Operator / Supervisor / Administrator), I need to log into the system with my credentials so that I can access features appropriate to my role.	• User can enter username and password. • System validates credentials against the ERP. • Session token is created upon successful authentication. • User is redirected to the appropriate dashboard based on role.	High
US-1.3	As an Administrator, I need to search users.	• Ability to filter users based on their role. • Ability to search users based on name or email.	High
US-1.4	As an Administrator, I need to see a list of all MES users with their details so that I can manage accounts.	• The user list displays each account's full name, email, role, and work centre. • Clicking a user row opens a password management dialog for that account. • The list refreshes automatically after a new user is created.	High
US-1.5	As an Administrator, I need to generate and assign a temporary password to a user so that they can log in for the first time.	• Admin can generate a random secure password for any user. • The generated password is sent to the ERP via AdminSetPassword. • The target account is flagged <i>Need To Change Password</i> after the password is set. • Success or failure is reported to the admin with a clear message.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-1.6	As an Administrator, I need to change the role and work-centre assignment of an existing user so that access rights stay current.	• Admin can select a new role for any user. • Work-centre list updates based on the selected role. • All active tokens for the target user are revoked on save. • The user list refreshes after a successful change.	High
US-1.7	As an Administrator, I need to activate or deactivate a user account so that access can be revoked without deleting the account.	• Admin can toggle the active status of any user. • Deactivation immediately revokes all active tokens. • An admin cannot deactivate their own account. • The user row updates its visual status indicator immediately.	High

Domain 2: Machine Monitoring

ID	User Story	Acceptance Criteria	Priority
US-2.1	As an Operator or Supervisor, I need to see the list of machines in my work centre with their current status so that I can see which machines are currently active.	• Machine list is scoped to the authenticated user's work centre. • Each card shows machine name, current status (<i>Idle</i>, <i>Starting</i>, <i>OutOfOrder</i>), and active production order number. • The list refreshes automatically every 5 seconds.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-2.2	As an Operator or Supervisor, I need to view the production orders assigned to a machine so that I know what work needs to be done.	- Only <i>Planned</i>, <i>Firm Planned</i>, or <i>Released</i> orders are shown. - Each card displays order number, item description, planned quantity, planned start and end dates, and operation number. - Orders can be searched and filtered by status, and sorted by planned start date.	High
US-2.3	As an Operator or Supervisor, I need to start a production operation so that the system records that the order has begun.	- Only <i>Released</i> routing lines may be started. - An operation cannot be started if it is already running. - A machine cannot run two operations simultaneously. - A new MES Operation Status record is created with status <i>Running</i>.	High
US-2.4	As a Supervisor or Operator, I need to monitor the current status of all active operations on a machine so that I can follow the progress of each order.	- The Operations Progress tab shows all operations whose latest status record is <i>Running</i> or <i>Paused</i>. - Each card shows order number, operation number, current status, and last updated timestamp. - Data refreshes automatically every 5 seconds.	High
US-2.5	As an Operator or Supervisor, I need a tabbed view for a machine so that I can navigate between pending orders and active operation progress without leaving the machine context.	- Selecting a machine from the list opens a dedicated machine page. - The page provides at least two tabs: <i>Orders</i> and <i>Progress</i>. - Switching tabs preserves the state of each tab without reloading data. - The active tab is visually highlighted in the navigation bar.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-2.6	As an Operator or Supervisor, I need to filter and sort production orders so that I can quickly find the most relevant work.	• Orders can be searched by order number, item description, or date. • Orders can be filtered by status (<i>All, Planned, Firm Planned, Released</i>). • Orders can be sorted by planned start date ascending or descending. • On mobile, the status filter is accessible via a compact popup menu.	High
US-2.7	As a Supervisor or Operator, I need to view the history of completed operations for a machine so that I can audit past production.	• The History tab shows all operations with status <i>Finished</i> or <i>Cancelled</i>. • Each card shows order number, status badge, product name, produced quantity, start time, and end time. • The list supports free-text search and sort by start date. • Tapping a card navigates to the full operation detail page.	High

Domain 3: Production Execution & Traceability

ID	User Story	Acceptance Criteria	Priority
US-3.1	As an Operator or Supervisor, I need to declare the quantity produced in a cycle so that the system records production progress.	• Declared quantity must be positive and not exceed remaining quantity. • Each declaration creates a new progression cycle record. • Cumulative quantity and progress percentage update immediately.	High
US-3.2	As an Operator or Supervisor, I need to pause and resume an active operation so that interruptions are tracked accurately.	• A Running operation can be paused; a Paused one can be resumed. • A machine cannot resume if another operation is already running.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-3.3	As a Supervisor, I need to finish or cancel an operation so that the machine is released and the order is closed.	- Finishing a complete order (100%) shows a green confirmation dialog. - Cancelling an incomplete order shows a red warning dialog. - Closing sets the execution end time and inserts an Idle machine status.	High
US-3.4	As an Operator or Supervisor, I need a dedicated operation detail page with live data in one place.	- Shows status, order number, product, required and produced quantities. - Live data streams without a manual refresh. - Adapts between mobile and desktop layouts.	High
US-3.5	As an Operator or Supervisor, I need to see all production cycles declared for an operation.	- Cycle table lists operator, quantity, cumulative total, and timestamp. - Records ordered newest-first with pagination. - Refreshes after each new declaration.	High
US-3.6	As an Operator or Supervisor, I need a visual chart of declarations over time to identify performance trends.	- Line chart plots cycle quantity vs. declaration time. - Touch tooltip shows operator, quantity, cumulative total, timestamp. - Auto-scrolls to most recent data point on load and refresh.	Medium
US-3.7	As an Operator or Supervisor, I need to see the Bill of Materials for an operation to know which components are required.	- Components linked to the current routing step are colour-coded by availability (Available / Low Stock / Missing). - Shared components not linked to this step are shown in neutral style. - BOM data refreshes after every consumption event.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-3.8	As an Operator or Supervisor, I need to report scrapped units with a reason code so that waste is accurately tracked.	- Scrap can only be declared when the operation is Running. - A reason code must be selected before submission. - Each declaration records quantity, code, and operator.	High
US-3.9	As a Supervisor, I need to declare production on behalf of an operator so that output is attributed to the correct person.	- Supervisor role sees an operator selector in the declare dialog. - Selected operator's ID is written to the Declared By field. - If no operator is selected, the supervisor's ID is used.	Medium
US-3.10	As an Operator or Supervisor, I need to scan components into an operation so that consumption is recorded against the BOM.	- Live camera feed reads DataMatrix barcodes. - Internal format parsed locally; external resolved via API. - Duplicate scan increments quantity on the existing row. - Confirming scans inserts consumption records and refreshes the BOM.	High
US-3.11	As an Operator or Supervisor, I need to print a label for a production order so that produced parts are correctly identified.	- Label includes DataMatrix barcode encoding order, machine, item data. - PDF generation supports landscape/portrait toggle. - Print button available only when operation is Finished or progress $\geq$ 100%.	Medium
US-3.12	As an Operator or Supervisor, I need a label printed for each declared unit so that individual pieces carry traceable information.	- One PDF page per declared unit with a label counter. - Same DataMatrix encoding as the production label. - Launched automatically after a successful production declaration.	Medium

Domain 4: Administration & Monitoring

ID	User Story	Acceptance Criteria	Priority
US-4.1	As an Administrator, I need a performance dashboard for all machines so that I can monitor production health across the facility.	• Dashboard shows uptime %, operation count, total produced, and scrap per machine. • Configurable time-range (hours back) filter. • Responsive grid: 1 col mobile, 2 col tablet, 3 col desktop.	High
US-4.2	As an Administrator, I need a filterable, paginated activity log of all system actions so that I can audit operator and machine activity.	• Log entries include type, operator, machine, action, and timestamp. • Filterable by action type and time range. • Paginated at 15 rows per page.	High
US-4.3	As an Administrator, I need a persistent sidebar navigation so that I can move between admin sections without losing context.	• Sidebar has sections for Users & Roles, Machine Dashboard, Activity Logs, and Settings. • All pages kept alive simultaneously via IndexedStack. • Active section is visually highlighted.	High
US-4.4	As any user, I need the application available in English, French, and Arabic so that I can use it in my preferred language.	• Language selector on login screen and in profile page. • All UI strings externalised; no hard-coded text. • Arabic locale activates RTL layout automatically. • Language preference persisted across sessions.	High

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-4.5	As a first-time user, I need guided coach-mark tutorials so that I can learn the interface without external documentation.	- Tutorials shown exactly once per install per screen. - Coach marks cover: machine list, machine detail tabs, operation detail, and admin dashboard. - Localised skip button text.	Medium

Remaining

ID	User Story	Acceptance Criteria	Priority
US-x.1	As a Supervisor, I need to create a new production order so that I can respond to urgent production needs.	- Form captures: Machine, ProductNo, Planned Quantity, Start Time - Validates product exists in ERP - Validates machine can produce the product - Order is saved to ERP database - Order appears in machine's "Orders to be done" within a short time frame - Audit trail captures order creation	Medium
US-x.2	As an Operator, I need to be able to declare production on another machine of the same type so that I can continue working when my terminal is broken.	- Operator can access machines of same type as their assigned machines - System validates operator is authorized for that machine type - Declaration process identical to US-4.1 - Audit trail shows which machine declaration was made from	Low

2.3 Sprint Plan

The project was organised into four two-week sprints, each delivering a functional increment verified by a sprint review with the professional supervisor.

Table 2.7. MES project sprint plan

Sprint	Domain	Scope
Sprint 1	Authentication & User Management	Secure login, session token management, role-based access control, administrator user creation, password generation and reset, role and work-centre change, account activation/deactivation.
Sprint 2	Machine Monitoring & Production Order Management	Real-time machine status tracking, production order list per machine, operation start, tabbed machine view with Orders / Progress / History tabs, order filtering and sorting, completed operation history.
Sprint 3	Production Execution & Traceability	Production cycle declarations, pause/resume/finish/cancel operations, operation detail page with live data, production chart, Bill of Materials visibility, component barcode scanning, scrap declaration, supervisor proxy declaration, and label printing.
Sprint 4	Administration, System Configuration	Administrator dashboard and activity log, admin navigation shell, multi-language support (EN/FR/AR) and RTL layout, in-app onboarding tutorials.

Continued on next page

2.4 Global Application Architecture

2.4.1 Physical Architecture

The physical architecture of the MES system rests on two main nodes:

- **Business Central Server (on-premises / SaaS):** hosts the AL extension that exposes OData V4 endpoints and the business REST APIs. In on-premises mode, the server runs on a BC210 instance accessible on port 7048. In SaaS mode, the base URLs follow the schema `api.businesscentral.dynamics.com/v2.0/<tenantId>/<environment>/ODataV4/`.
- **Flutter Client (mobile / web / desktop):** application compiled for Android, iOS, Web or desktop, communicating with the BC server via HTTP POST requests to OData V4 endpoints and custom APIs.

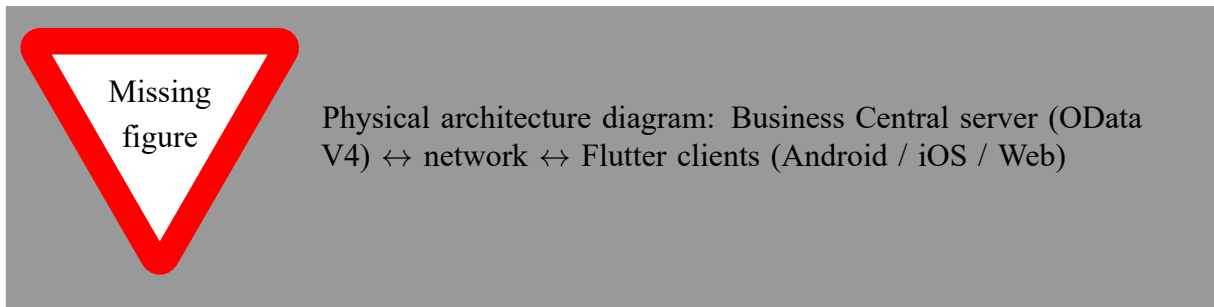


Figure 2.1. MES System Physical Architecture

2.4.2 Software Architecture

The software architecture of the system is organised around two distinct parts.

AL Back-end (Business Central)

The back-end is structured in six layers:

- **Tables (1-Tables):** define the persistent data model — MES User, MES Auth Token, MES Machine Status, MES Operation Status, MES Operation Progression.
- **Enums (2-Enum):** define the enumerated types — MES User Role (Operator, Supervisor, Admin), MES Machine Status (Idle, Starting, OutOfOrder), MES Operation Status (Running, Paused, Finished).
- **Codeunits (3-CodeUnit):** contain the business logic — MES Auth Mgt (authentication, tokens), MES Password Mgt (SHA-256 hashing), MES Machine Actions (machine and operation management), MES Unbound Actions (OData V4 facade).
- **API Pages (4-API):** expose data for reading/writing via the BC REST API — employees, MES users, work centres.
- **Permission Sets (5-PermissionSet):** define access rights to AL objects for BC users.
- **Pages (6-Pages):** administration and debugging interfaces in the BC client (lists, cards, API debug page).

Flutter Front-end

The Flutter front-end follows a three-layer architecture:

- **Data:** contains data models (models) and HTTP services (services) that communicate with BC endpoints via the `http` library.
- **Domain:** contains the providers (Provider pattern) that expose the application state to widgets — `AuthProvider`, `MesUserProvider`, `MesMachinesProvider`, `MachineordersProvider`, etc.

- **Presentation:** contains the pages and widgets of the user interface, organised by functional domain (auth, admin, machine) with dedicated responsive layouts (mobile, tablet, PC).

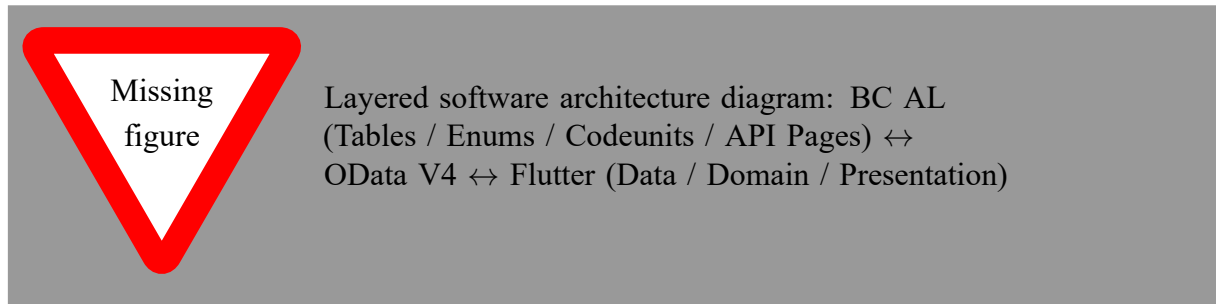


Figure 2.2. MES System Software Architecture

2.5 Work Environment

2.5.1 Software Environment

Table 2.8 summarises the main tools and technologies used in this project.

Table 2.8. MES project software environment

Logo	Tool / Technology	Role in the project
	Microsoft Dynamics 365 Business Central	ERP platform hosting the AL back-end and business data
	AL Language (Application Language)	Development of the BC extension (tables, codeunits, API pages)
	Flutter (Dart)	Development of the cross-platform mobile/web application
	Visual Studio Code	Primary development environment (AL and Flutter)
	Git / GitHub	Version control and collaboration
	Postman	Testing and validation of OData V4 endpoints

Continued on next page

Logo	Tool / Technology	Role in the project
	lucid chart	Testing and validation of OData V4 endpoints

2.5.2 Development Environment

[To be completed: include workstation specifications (OS, RAM, CPU), tool versions (Flutter SDK, Dart SDK, Business Central version, VS Code extensions), and any specific configuration details such as Docker setup or BC on-premises instance parameters.]

Conclusion

In this chapter, we analysed and specified the functional and non-functional requirements of the MES system, identified the actors and their interactions, and presented the overall architecture of the solution. We also described the work environment and the adopted Scrum methodology. The following chapters detail the work carried out sprint by sprint, presenting the conceptual aspects and the results obtained at the end of each iteration.

Chapter 3

Sprint 1 Authentication & User Management

Contents

Introduction	26
3.1 Sprint Backlog	26
3.2 Design	30
3.2.1 Use Case Diagram	30
3.2.2 Textual Use Case Description	31
3.2.3 Sequence Diagrams	32
3.2.4 Class Diagram	39
3.3 Implementation	40
3.3.1 Backend Implementation	40
Backend Structure Overview	40
MES User Table Implementation	41
Authentication Token Table	42
Password Management and Encryption	43
Authentication Logic — Login, Logout, and Change Password	43
3.3.2 REST API and Web Services Implementation	44
API Pages for MES User Management	44
OData Functions and Web Service Configuration	44
3.3.3 Frontend Implementation	44
Service Layer (API Communication)	45
State Management Layer using Provider	45
User Interface	46
3.4 Testing and Validation	49
3.4.1 Integration testing	49
Conclusion	50

Introduction

This sprint focuses on delivering a complete authentication and user management module for the *MES* application. The primary goal is to develop a secure authentication system covering *login*, *logout*, and password management, alongside role-based page access control to ensure each user can only reach features appropriate to their role.

In addition, the Administrator is given the ability to add new MES users, generate secure passwords on their behalf, change a user's role and work-centre assignment, and activate or deactivate accounts. By the end of this sprint, a fully functional *login* screen, *change password* flow, *admin user management dashboard*, *role management dialog*, and *logout* functionality will all be in place.

3.1 Sprint Backlog

This sprint covers **Domain 1: Authentication & User Management**.

Table 3.1. Story Backlog

ID	User Story	Tasks
US-1.1	As an Administrator, I need to assign access to each user so that users only access features appropriate to their role.	<i>Business Central (AL)</i> • Implement the functions <code>AdminCreateUser()</code>, and <code>AdminSetActive()</code> in codeunit MES Unbound Actions. • Expose <code>AdminCreateUser()</code>, and <code>AdminSetActive()</code> through codeunit MES Web Service. • Create API page MES User Create API. <i>Flutter</i> • Update <code>ApiService</code> to consume the new endpoints. • Implement <code>MesUserService</code> to consume <code>fetchMesUsers()</code> and <code>createMesUser()</code> endpoints. • Create provider <code>MesUserProvider</code> to manage its state. • Create widget <code>AddUserDialog</code>. • Implement role-based navigation in <code>_AuthGate</code>.

Continued on next page

ID	User Story	Tasks
US-1.2	As a User (Operator / Supervisor / Administrator), I need to log into the system with my credentials so that I can access features appropriate to my role.	<i>Business Central (AL)</i> • Create the tables MES USER and MES AuthToken. • Create enum MES UserRole. • Implement functions MakeSalt(), HashPassword(), and VerifyPassword() in codeunit MES Password Mgt. • Implement functions CreateUser(), SetPassword(), ValidateCredentials(), IssueNewToken(), ValidateToken(), TouchToken(), Logout(), ValidateChangePassword(), ValidateAdminToken(), SetActive(), and CleanupExpiredTokens() in codeunit MESAuthMgt. • Implement functions Login(), Logout(), Me(), and ChangePassword() in codeunit MES Unbound Actions. • Expose Login(), Logout(), Me(), and ChangePassword() in codeunit MES Web Service. <i>Flutter</i> • Implement ApiService to consume the new endpoints. • Create provider AuthProvider to manage its state. • Implement authentication routing in _AuthGate. • Create page LoginPage. • Create AppConstants.
US-1.3	As an Administrator, I need to search users.	<i>Flutter</i> • Implement user search in AddUserPage. • Implement role filtering in AddUserPage. • Combine role and search filters in AddUserPage.

Continued on next page

ID	User Story	Tasks
US-1.4	As an Administrator, I need to see a list of all MES users with their details so that I can manage accounts.	<i>Business Central (AL)</i> • Create the function <code>fetchAllMESUsers</code> in <code>codeunit MES Unbound Actions</code>. • Expose <code>fetchAllMESUsers</code> through <code>codeunit MES Web Service</code>. <i>Flutter</i> • Create model <code>MesUser</code>. • Implement <code>MesUserService</code> to consume the new end point. • Implement <code>MesUserProvider</code> to manage its state and its streams. • Implement user list display in <code>AddUserPage</code>. • Create the widgets <code>GeneratePasswordDialog</code> and <code>EmployeeAvatar</code>.
US-1.5	As an Administrator, I need to generate and assign a temporary password to a user so that they can log in for the first time.	<i>Business Central (AL)</i> • Implement function <code>AdminSetPassword()</code> in <code>codeunit MES Unbound Actions</code>. • Expose <code>AdminSetPassword()</code> through <code>codeunit MES Web Service</code>. <i>Flutter</i> • Create widget <code>GeneratePasswordDialog</code>. • Update <code>ApiService</code> to consume <code>adminSetPassword()</code>. • Update <code>AuthProvider</code> to account for the new functions in the service.
US-1.6	As an Administrator, I need to change the role and work-centre assignment of an existing user so that access rights stay current.	<i>Business Central (AL)</i> • Implement function <code>AdminChangeUserRole()</code> in <code>codeunit MES Unbound Actions</code>. • Expose <code>AdminChangeUserRole()</code> through <code>codeunit MES Web Service</code>.

Continued on next page

ID	User Story	Tasks
		<i>Flutter</i> • Update MesUserService to consume <code>changeUserRole()</code>. • Update MesUserProvider to account <code>changeUserRole()</code>. • Create widget <code>ChangeRoleDialog</code>.
US-1.7	As an Administrator, I need to activate or deactivate a user account so that access can be revoked without deleting the account.	<i>Flutter</i> • Update ApiService to account for <code>toggleUserActiveStatus()</code>. • Update AuthProvider to account for <code>toggleUserActiveStatus()</code>. • Integrate activate and deactivate actions in <code>AddUserPage</code>.

Table 3.2. Business Rules

ID: Business Rule	Description
Authentication Rules	
BR-AUTH-001 Credential Validation	• Credentials must be validated against the ERP database. • Generic validation failure to prevent user enumeration.
BR-AUTH-002 Session Management	• Each session generates a unique token (<i>GUID</i>). • Token lifetime: 12 hours. • Token expires automatically after this period. • Only one active token per device.
BR-AUTH-003 Password Security	• Minimum length: 8 characters. • Must contain uppercase, lowercase, number, and special character. • Encryption using <i>PBKDF2-SHA256</i>. • Unique salt per user (randomly generated).
User Administration Rules	

Continued on next page

ID: Business Rule	Description
BR-ADMIN-001 Role-Work-Centre Constraint	• An Operator or Supervisor must be assigned to at least one work centre. • An Administrator must have no work-centre assignment. • These constraints are enforced both in <code>AdminCreateUser()</code> and <code>AdminChangeUserRole()</code>.
BR-ADMIN-002 Role Change Revokes Active Tokens	• <code>AdminChangeUserRole()</code> calls <code>RevokeAllTokensForUser()</code> after updating the role and work-centre fields, forcing the affected user to re-authenticate and receive a new session reflecting their updated permissions. • The same revocation logic applies when an account is deactivated via <code>AdminSetActive()</code>.

3.2 Design

3.2.1 Use Case Diagram

The UML Use Case Diagram (Figure 3.1) shows the three primary actors (Operator, Supervisor, Administrator) and their interactions with the *MES* authentication and user management system. In addition to the core login, logout, change password, and user creation flows, the diagram includes the *Change Role / Work Centre* and *Activate / Deactivate Account* use cases available exclusively to the Administrator.

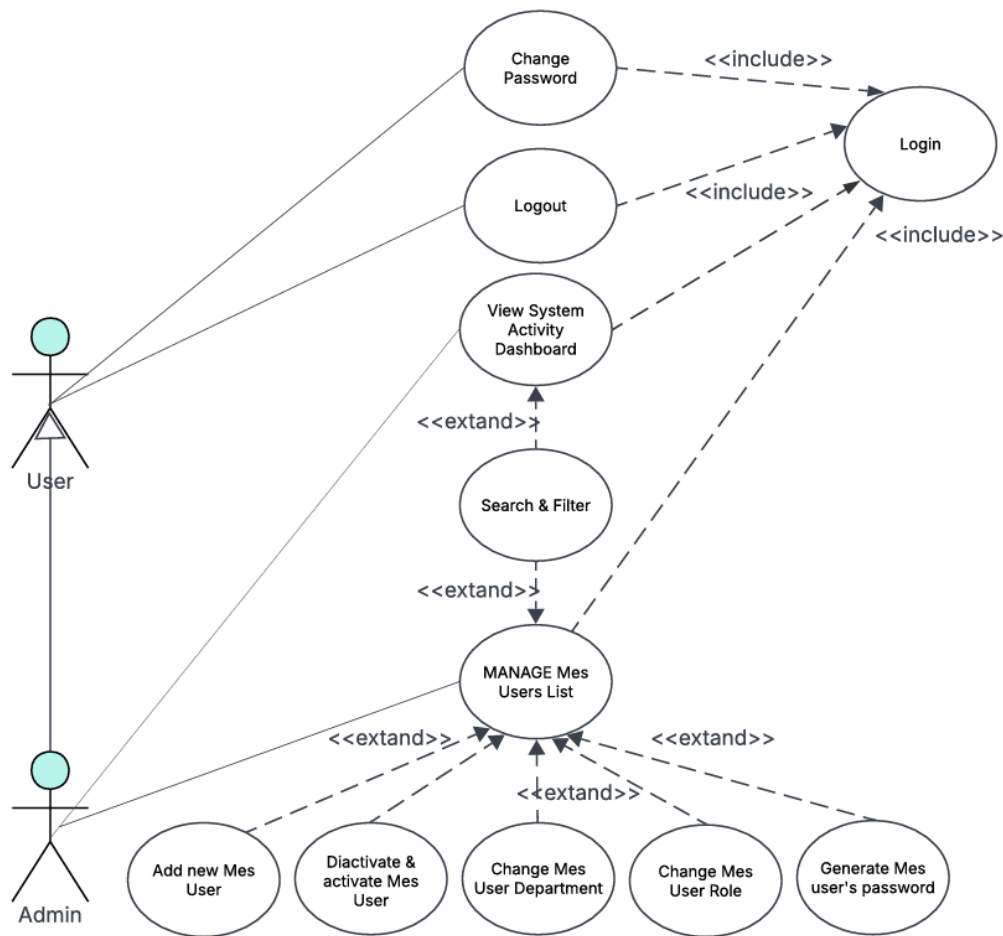


Figure 3.1. UML Use Case Diagram.

3.2.2 Textual Use Case Description

Use Case Name	Login
Primary Actors	Operator, Supervisor, Administrator
Preconditions	• The user account exists in the ERP system. • The user has an assigned role and department. • The account must have a password (there is a possibility that there is an account without a password). • The user account is active.

Continued on next page

Postconditions

- If the credentials are valid, the user is successfully authenticated.
 - A session token is generated and stored securely.
 - The user is redirected to the appropriate dashboard based on their role (Admin or MES interface).
 - If the *need change password* flag is set to true, they are redirected to the *Change Password* page.
 - If the credentials are invalid, access is denied and an error message is displayed.
-

Main Scenario

1. The user launches the application.
 2. The system displays the *login screen*.
 3. The user enters *User ID* and *password*.
 4. The system automatically detects the device ID.
 5. The user clicks the *Login* button.
 6. The system validates the credentials against the ERP database.
 7. The system generates a secure session token.
 8. The system checks the user role.
 9. The system redirects the user to the appropriate dashboard (Admin or MES).
-

Table 3.3. Textual Use Case Description — Login

3.2.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for each authentication flow.

Figure 3.2 depicts the login flow, in which the user submits credentials, the AL layer validates them against the Business Central user store, and the application either returns a structured error or issues a session token and redirects the user to the appropriate page based on their role and password-change flag.

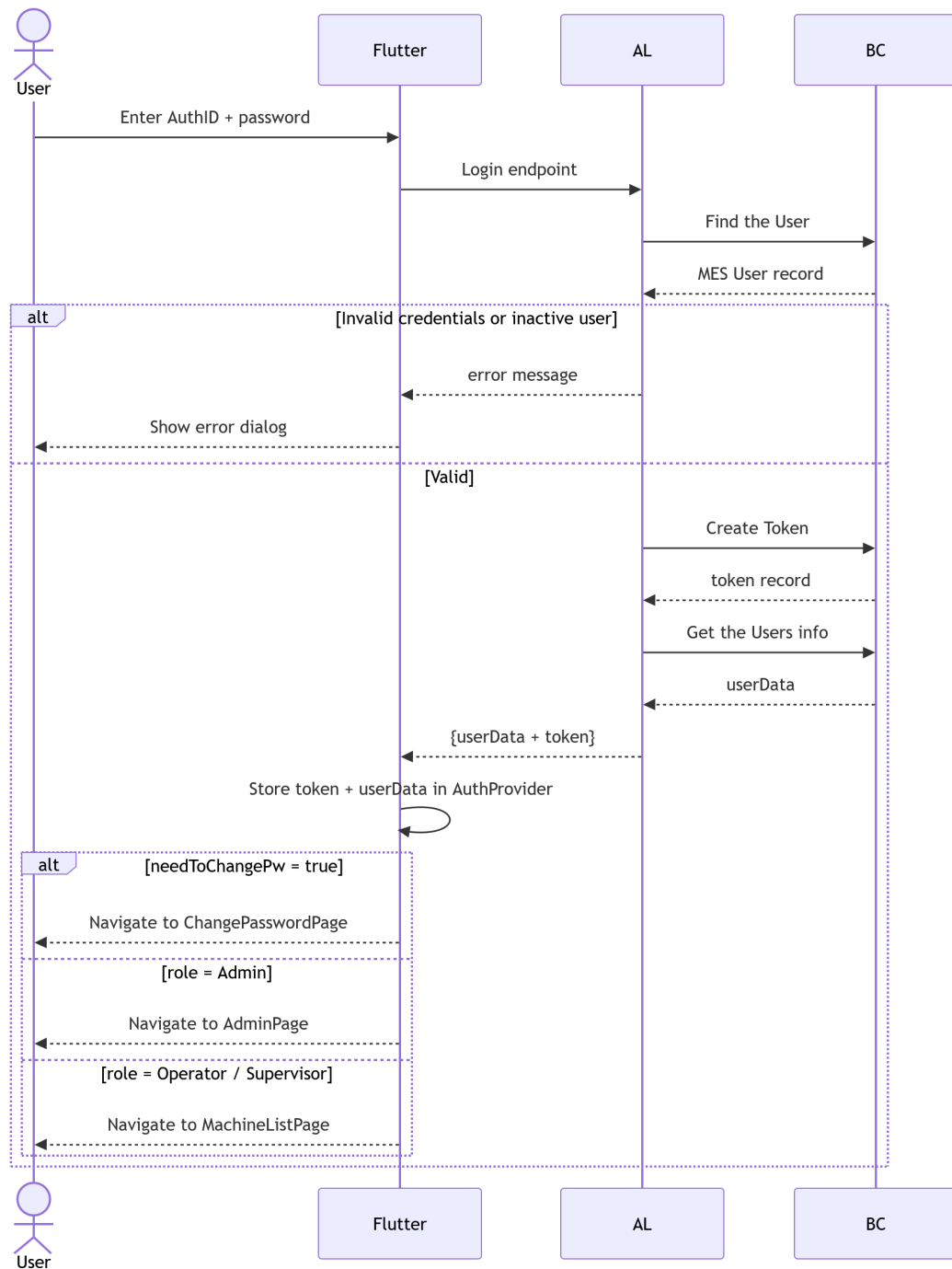


Figure 3.2. SD-01: Login.

Figure 3.3 presents the token validation flow, which is invoked by the AL backend before processing any authenticated request. The backend retrieves the token record from Business Central, verifies its validity and expiry, confirms that the associated user account is still active, and either updates the LastSeenAt timestamp or returns an error that forces the Flutter client to redirect the user to the login page.

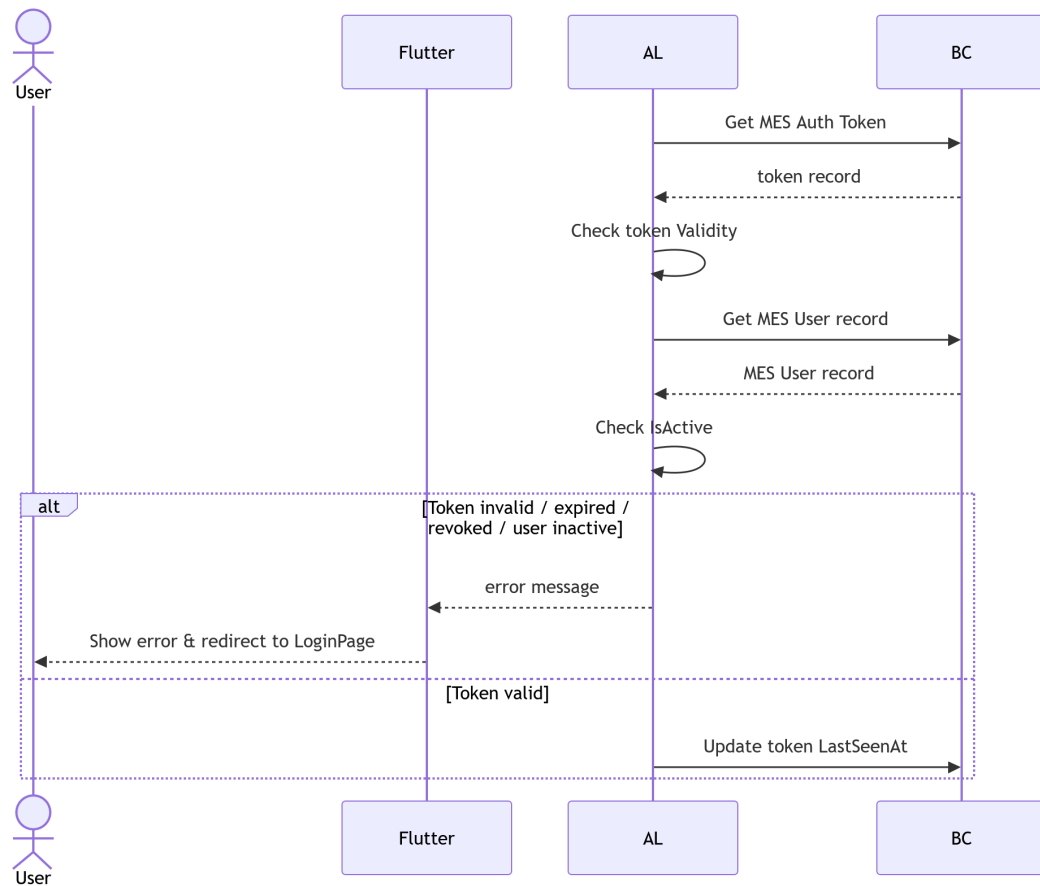


Figure 3.3. SD-02: Token Validation.

Figure 3.4 illustrates the logout flow, in which the Flutter client sends a logout request, the AL layer retrieves and revokes the current session token in Business Central, and the client clears its local AuthProvider state before navigating back to the login page.

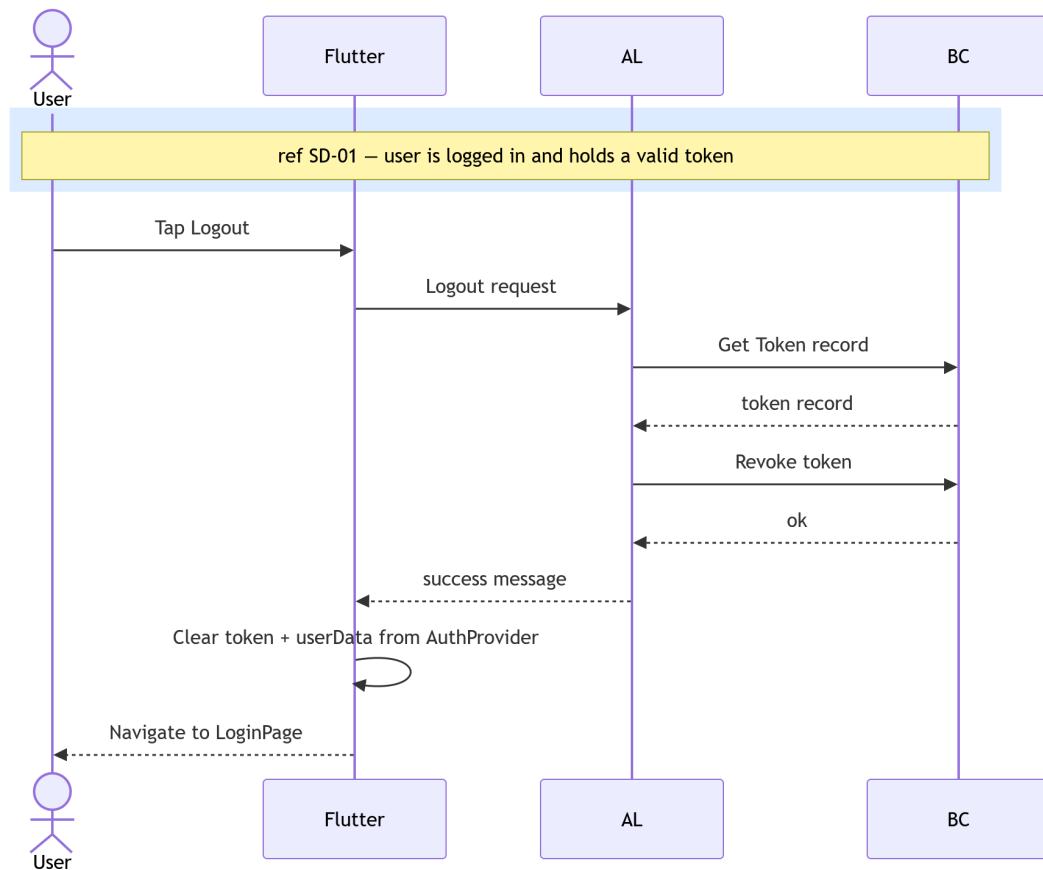
**Figure 3.4.** SD-03: Logout

Figure 3.5 describes the change-password flow. The Flutter client first validates locally that the two password fields match and satisfy the strength rules, then forwards the request to the AL layer, which re-validates the token (SD-02), verifies the current password hash, checks the new password strength, and — on success — updates the credential record in Business Central and revokes all existing tokens for the account.

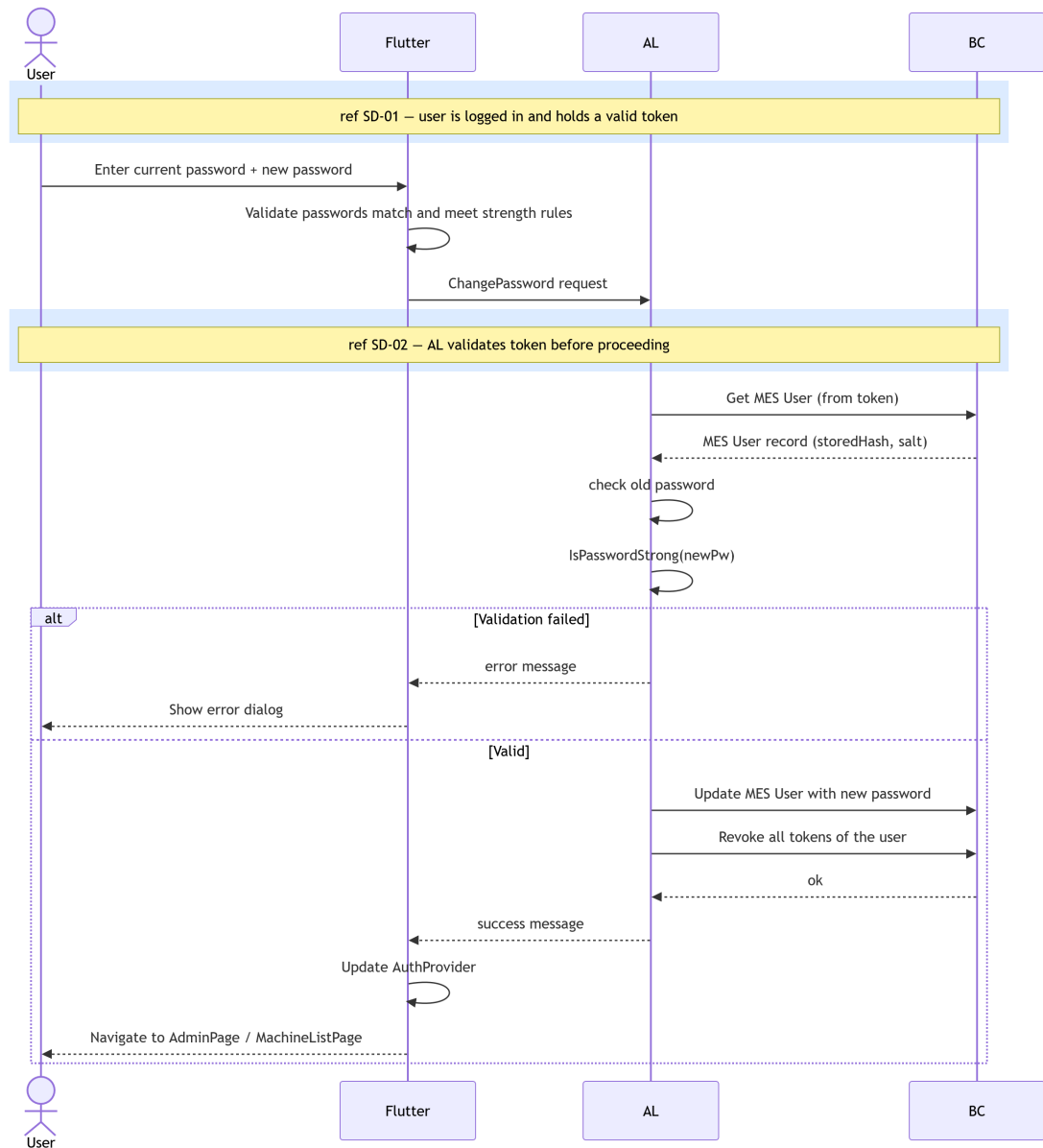


Figure 3.5. SD-03: Change Password

Figures 3.6 through 3.9 cover the four administrator user-management operations. All four follow the same pattern: the Flutter client sends an admin-scoped request, the AL layer validates the token and confirms that the caller holds the Admin role (SD-02), executes the requested Business Central write operations, and triggers a reactive refresh so that the updated user list is reflected immediately in the UI.

Figure 3.6 shows the user creation flow, in which the AL layer inserts a new *MES User* record with *IsActive* and *NeedToChangePw* set to true, then iterates over the supplied work-centre list to create the corresponding assignment records.

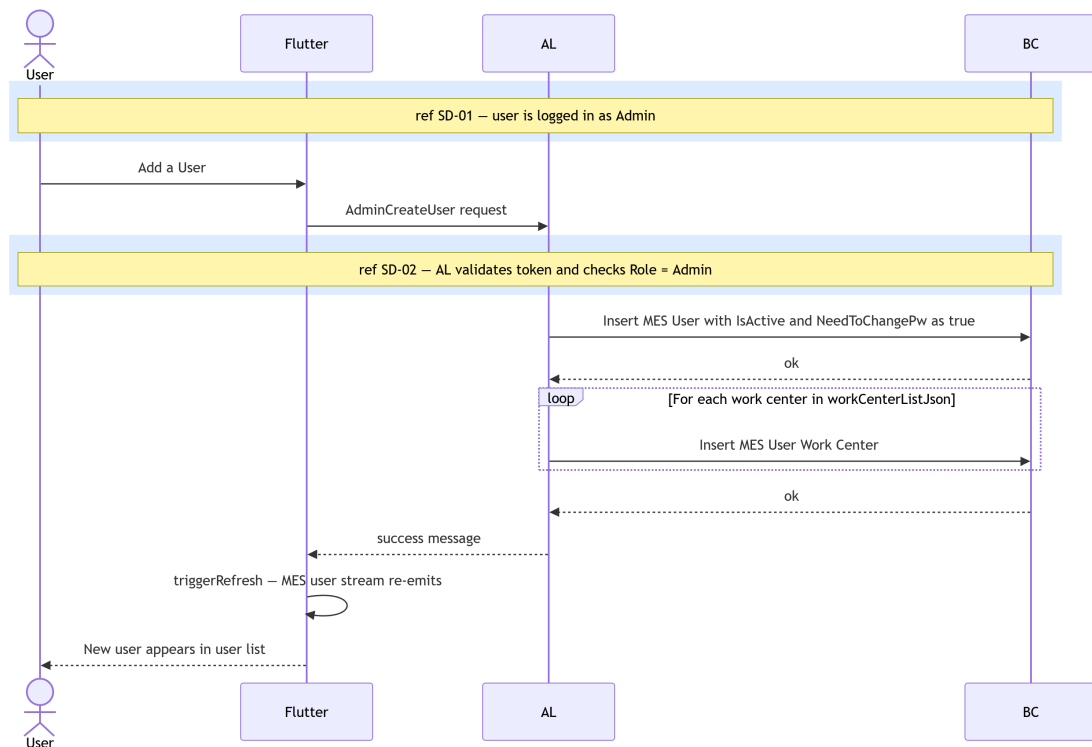


Figure 3.6. SD-05: Admin - Create User.

Figure 3.7 depicts the admin set-password flow. The administrator generates a temporary credential, the AL layer marks the target account as requiring a password change on next login, revokes all active tokens for that account, and returns the new credential to the Flutter client as copyable text.

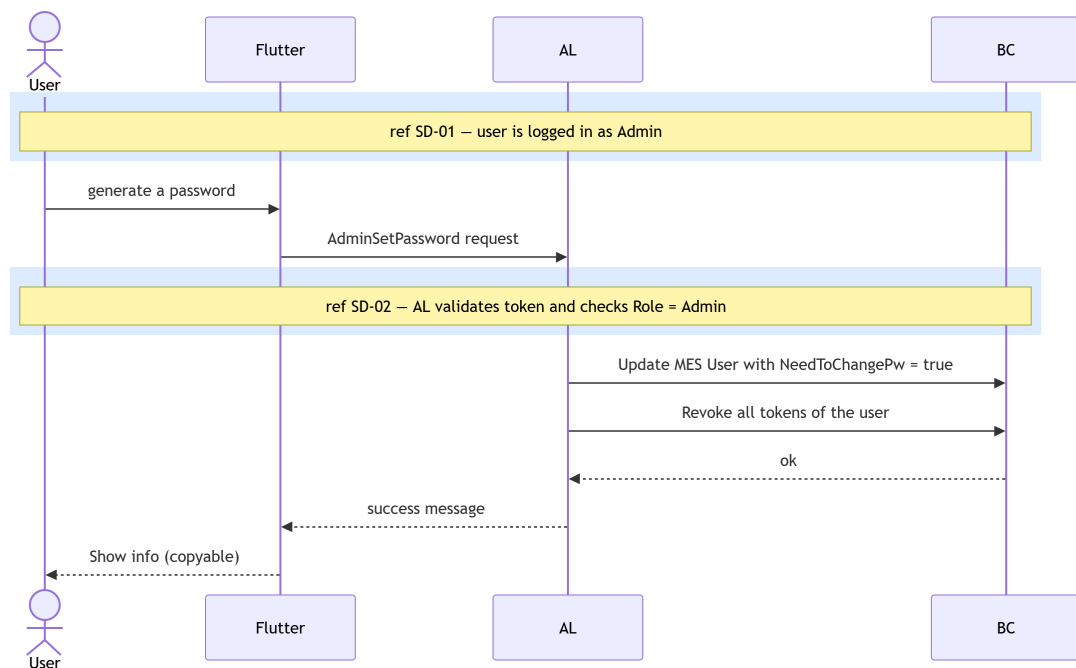


Figure 3.7. SD-06: Admin - Set Password.

Figure 3.7 depicts the admin set-password flow. The administrator generates a temporary credential, the AL layer marks the target account as requiring a password change on next login, revokes all active tokens for that account, and returns the new credential to the Flutter client as copyable text.

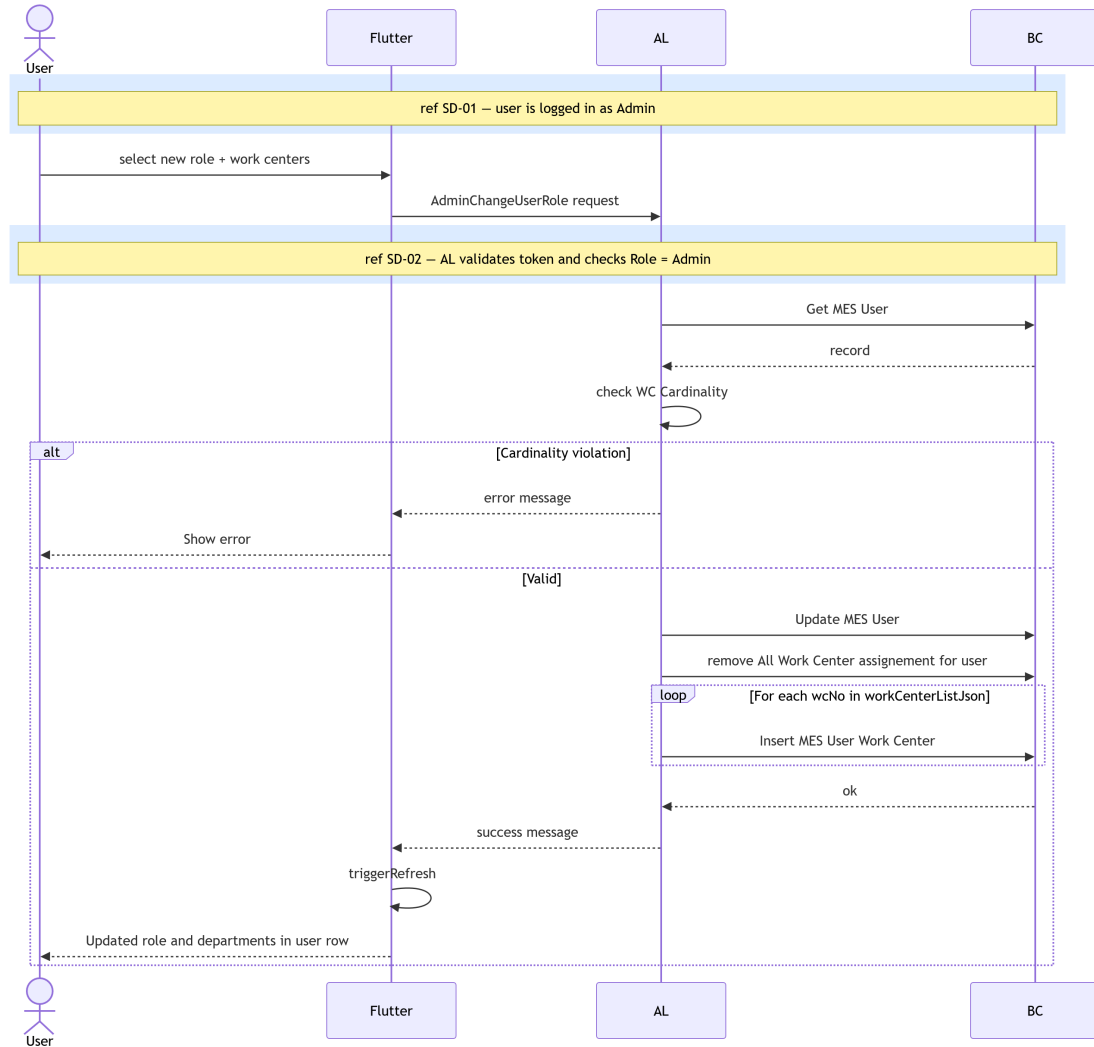


Figure 3.8. SD-07: Admin - Change Role / Work Centre.

Figure 3.9 presents the account activation and deactivation flow. When deactivating an account, the AL layer additionally revokes all active session tokens for that user, ensuring that any currently authenticated sessions are immediately invalidated.

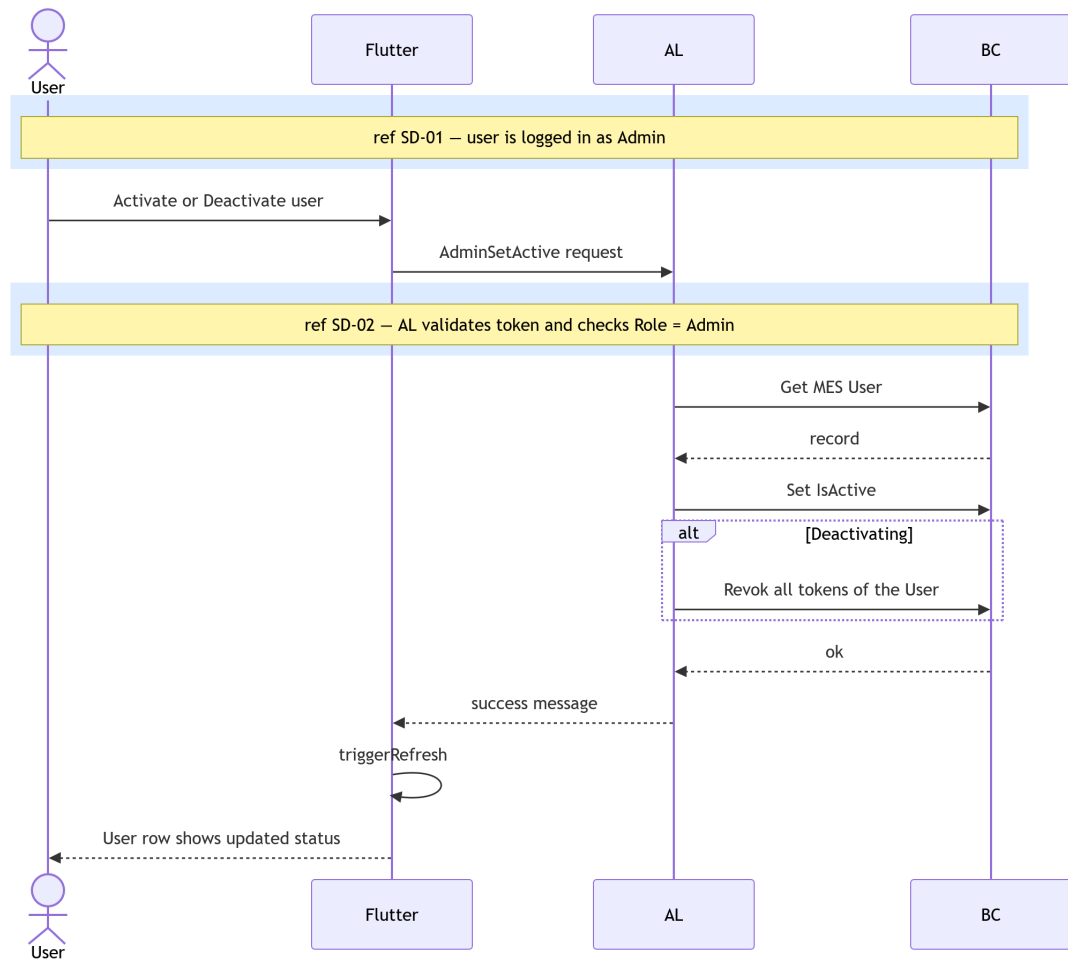


Figure 3.9. SD-08: Admin - Toggle Account Active Status.

3.2.4 Class Diagram

The UML Class Diagram of the *MES* authentication domain, shown in Figure 3.10, depicts the *MesUser*, *AuthToken*, and *PasswordPolicy* classes with their attributes and associations.

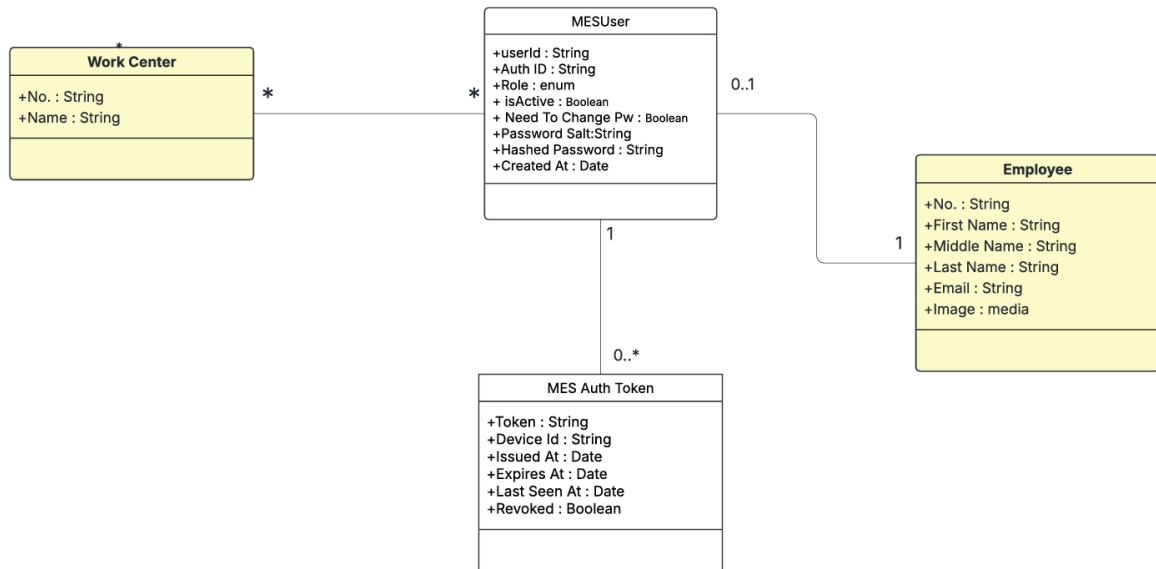


Figure 3.10. UML Class Diagram.

3.3 Implementation

3.3.1 Backend Implementation

Backend Structure Overview

The backend project structure is illustrated in Figure 3.11.

```

AL-backend/
  src/
    1-Tables/
      NEW/
        MES_AuthToken.al
        MES_USER.al
    2-Enum/
      NEW/
        MES_UserRole.al
    3-CodeUnit/
      NEW/
        MESAAuthMgt.al
        MESPASSWORDMgt.al
        MESSetup.al
    4-API/
      NEW/
        MESAAuthActions.al
        MESAAuthApi.al
    5-PermissionSet/
      NEW/
        MESAAuthApi.permissionset.al
    6-Pages/
      NEW/
        MESApiDebug.Page.al
        MESUserCard.Page.al
        MESUserList.Page.al

```

Figure 3.11. Backend project structure.

MES User Table Implementation

The MES User table is implemented to store user information, as described in Table 3.4.

Name	Type	Description
User Id	Code[50]	Primary key. Login identifier used at the MES login prompt (e.g., JD0E, 0P001). If blank on insert, a GUID-derived value is assigned automatically.
Employee ID	Code[50]	Foreign key to Employee."No.". Required for every MES account and enforced as unique (one Employee maps to one MES User).
Auth ID	Text[100]	Auto-generated external identifier in the form AUTH-XXXXXXX. Used for display/external references (not used for login). Uniqueness enforced.
Role	Enum (MES User Role)	Authorization role for the MES application (e.g., Operator / Supervisor / Admin). Used to control permissions and access.

Continued on next page

Name	Type	Description
Work Center No.	Code[20]	Foreign key to Work Center. "No.". Work center assignment: required for Operator and Supervisor; must be blank for Admin.
Is Active	Boolean	Account enable/disable flag. If false, authentication is blocked and existing tokens are revoked when deactivated.
Need To Change Pw	Boolean	If true, the user must change their password before using the MES application (set at creation and on forced password resets).
Password Salt	Text[64]	Random hex salt used in password hashing so identical passwords produce different hashes across users.
Hashed Password	Text[128]	Hashed password value (hex string). Passwords are never stored in plaintext; hashing uses the password plus salt.
Created At	DateTime	UTC timestamp set on insert only (audit trail/reporting).

Table 3.4. MES User (Table 50101) — Field Dictionary**Authentication Token Table**

The MES Auth Token table stores session information as described in Table 3.5.

Name	Type	Description
Token	Guid	Primary key. Raw GUID token presented by the client as a Bearer credential. Generated at issue time (never reused). Used for all direct token lookups (validation/logout).
User Id	Code[50]	Foreign key to MES User. "User Id". Identifies the MES user who owns this session/token. Enforces referential integrity via TableRelation.
Device Id	Text[100]	Optional identifier for the device that requested the token (e.g., tablet-floor-02, scanner-01). Stored for traceability/audit only; not used for validation.
Issued At	DateTime	UTC timestamp set when the token is created. Useful for audit trail and token age inspection.

Continued on next page

Name	Type	Description
Expires At	DateTime	UTC timestamp after which the token must be rejected even if not revoked. Set to <code>Issued At + TTL</code> . Token validation enforces <code>Expires At > CurrentDateTime()</code> .
Last Seen At	DateTime	Updated on every successful token validation. Supports session monitoring (activity/idle detection). Not used for expiry enforcement (use <code>Expires At</code>).
Revoked	Boolean	If true, token is explicitly invalidated and must be rejected even if not expired. Set on logout and during bulk revocation (e.g., password change, account deactivation).

Table 3.5. MES Auth Token (Table 50106) — Field Dictionary

Key points of the token design include a unique *GUID* token, auto-expiry after 12 hours, last activity tracking, and soft revocation.

Password Management and Encryption

The password management codeunit ensures that plaintext is never persisted. A 50-character random *salt* is generated per user and the password is hashed using *PBKDF2-SHA256* with 1000 iterations before storage. Data classification attributes mark sensitive fields as *Customer Content* or *System Metadata*.

Key points include: passwords are never stored in plaintext; a unique salt per user (50 characters) is generated; *PBKDF2* hashing with 1000 iterations is applied; and data is classified as *Customer Content* or *System Metadata*.

Authentication Logic — Login, Logout, and Change Password

Login

The login validates the user credentials against the MES `User` table. If the credentials are correct and the account is active, a token will be generated and stored in the MES `Auth Token` table.

The system returns a *JSON* login response object which includes the generated session token, the authenticated user's role string, and a boolean *needChangePassword* flag consumed by the Flutter frontend to decide the post-login navigation target.

Logout

On logout, the user's session authentication token will be expired. The token record matching the supplied session token is located in the MES `Auth Token` table and its expiry timestamp is set to the current datetime, immediately invalidating the session (soft revocation).

Change Password

Changing password requires a valid authentication token, a correct current password, and a new password that complies with security rules.

3.3.2 REST API and Web Services Implementation

To enable communication between Flutter and Microsoft Dynamics Business Central, a set of *RESTful APIs* were implemented using AL.

API Pages for MES User Management

The *API Page* definition for the `/mesUsers` endpoint exposes the MES User table as a read-only *OData* feed.

The *API Page* definition for the `/createMesUser` endpoint differs from the read endpoint by setting validation triggers. `DelayedInsert = true` buffers all field values until the complete request body has been received before committing the new MES User record to the database.

Key point: the difference between these two APIs is the *Delayed Insert* flag. When set to true, it delays insertion until all fields are received, which is required for POST operations. GET is not affected.

OData Functions and Web Service Configuration

Built MESUnboundActions codeunit (50125): `Login()`, `Logout()`, `Me()`, `ChangePassword()`. Exposed these functions in the MES Web Service codeunit (50126) exposed via ODataV4. Configured WebServices.xml to publish MES Web Service (codeunit 50126) as an ODataV4 web service under the service name MESWebService.

3.3.3 Frontend Implementation

The frontend of this system was developed using Flutter. The application is structured into four main layers: a *UI Layer* consisting of Screens and Widgets; a *State Management Layer* built on Providers; a *Service Layer* for API communication; and a *Model Layer* for Data Models.

The data flow follows a unidirectional pattern from the *UI Layer* through the *State Management Layer* (Provider pattern) and *Service Layer* to the Business Central REST API backend, as illustrated in Figure 3.12.

Service Layer (API Communication)

The *Service Layer* is responsible for handling all *HTTP* communication between Flutter and the Business Central backend.

Two services were implemented: the `api_service` manages authentication requests such as *login*, *logout*, and *password change*, while the `mes_user_service` handles MES user operations including fetching and creating new users.

`api_service.dart`

The class declaration and constructor show the base URL configuration and *HTTP* client initialisation used by all authentication endpoints. The *login* method sends user credentials and device ID to the `/login` endpoint and parses the returned session token, role, and *need-ChangePassword* flag. The *logout* method sends the active session token to the `/logout` endpoint to trigger server-side token revocation.

The *change password* method submits the current password, new password, and session token to the `/changePassword` endpoint. Shared error-handling and response-parsing utilities are used by all methods in the service class to propagate structured exceptions to the *State Management Layer*.

`mes_user_service.dart`

The class declaration and *fetch users* method send an authenticated *HTTP GET* request to the `/mesUsers` endpoint and deserialise the response into a list of `MesUser` model objects. The *create user* method sends a *HTTP POST* request to the `/createMesUser` endpoint with the new user's role and employee reference in the request body. The *generate password* method and associated response deserialisers handle the password generation endpoint and map the *JSON* result into the `GeneratedPassword` model.

State Management Layer using Provider

The *Provider* pattern is implemented to ensure reactive UI updates and centralised data handling. Two providers were implemented: `AuthProvider`, which manages authentication state, and `MesUserProvider`, which handles MES user data. Each provider wraps the corresponding

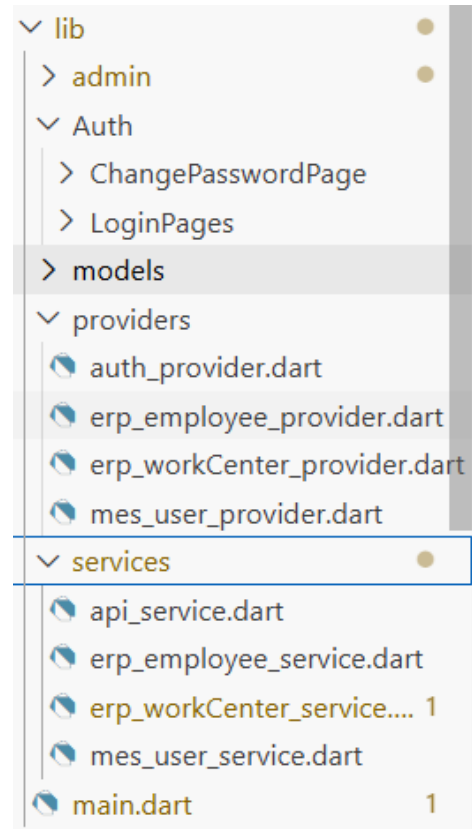


Figure 3.12. Flutter layered architecture.

service layer calls and exposes reactive state (authenticated user, user list, loading flag, error message) consumed by the *UI Layer* via `context.watch`.

User Interface

The *Login Page*, shown in Figure 3.13, is the application entry screen presented to all users on launch. It contains *User ID* and *Password* input fields and a *Login* button. Successful authentication redirects the user to the role-appropriate dashboard.

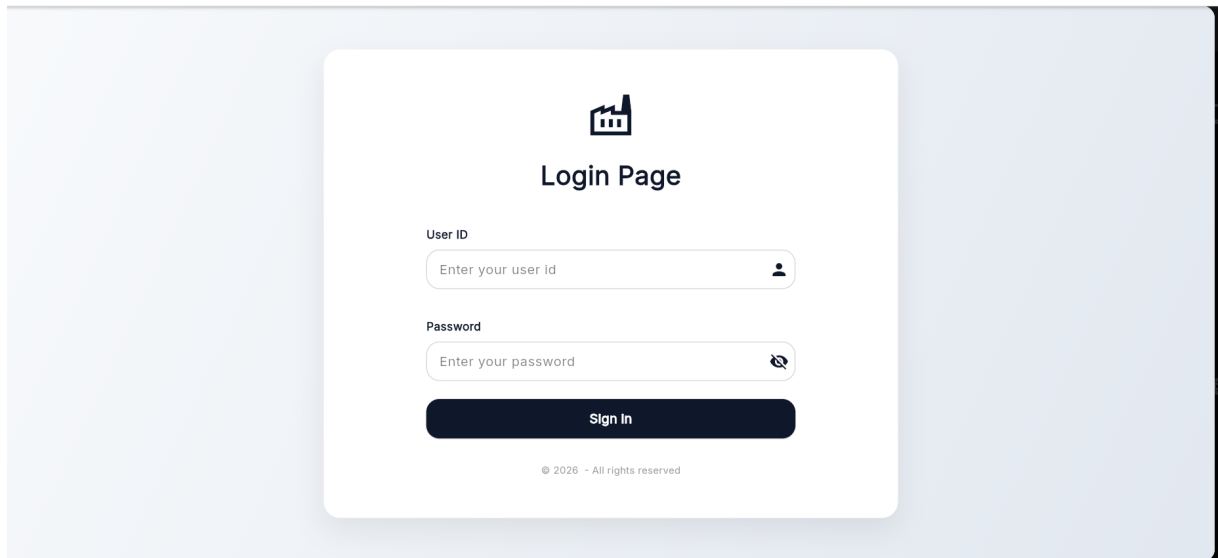


Figure 3.13. *Login Page.*

The *MES User List Page*, shown in Figure 3.14, is the Administrator dashboard screen listing all registered *MES* users with their assigned roles and active status, providing action buttons to assign roles or generate passwords for each employee.

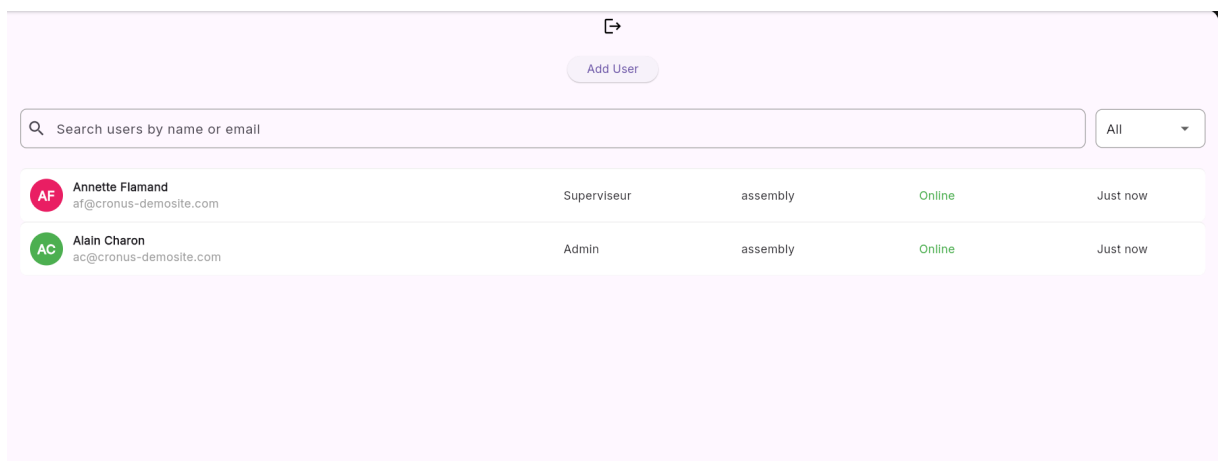
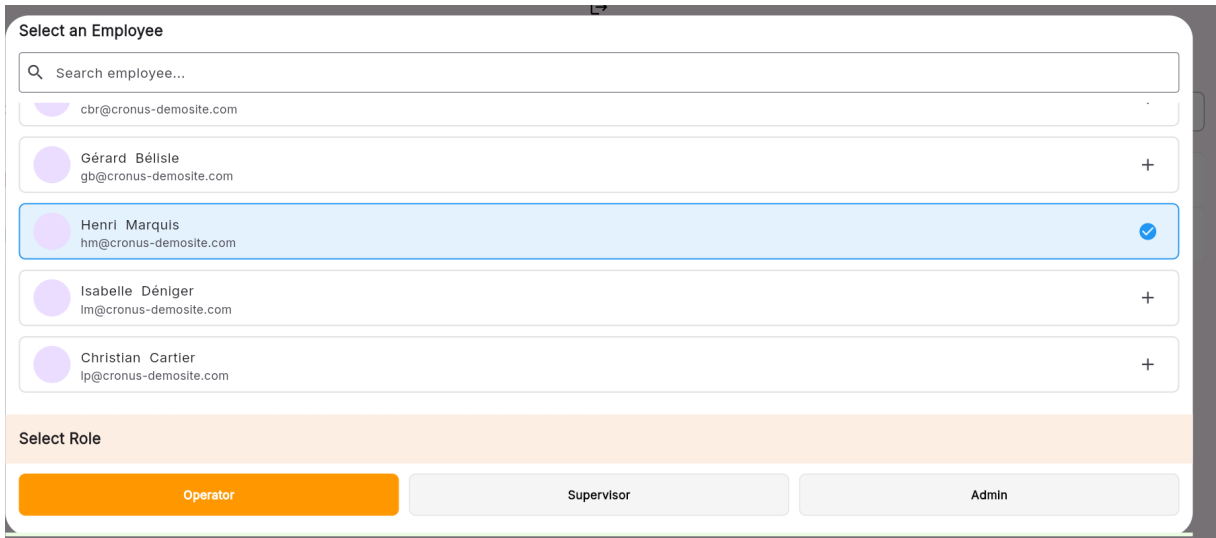


Figure 3.14. *MES User List Page.*

The *Assign MES Role Dialog*, shown in Figure 3.15, is a modal dialog allowing the Administrator to select a Business Central employee record and assign them an *MES* role (Operator or

Supervisor), creating a new MES User record via the `/createMesUser` endpoint.

The dialog is titled "Select an Employee" and contains a search bar with the placeholder "Search employee...". Below the search bar is a list of employees, each with a circular profile icon, their name, and email address. The employees listed are: cbr@cronus-demosite.com, Gérard Bélisle (gb@cronus-demosite.com), Henri Marquis (hm@cronus-demosite.com), Isabelle Déniger (lm@cronus-demosite.com), and Christian Cartier (lp@cronus-demosite.com). The entry for Henri Marquis is highlighted in blue and has a blue checkmark icon to its right. Below the employee list is a section titled "Select Role" with three buttons: "Operator" (orange), "Supervisor" (light gray), and "Admin" (light gray).

Select an Employee	
Search employee...	
 cbr@cronus-demosite.com	
 Gérard Bélisle gb@cronus-demosite.com	+
 Henri Marquis hm@cronus-demosite.com	✓
 Isabelle Déniger lm@cronus-demosite.com	+
 Christian Cartier lp@cronus-demosite.com	+

Select Role

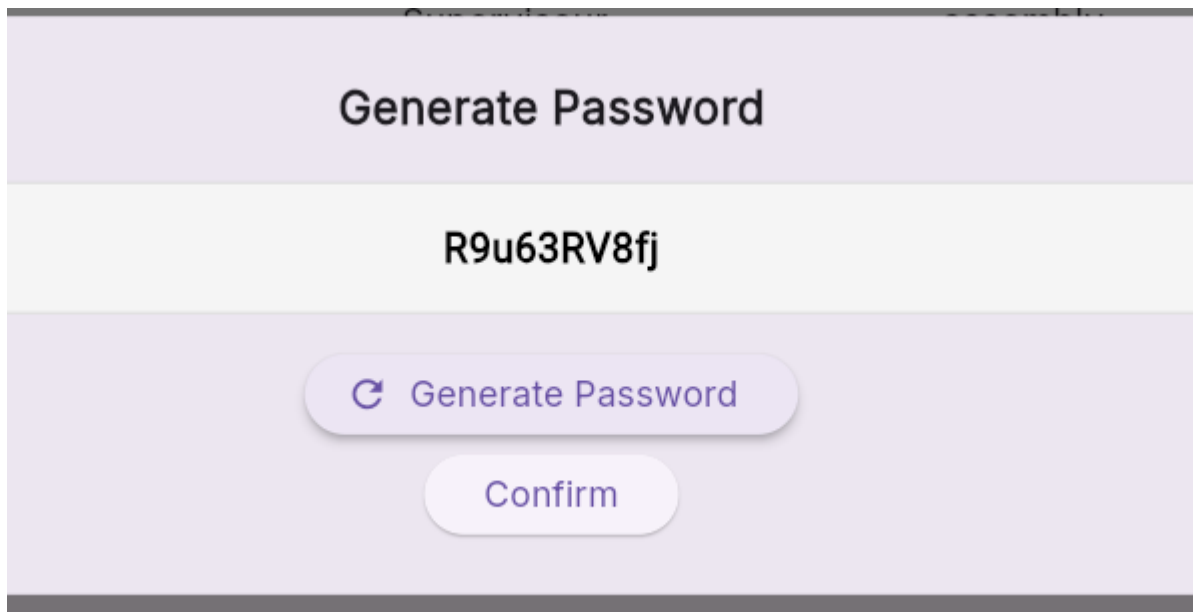
Operator

Supervisor

Admin

Figure 3.15. *Assign MES Role Dialog.*

The *Generate Password Dialog*, shown in Figure 3.16, is a modal dialog through which the Administrator triggers server-side secure password generation for a selected *MES* user. The generated password is displayed once and must be communicated to the user out-of-band.

The dialog has a light purple header with the title "Generate Password". Below the header is a light gray section displaying the generated password "R9u63RV8fj". At the bottom is a light purple section containing two buttons: "Generate Password" (with a circular arrow icon) and "Confirm".

Generate Password

R9u63RV8fj

Generate Password

Confirm

Figure 3.16. *Generate Password Dialog.*

The *Password Generated Successfully Dialog*, shown in Figure 3.17, is a confirmation dialog showing the plaintext password to the Administrator for one-time transmission to the user. The password is not stored in plaintext on the server after this point.

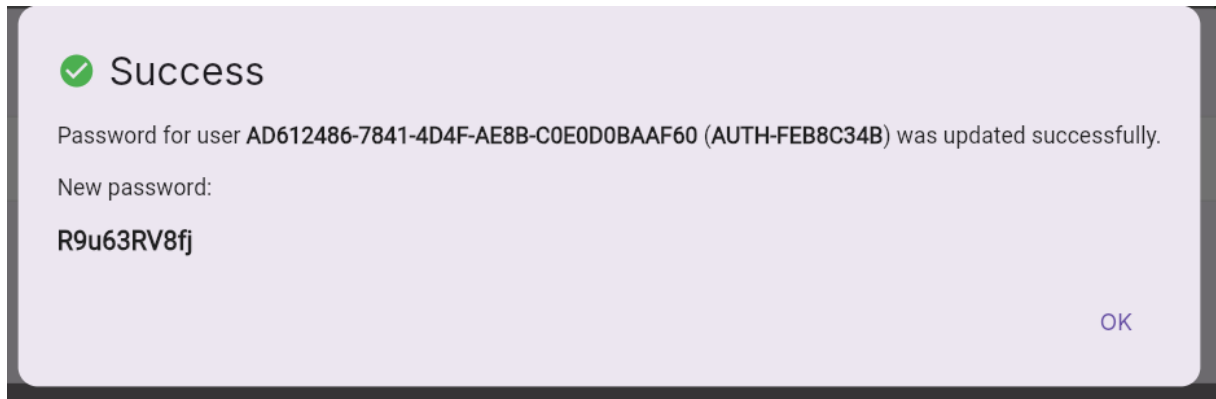


Figure 3.17. *Password Generated Successfully Dialog.*

The *Change Password Page*, shown in Figure 3.18, is presented to users whose *need-ChangePassword* flag is true after login, requiring them to enter their current password and choose a new password satisfying the rules defined in BR-AUTH-003 before accessing the main application.

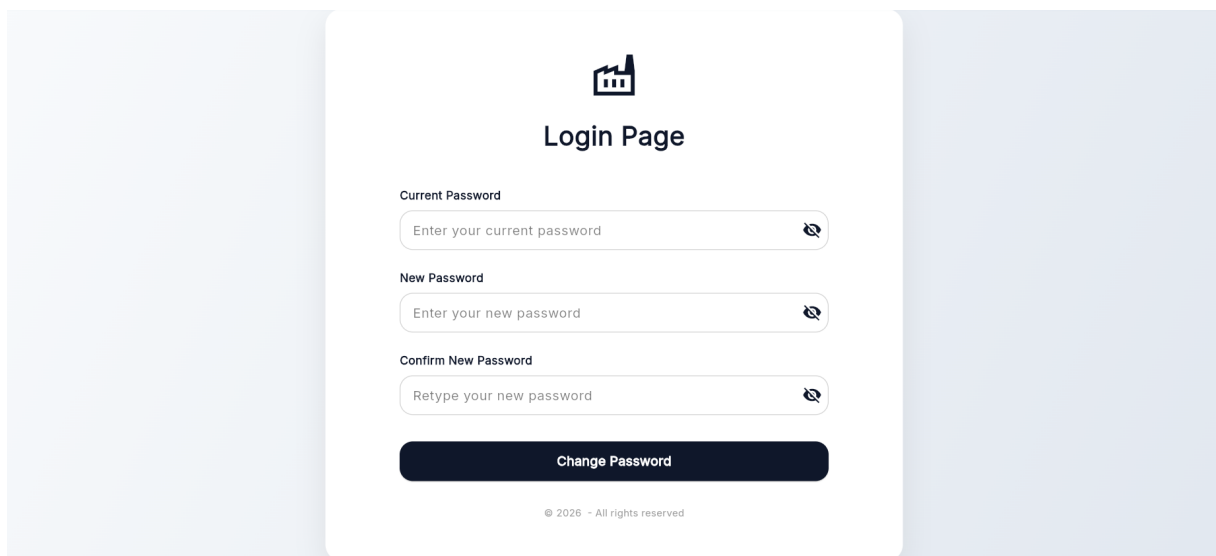


Figure 3.18. *Change Password Page.*

The *Change Role Dialog*, shown in Figure 3.19, is a modal dialog through which the Administrator updates the role and work-centre assignment of an existing *MES* user. The dialog pre-populates the current role and presents a three-button row (Operator / Supervisor / Admin); selecting Admin hides the work-centre list, selecting Operator shows a single-select list, and selecting Supervisor shows a multi-select list. An inline error banner displays any server-side validation failure. Tapping *Save Changes* submits the update and revokes all active tokens for the affected user.



Figure 3.19. *Change Role Dialog.*

The user row in the *MES User List Page* now includes a `PopupMenuButton` action menu accessible via a trailing icon on each row. The menu exposes two actions: *Change Role / Department*, which opens the `ChangeRoleDialog` described above, and *Activate* or *Deactivate*, whose label switches dynamically based on the user's current `isActive` flag. Selecting *Deactivate* immediately revokes all active sessions for that account; selecting *Activate* re-enables login.

3.4 Testing and Validation

3.4.1 Integration testing

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 3.20 shows a test case for the `/mesUsers` endpoint.

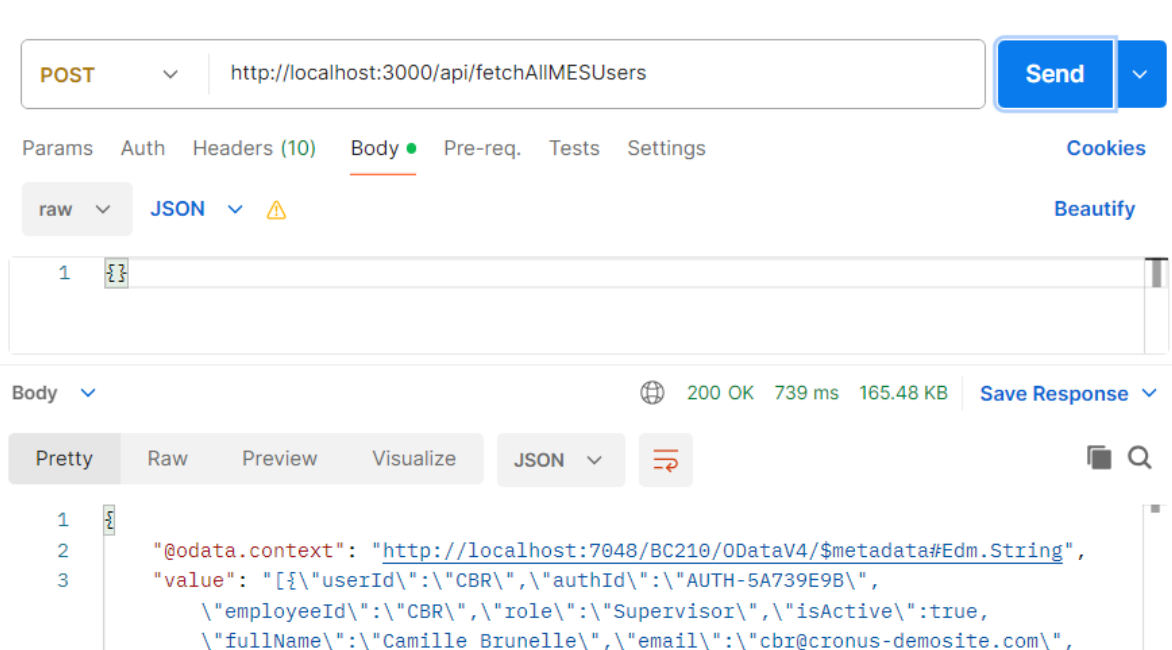


Figure 3.20. Postman endpoint test — `fetchAllMesUsers`.

Conclusion

This sprint successfully delivered a complete authentication and user management foundation for the *MES* application. The backend, built in Microsoft Dynamics Business Central using AL, implements secure credential validation, *PBKDF2-SHA256* password hashing, session token management with automatic expiry, and a full set of *RESTful* API endpoints exposed through MES Web Service (codeunit 50126). The Flutter frontend provides a clean, role-aware user experience covering login, logout, forced password change, administrator-level user creation, role and work-centre management, and account activation control.

All business rules defined at the start of the sprint — credential validation (BR-AUTH-001), session management (BR-AUTH-002), password security (BR-AUTH-003), role-work-centre constraint (BR-ADMIN-001), and token revocation on role change (BR-ADMIN-002) — have been implemented and verified. The layered architecture adopted on the frontend ensures that the codebase remains maintainable and extensible as new sprints introduce additional *MES* features.

Chapter 4

Sprint 2 Machine & Production Management

Contents

Introduction	52
4.1 Sprint Backlog	52
4.2 Business Rules	55
4.3 Design	57
4.3.1 Use Case Diagram	57
4.3.2 Textual Use Case Description	57
4.3.3 Sequence Diagrams	58
4.3.4 Class Diagram	63
4.4 Implementation	64
4.4.1 Backend Implementation	64
Backend Structure Overview	64
MES Machine Status Table	64
MES Operation Status Table	64
Enumerations	65
Machine Actions Business Logic	65
4.4.2 REST API and Web Services Implementation	66
4.4.3 Frontend Implementation	67
Model Layer	67
Service Layer	67
State Management Layer	68
User Interface	68
4.5 Testing and Validation	71
4.5.1 Integrations tests	71
Conclusion	72

Introduction

This sprint covers **Machine Monitoring, Production Order Management, and Operation History** in the *MES* application, giving Operators and Supervisors real-time visibility into machine states and a full audit trail of completed operations.

It addresses three related sub-domains: live machine status tracking from Business Central to the Flutter frontend, production order management with operation lifecycle initiation, and a history view showing all completed and cancelled operations per machine.

4.1 Sprint Backlog

This sprint covers **Domain 2: Machine Monitoring, Production Order Management & History**.

Table 4.1. Sprint Backlog

ID	User Story	Tasks
US-2.1	As an Operator or Supervisor, I need to see the list of machines in my work centre with their current status so that I can see which machines are currently active.	<i>Business Central (AL):</i> • Create table MES MachineStatus. • Create enum MES Machine Status. • Create function FetchMachines() in codeunit MES Machine Fetch. • Expose FetchMachines() through codeunit MES Web Service. • Create page MES Machine List. <i>Flutter:</i> • Create model MachineModel. • Create service MESMachineListService to consume the FetchMachines() API. • Create provider MesMachinesProvider. • Create page MachineListPage. • Create widget MachineCard. • Implement automatic machine list refresh in MESMachineListService.
US-2.2	As an Operator or Supervisor, I need to view the production orders assigned to a machine so that I know what work needs to be done.	<i>Business Central (AL):</i> • Create function getMachineOrders() in codeunit MES Machine Fetch. • Expose getMachineOrders() in codeunit MES Web Service.

Continued on next page

ID	User Story	Tasks
		<i>Flutter:</i> • Create model MachineOrderModel. • Create service ErpMachineOrdersService to consume the getMachineOrders() API. • Create provider MachineordersProvider. • Create page MachineOrderPage. • Create widget OrderCard. • Create model BadgeStyle.
US-2.3	As an Operator or Supervisor, I need to start a production operation so that the system records that the order has begun.	<i>Business Central (AL):</i> • Create table MES Operation state. • Create enum MES Operation Status. • Create function TryStartOperation() in codeunit MES Machine Validation. • Create function InsertMESOperation() in codeunit MES Machine Write. • Expose InsertMESOperation() through codeunit MES Web Service. • Create page MES Operations. <i>Flutter:</i> • Implement ErpMachineOrdersService that Consumes the startOperation API. • Implement MachineordersProvider to manage its state. • Implement start operation action handling in ActionButtons. • Implement automatic navigation to OrdersProgressionPage
US-2.4	As a Supervisor or Operator, I need to monitor the current status of all active operations on a machine so that I can follow the progress of each order.	<i>Business Central (AL):</i> • Create function fetchMachineOperationStatus() in codeunit MES Machine Fetch. • Expose fetchMachineOperationStatus() through codeunit MES Web Service.

Continued on next page

ID	User Story	Tasks
		<i>Flutter:</i> • Update <code>ErpMachineOrdersService</code> to Consume the <code>fetchMachineOperationStatus</code>. • Create model <code>OperationStatusStyle</code>. • Create page <code>OrdersProgressionPage</code>. • Create the widgets <code>OperationCard</code>, <code>OperationStatusBadge</code>, widget <code>OperationInfoGrid</code>, and <code>OperationProgressBar</code>. • Implement automatic refresh for operation status monitoring.
US-2.5	As an Operator or Supervisor, I need a tabbed view for a machine so that I can navigate between pending orders and active operation progress without leaving the machine context.	<i>Flutter:</i> • Create page <code>MachineMainPage</code>. • Create the widgets <code>TopNavigationBar</code> and <code>NavButton</code>. • Implement tab navigation in <code>MachineMainPage</code>. • Implement navigation from <code>MachineCard</code> to <code>MachineMainPage</code>.
US-2.6	As an Operator or Supervisor, I need to filter and sort production orders so that I can quickly find the most relevant work.	<i>Flutter:</i> • Create widget <code>GlobalSearchBar</code>. • Implement order search, filtering, and sorting in <code>MachineOrderPage</code>.
US-2.7	As a Supervisor or Operator, I need to view the history of completed operations for a machine so that I can audit past production.	<i>Business Central (AL):</i> • Create function <code>fetchOperationsStatusAndProgress()</code> in codeunit <code>MES Machine Fetch</code> and Expose it in codeunit <code>MES Web Service</code>.

Continued on next page

ID	User Story	Tasks
		<i>Flutter:</i> • Update <code>ErpMachineOrderService</code> to consume the <code>fetchMachineOperationStatusAndProgress</code> API. • Update <code>MachineordersProvider</code> to manage its state. • Create page <code>MachineHistoryPage</code>. • Create the widgets <code>HistoryCard</code>, <code>HistoryBadgeAndId</code>, and <code>HistoryInfoGrid</code>. • Implement history search and sorting in <code>MachineHistoryPage</code>. • Implement navigation from <code>HistoryCard</code> to <code>OperationDetailPage</code>.

4.2 Business Rules

Table 4.2. Business Rules

ID: Business Rule	Description
Machine Status Rules	
BR-MCH-001: Machine Status Tracking	• Machine status is stored as an append-only log in the MES Machine Status table; the latest record per machine represents the current state. • Valid statuses are <i>Idle</i>, <i>Starting</i>, and <i>OutOfOrder</i>. • The Flutter frontend polls the status endpoint every 5 seconds via a <i>Stream</i> to maintain a live view.
BR-MCH-002: Work Centre Scoping	• The machine list is always filtered to the work centre stored in the authenticated user's session token response. • An Operator or Supervisor can only see machines belonging to their assigned work centre.
Operation Lifecycle Rules	

Continued on next page

ID: Business Rule	Description
BR-OPS-001: Start Operation Validation	• Only routing lines with status <i>Released</i> may be started. • An operation that already has a <i>Running</i> record may not be started again. • A machine may not run more than one operation at a time; attempting to start a second operation on a busy machine is rejected with a descriptive error message.
BR-OPS-002: Immutable Operation History	• Each state transition (start, pause, resume) inserts a <i>new</i> record into MES Operation Status rather than modifying an existing one, preserving a full audit trail. • The latest status for a given (machine, order, operation) triple is determined by sorting on Last Updated At descending and taking the first record.
BR-OPS-003: Active Operation Query	• When fetching active operations for a machine, the backend groups records by (production order, operation) and returns only those whose most recent record has status <i>Running</i> or <i>Paused</i>. • Operations whose latest record is <i>Finished</i> are excluded from the active list.
History Query Rules	
BR-HIST-001: History Scope	• The History tab queries <code>fetchOperationsStatusAndProgress</code> with <code>fetchFinished = true</code>, returning only <i>Finished</i> and <i>Cancelled</i> executions. • The earliest <i>Running</i> status timestamp is resolved as <code>startDateTime</code>; the first <i>Finished</i> or <i>Cancelled</i> timestamp as <code>endDateTime</code>. • History is loaded as a one-shot Future on page open; it is not polled, as completed records are immutable.

4.3 Design

4.3.1 Use Case Diagram

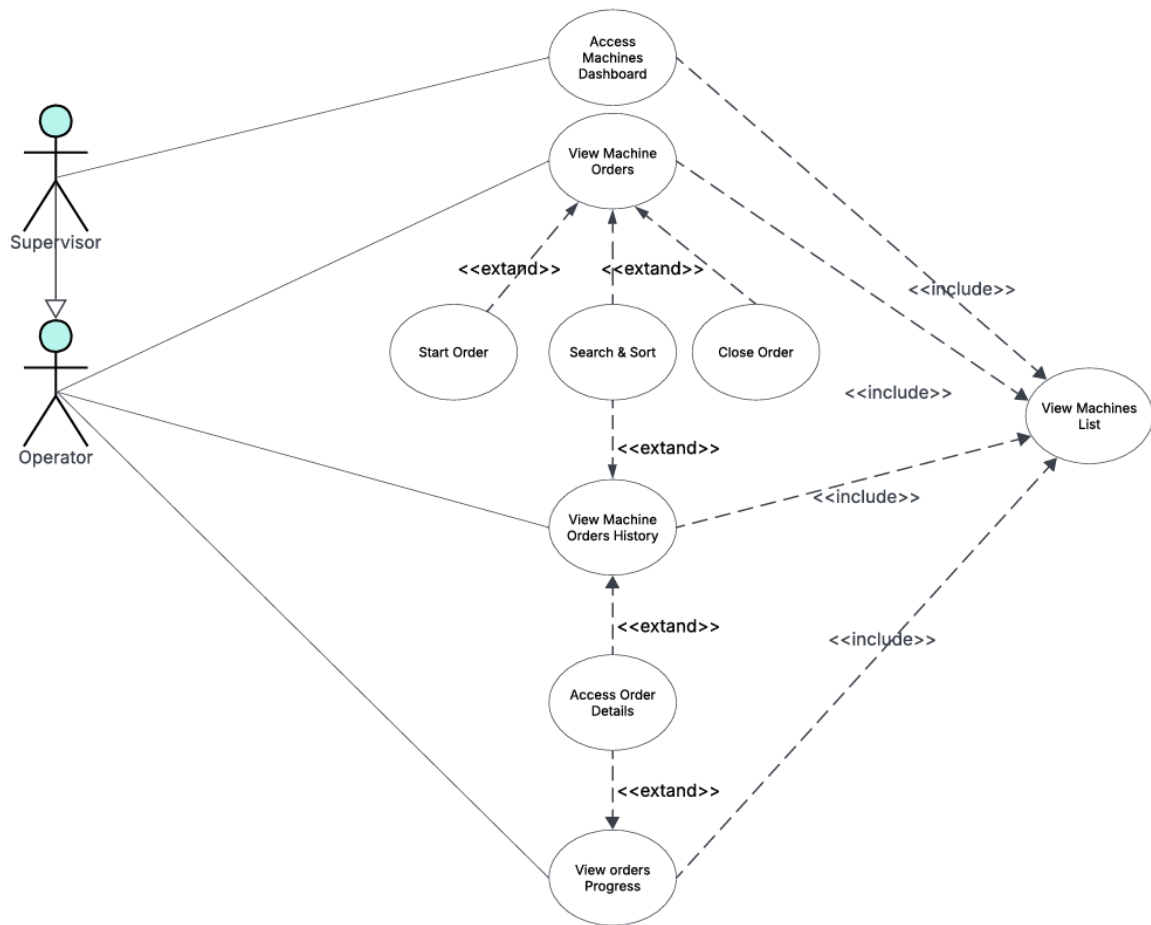


Figure 4.1. UML Use Case Diagram — Machine & Production Management.

The UML Use Case Diagram (Figure 4.1) shows the two primary actors (Operator and Supervisor) and their interactions with the machine monitoring, production order management, and history subsystem. In addition to viewing machines, browsing orders, and managing operation lifecycle transitions, the diagram includes the *View Operation History* use case, which allows both actors to access the completed and cancelled operation log for any machine in their work centre.

4.3.2 Textual Use Case Description

The following table (Table 4.3) provides the detailed textual description of the *Start Operation* use case.

Table 4.3. Textual Use Case Description — Start Operation

Use Case Name	Start Operation
Primary Actor	Operator
Preconditions	• The Operator is authenticated and has a valid session token. • The target routing line has status <i>Released</i>. • No other operation is currently <i>Running</i> on the same machine. • The operation has not already been started (no existing <i>Running</i> record for this order/operation pair).
Postconditions	• A new MES Operation Status record is inserted with status <i>Running</i> and the current <i>UTC</i> timestamp. • The <i>Operations Progress</i> tab reflects the new running operation within the next polling cycle. • If any precondition fails, a descriptive error dialog is shown to the operator and no record is inserted.
Main Scenario	1. The Operator navigates to the <i>Machine List Page</i> and selects a machine. 2. The system displays the <i>Machine Orders Page</i> listing all applicable production orders. 3. The Operator locates a <i>Released</i> order and taps <i>Start Order</i>. 4. The frontend sends a <i>POST</i> request to <code>MESMachinesActionsEndpoints_startOperation</code>. 5. The backend validates all preconditions (BR-OPS-001). 6. On success, a MES Operation Status record is inserted. 7. The frontend navigates to the <i>Orders Progression Page</i>.

4.3.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for the machine-listing and order-management flows introduced in Sprint 2.

Figure 4.2 depicts the machine-list polling flow. The Flutter client issues a `FetchMachines` request for each work centre resolved from the authenticated user’s profile every five seconds. For every machine returned by Business Central, the AL layer fetches the latest *MES Machine Status* row and the work-centre name before assembling the response, which the client uses to rebuild the *MachineCard* grid.

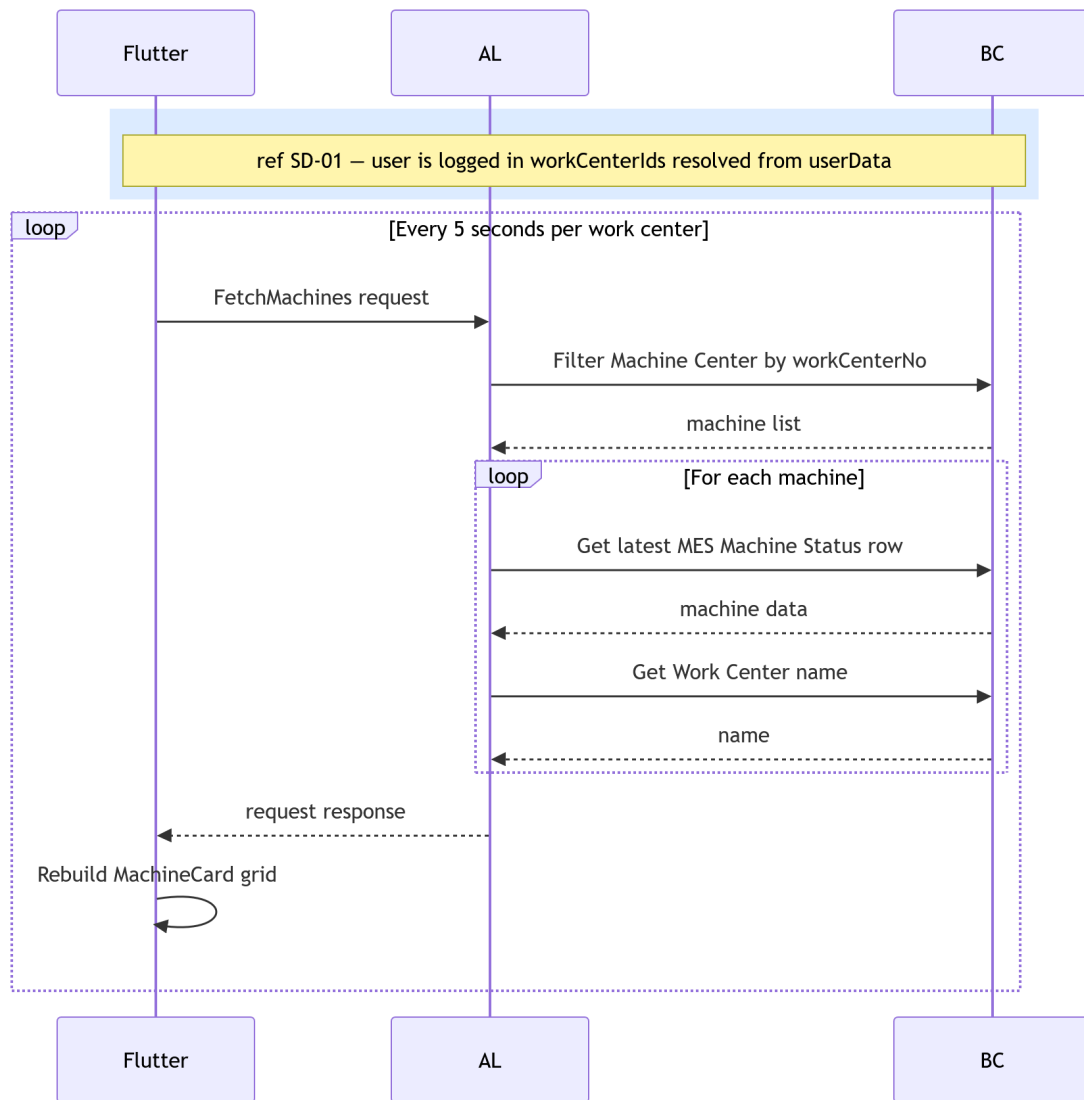


Figure 4.2. SD-09: Fetch Machines.

Figure 4.3 presents the machine-orders retrieval flow, triggered when the user opens the *Orders* tab. The AL layer filters *ProdOrderRoutingLine* records for the selected machine, skips any operation that has already been started, and enriches each remaining routing line with the corresponding *ProdOrderLine* details before returning the list to the Flutter client as a set of *OrderCard* entries.

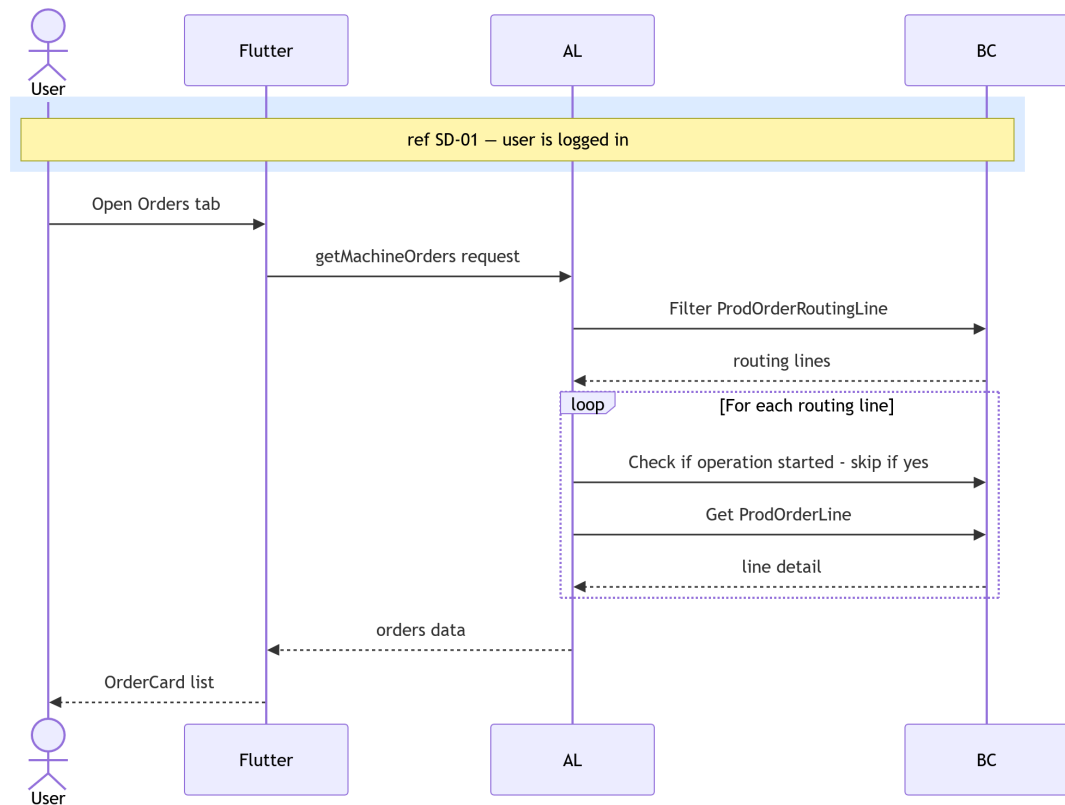


Figure 4.3. SD-10: Get Machine Orders.

Figure 4.4 illustrates the operation start flow. After the user selects a released order and taps *Start*, the AL layer confirms that the order is in the *Released* state and that neither the operation nor the machine is currently running. On success, it creates the *MES Operation Execution*, *MES Operation State*, and *MES Operation Progression* records, marks the machine as working, and records the initial user interaction before redirecting the Flutter client to the *Progress* tab.

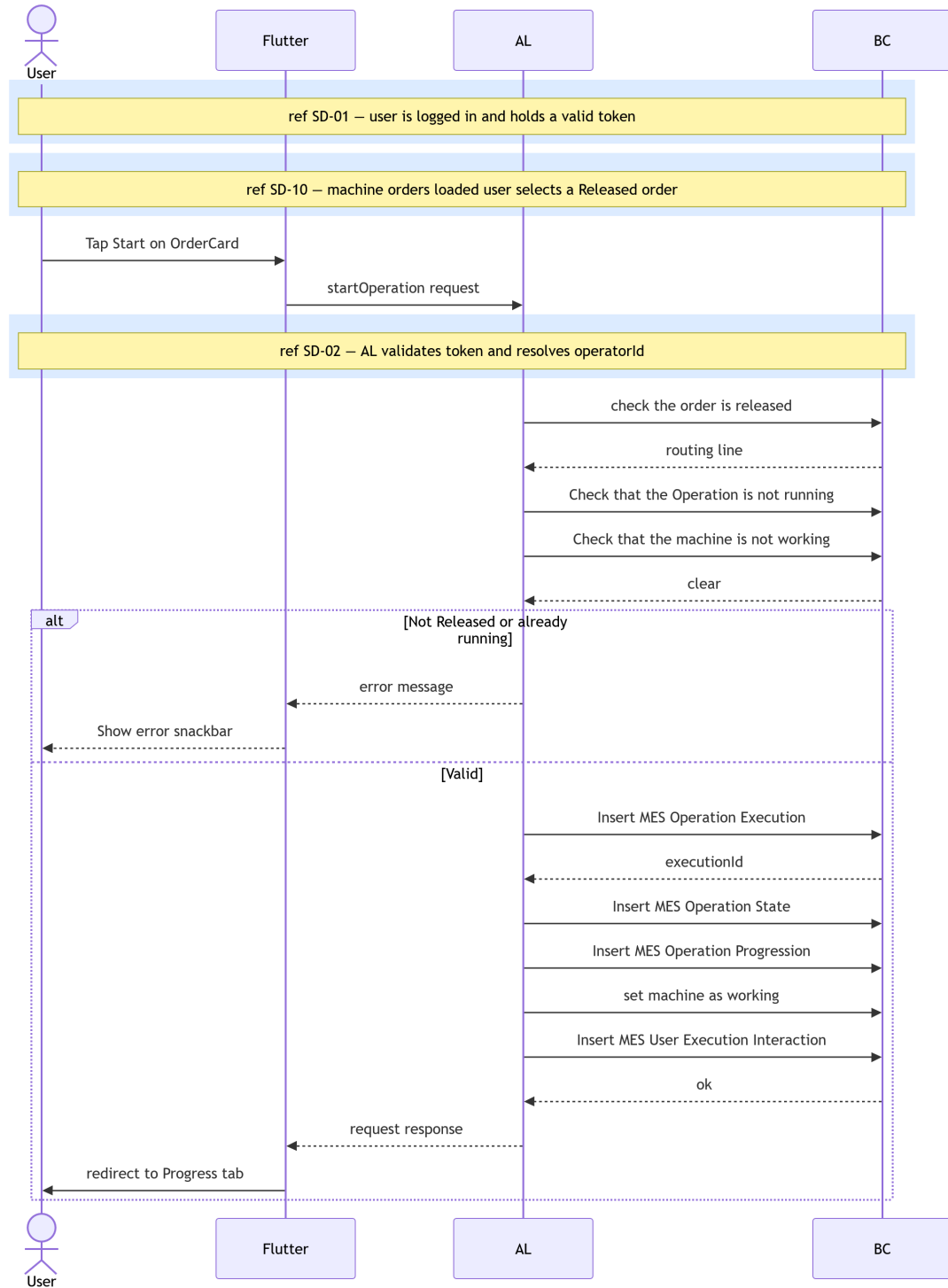


Figure 4.4. SD-11: Start Operation.

Figures 4.5 and 4.6 present the two polling flows that supply real-time and historical data to the *MachineDetailPage*. Both are triggered either on demand or on a fixed five-second interval once an operation is active.

Figure 4.5 describes the ongoing-operations state polling flow. The AL layer retrieves all active operations for the machine, resolves their current state and cumulative produced quantity from

Business Central, and returns the aggregated data to the Flutter client.

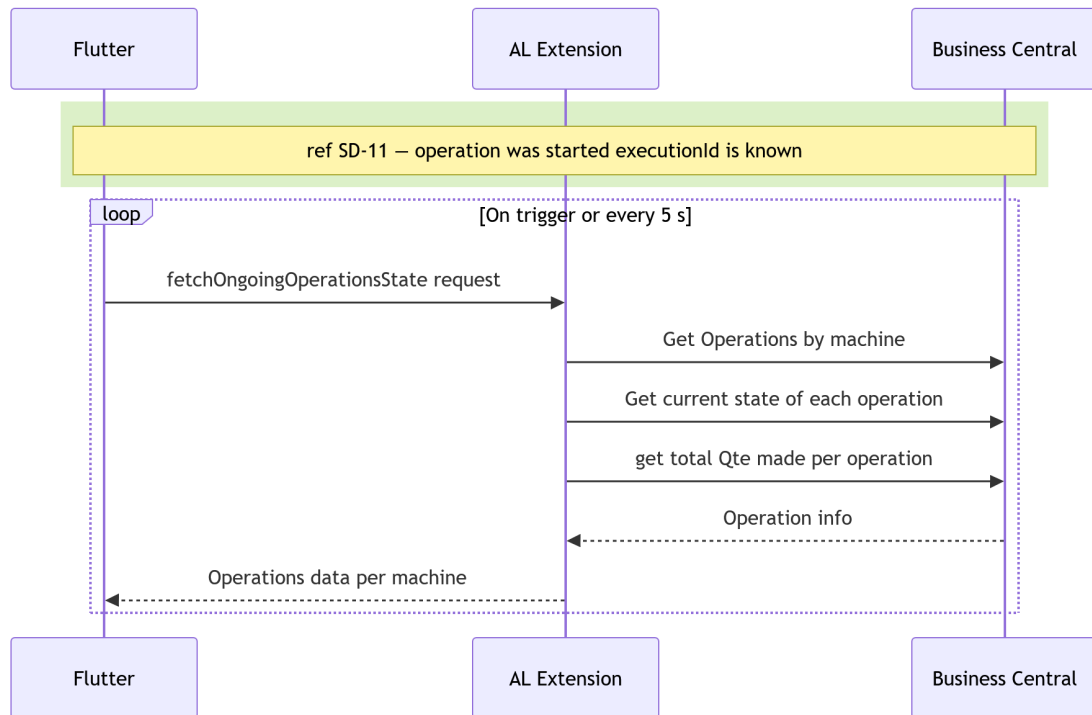


Figure 4.5. SD-23: Fetch Operation progress.

Figure 4.6 presents the operation history retrieval flow, triggered when the user opens the *MachineDetailPage*. The AL layer queries Business Central for finished and cancelled operations associated with the machine, enriches each record with production quantities and start and end times, and returns the assembled list so the Flutter client can display the operations history.

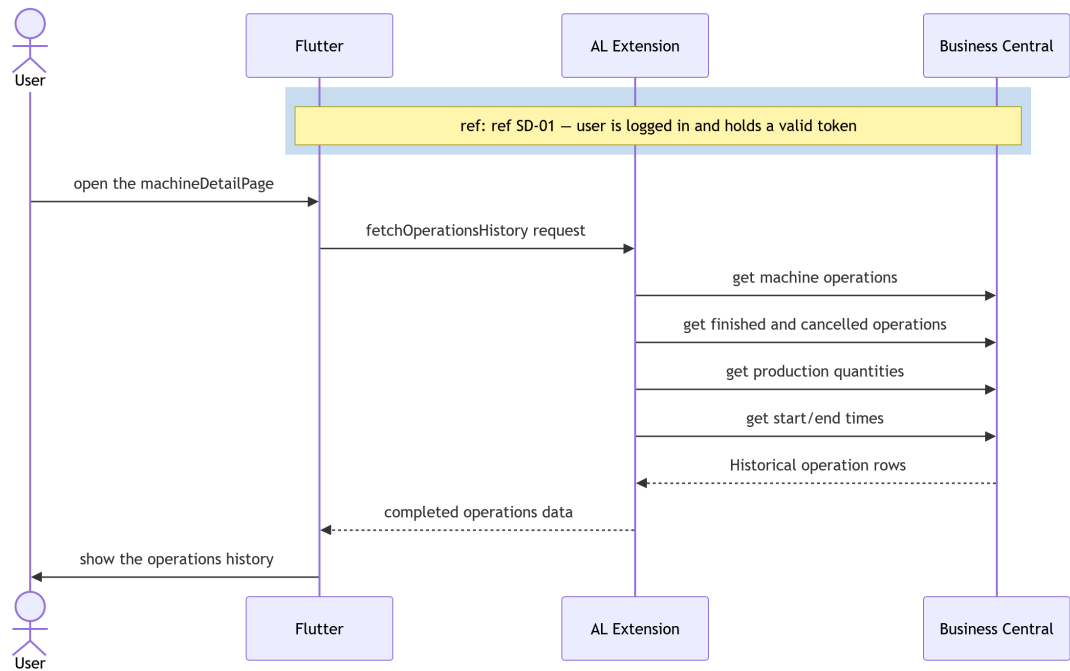


Figure 4.6. SD-24: Fetch Operation History

4.3.4 Class Diagram

The UML Class Diagram (Figure 4.7) of the machine and production management domain shows the MES Machine Status and MES Operation Status tables and their relationships with the Business Central standard tables Machine Center and Prod. Order Routing Line.

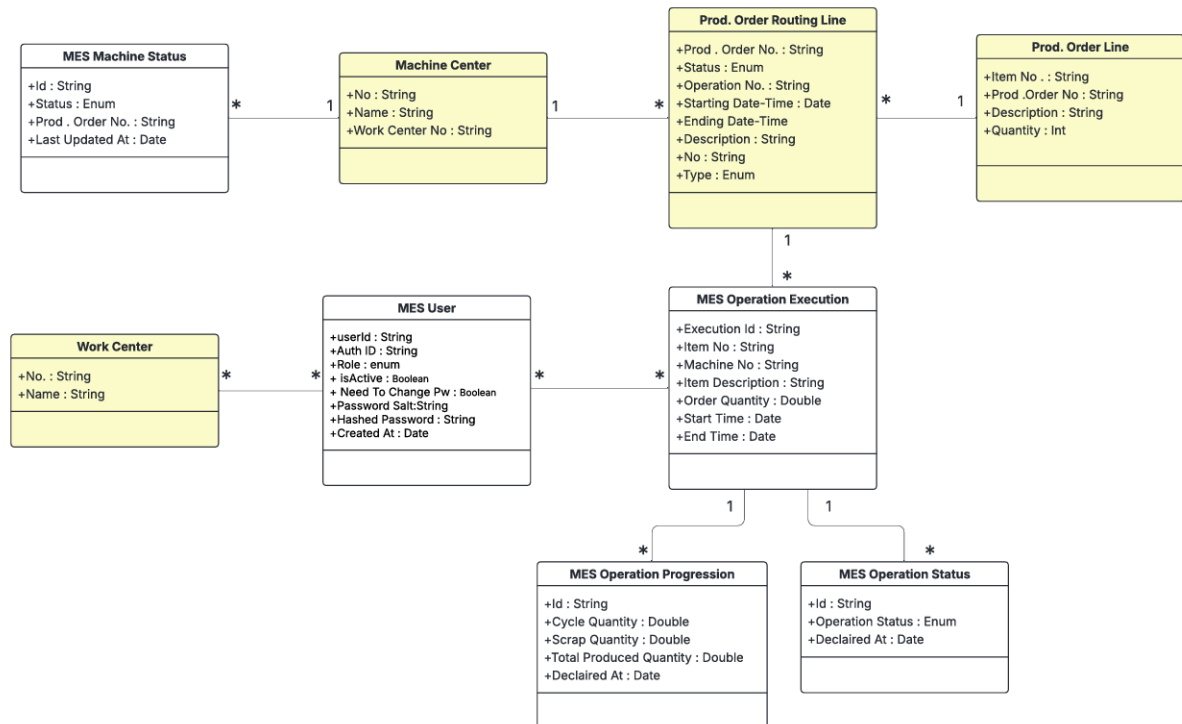


Figure 4.7. UML Class Diagram — Machine & Production Management.

4.4 Implementation

4.4.1 Backend Implementation

Backend Structure Overview

The machines domain follows the same layered structure established in Sprint 1. Two new sub-folders were added under `src/1-Tables/machines/` and `src/2-Enum/machines/`, and a new codeunit was placed under `src/3-CodeUnit/machines/`. The OData web service is published through the existing `WebServices.xml` mechanism under a new service name.

MES Machine Status Table

The MES Machine Status table (50107) stores the full history of machine state changes as an append-only log. Each insert creates a new record; the most recent record per machine represents the current state. Table 4.4 lists the fields and their descriptions.

Table 4.4. MES Machine Status (Table 50107) — Field Dictionary

Name	Type	Description
Id	Code[50]	Primary key. GUID-derived value assigned automatically on insert if blank.
Machine No.	Code[20]	Foreign key to Machine Center."No.". Identifies the machine this status record belongs to.
Status	Enum (MES Machine Status)	Current machine state: <i>Idle</i> , <i>Starting</i> , or <i>OutOfOrder</i> .
Current Prod. Order No.	Code[20]	Foreign key to Prod. Order Routing Line."Prod. Order No.". Records which production order the machine is currently executing, if any.
Last Updated At	DateTime	UTC timestamp set automatically on insert. Used to find the most recent status record for a given machine.

MES Operation Status Table

The MES Operation Status table (50108) implements the same append-only pattern for operation lifecycle events. Every state transition (start, pause, resume, finish) inserts a new record; queries always sort by Last Updated At descending to obtain the current state. Table 4.5 provides the full field dictionary.

Table 4.5. MES Operation State (Table 50108) — Field Dictionary

Name	Type	Description
Id	Code[50]	Primary key. GUID-derived identifier assigned automatically on insert.
Execution Id	Code[50]	Foreign key to MESOperationExecution.
Operator Id	Code[50]	Foreign key to MES User."User Id". The operator who triggered the state transition.
Operation Status	Enum (MES Operation Status)	Lifecycle state at the time of this record: <i>Running</i> , <i>Paused</i> , or <i>Finished</i> .
Last Updated At	DateTime	UTC timestamp set automatically on insert. Together with the composite key (Machine No, Prod Order No, Operation No), this field is used to retrieve the current status via descending sort.

Key points:

- Both tables use an append-only design: no Modify is called on existing records.
- The *secondary index* ExecutionTimeline("Execution Id", "Last Updated At") on MES Operation State supports efficient retrieval of the latest event per group.

Enumerations

Two enumerations support the domain. MES Machine Status (Enum 50101) defines three values: *Idle*, *Starting*, and *OutOfOrder*. MES Operation Status (Enum 50102) defines three values: *Running*, *Paused*, and *Finished*. Both enumerations are declared `Extensible = true` to allow future additions without modifying the base objects.

Machine Actions Business Logic

All machine and operation business logic is encapsulated in codeunit 50130 MES Machine Actions. The codeunit exposes four public procedures that are called directly by the OData web service layer.

FetchMachines. Accepts a *work centre number* and returns a *JSON* array of machine objects. For each Machine Center record scoped to the given work centre, the procedure looks up the latest MES Machine Status record via `FindLast()` and inlines the current status and active production order number into the response object. If no status record exists, the machine is reported as *Idle*.

getMachineOrders. Accepts a machine number and returns a *JSON* array of production routing lines whose status is *Planned*, *Firm Planned*, or *Released* and whose type is *Machine Center*. Routing lines that already have an existing MES Operation Status record are excluded from the result, ensuring that only work not yet started is shown in the order list. For each qualifying line, the corresponding Prod. Order Line is joined to supply item description and order quantity.

startOperation. Accepts a production order number, operation number, and machine number. The procedure delegates precondition checking to a [TryFunction] (TryStartOperation) that enforces BR-OPS-001: it verifies the routing line is *Released*, confirms no *Running* record exists for the same order/operation, and checks that the machine is not already busy with another operation. If all checks pass, a new MES Operation Status record with status *Running* is inserted by InsertMESOperation. On failure, the last error text is captured and returned as a structured *JSON* error response.

fetchMachineOperationStatus. Returns the current active operations for a machine as a *JSON* array. The procedure sets a descending composite key on (Machine No, Prod Order No, Operation No, Last Updated At) and iterates the result set, emitting at most one entry per (order, operation) pair — the one with the most recent timestamp. Only entries whose latest status is *Running* or *Paused* are included in the output, satisfying BR-OPS-003.

4.4.2 REST API and Web Services Implementation

Communication between Flutter and Business Central for the machines domain uses the same *OData V4 Unbound Actions* pattern established in Sprint 1. A new web service entry was added to WebServices.xml exposing codeunit 50130 under the service name MESMachinesActionsEndpoints. After deployment, the four procedures are reachable at the following endpoints:

- POST .../ODataV4/MESWebService_FetchMachines
- POST .../ODataV4/MESWebService_getMachineOrders
- POST .../ODataV4/MESWebService_startOperation
- POST .../ODataV4/MESWebService_fetchMachineOperationStatus

All endpoints follow the same *JSON* request/response convention used by the authentication layer: the request body carries named parameters as top-level *JSON* fields, and the response wraps the AL return value inside a "value" field which the Flutter service layer double-decodes.

4.4.3 Frontend Implementation

The machines domain retains the same four-layer call stack introduced in Sprint 1 (*Model* → *Service* → *Provider* → *UI*), but Sprint 2 introduced a significant reorganisation of the *UI layer* file structure, making the code-base easier to maintain and facilitating code reuse across components.

Model Layer

Two models were introduced. `MachineModel` (`mes_machine_model.dart`) holds the machine number, name, current status string, and active order number, populated from the `FetchMachines` response. `MachineOrderModel` (`erp_order_model.dart`) holds the full set of production order fields returned by `getMachineOrders`, including order number, status, operation number, planned start and end *DateTime* values, item number, item description, order quantity, and operation description.

Service Layer

Two service classes handle all network communication for the machines domain. `MESMachineListService` (`mes_MachineList.dart`) exposes a `fetchMachines` method that posts the work centre number and parses the double-encoded *JSON* array into a list of `MachineModel` objects. It also exposes a `streamMachines` method that wraps `fetchMachines` in an `async*` generator loop with a

5-second `Future.delayed` delay, emitting an updated list on every cycle and yielding an empty list on error to prevent the *StreamBuilder* from entering an error state.

`ErpMachineOrdersService` (`erp_order_service.dart`) exposes three methods: `getMachineOrders` for fetching the order list, `getStartOperationValidation` for triggering the start operation endpoint and parsing the success/failure result, and `fetchMachineOperationStatus` for retrieving active operation records. A `streamMachines` generator on this service applies the same 5-second polling pattern for the operations progress view.

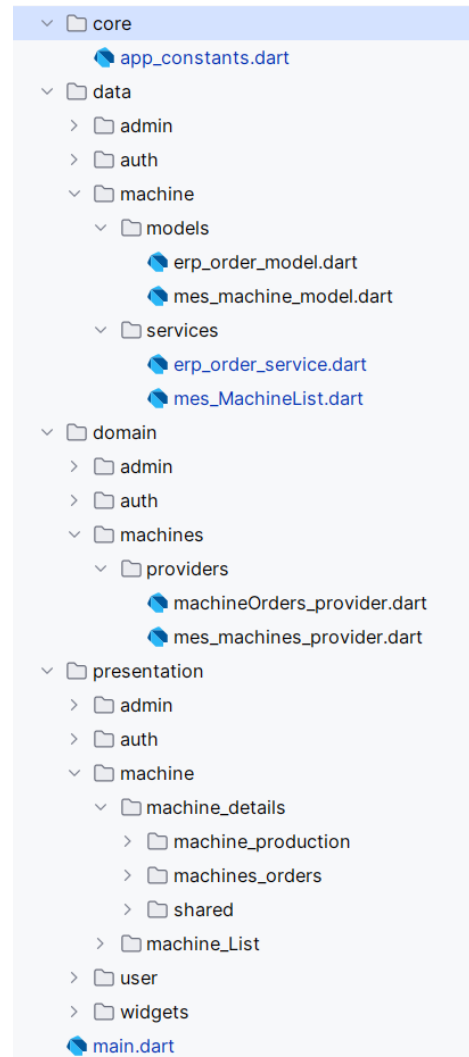


Figure 4.8. Flutter machines domain — layered architecture.

State Management Layer

Two providers were introduced. `MesMachinesProvider` (`mes_machines_provider.dart`) is a plain (non-`ChangeNotifier`) provider that delegates directly to the `MESMachineListService` stream. Because all state is carried by the stream itself, no `notifyListeners` mechanism is needed and the *StreamBuilder* in the UI subscribes directly to the emitted values.

`MachineordersProvider` (`machineOrders_provider.dart`) wraps both the order list fetching and the operation start flow with the standard `ChangeNotifier` pattern. It exposes `getMachineOrders` which populates a `machineOrders` list, and `startOrder` which calls the service and returns a boolean success flag. It also exposes `getMachineOperationsStatusStream` which passes the polling stream through directly to the UI layer.

User Interface

The *Machine List Page* is the entry point for operators after login. It displays a live-updating grid (tablet/desktop) or list (mobile) of machine cards scoped to the authenticated work centre. Each card shows the machine name, its current status with a colour-coded badge, and the active production order number. The layout adapts across breakpoints: a single-column list below 600 dp, a two-column grid below 1024 dp, and a four-column grid above. As illustrated in Figure 4.9, the interface is consistent across both desktop and mobile platforms.

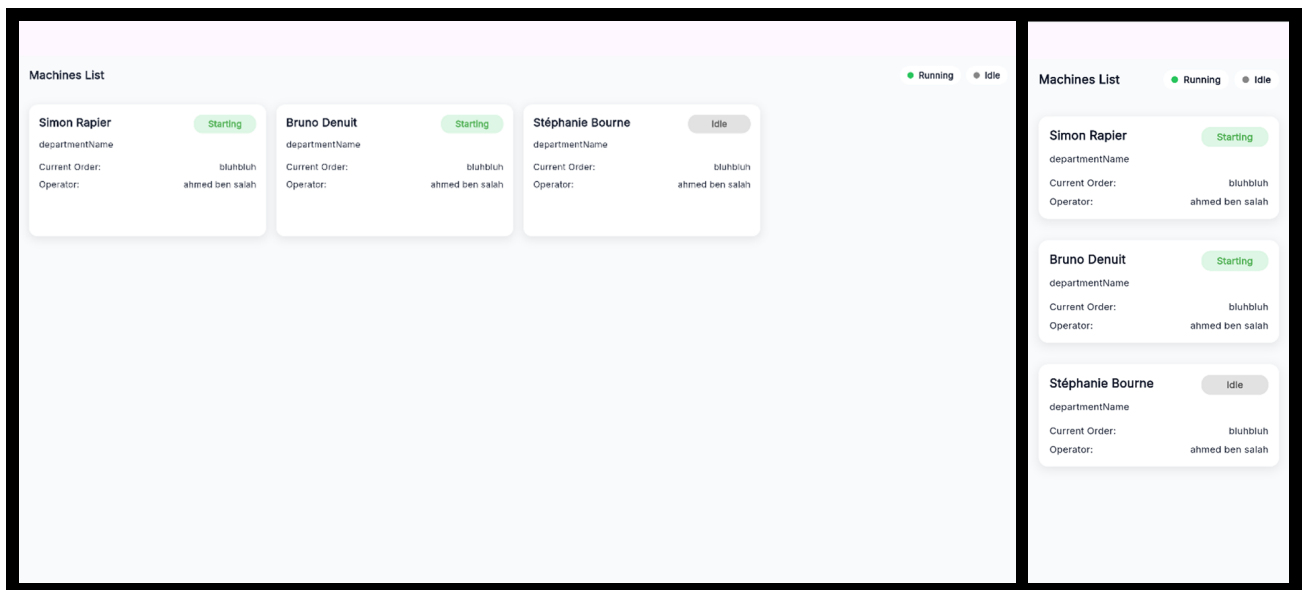


Figure 4.9. *Machine List Page* — desktop and mobile views.

The user interface for the machines domain is split across two top-level pages accessible via a shared *Top Navigation Bar* with tabs for *Orders*, *Progress*, *Consumable*, and *History*.

The *Machine Orders Page* displays the production routing lines assigned to the selected machine. A *Global Search Bar* allows filtering by free text (order number, item description, or date), by

status, and sorting by planned start date. Order cards use a responsive layout that switches between a stacked *narrow layout* and a side-by-side *wide layout* at a 520 dp breakpoint. Only *Released* orders display an active *Start Order* button; *Planned* and *Firm Planned* orders show it disabled. Figure 4.10 illustrates both the desktop and mobile views.

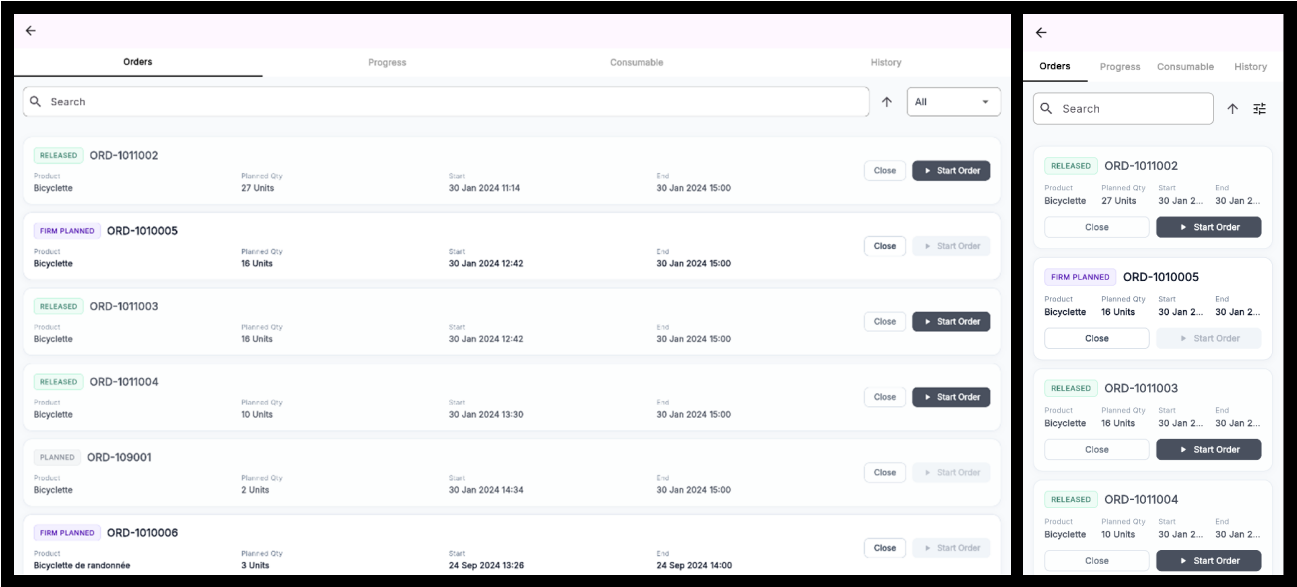


Figure 4.10. Machine Orders Page — desktop and mobile views.

The *Orders Progression Page* displays the currently active operations on a machine, polling every 5 seconds. Each operation card shows a colour-coded status badge (green for *Running*, amber for *Paused*), the order and operation identifiers, the last updated timestamp, and a progress bar. The card layout also switches between narrow and wide variants at the 520 dp breakpoint. *Pause* and *Resume* action buttons adapt their colour and label to the current operation status, as shown in Figure 4.11.

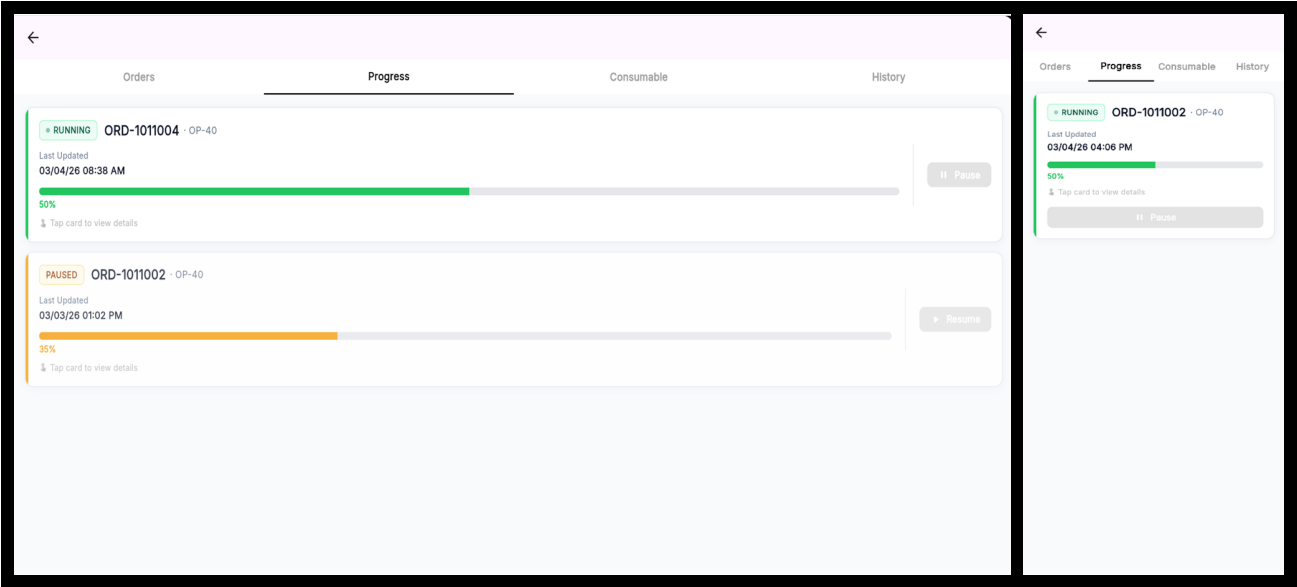


Figure 4.11. Orders Progression Page — desktop and mobile views.

The *Machine History Page* (Figure 4.12) displays completed operations assigned to the selected machine, limited to *Finished* and *Cancelled* statuses. It shares the same layout and filtering behaviour as the *Machine Orders Page* — including free-text search, status filtering, and sort controls — but omits the *Start Order* button. Tapping an operation card navigates to its *Operation Detail Page* for review. The *Machine History Page* is accessible via the *History* tab in the *TopNavigationBar*. Figure 4.12 shows the desktop and mobile views.

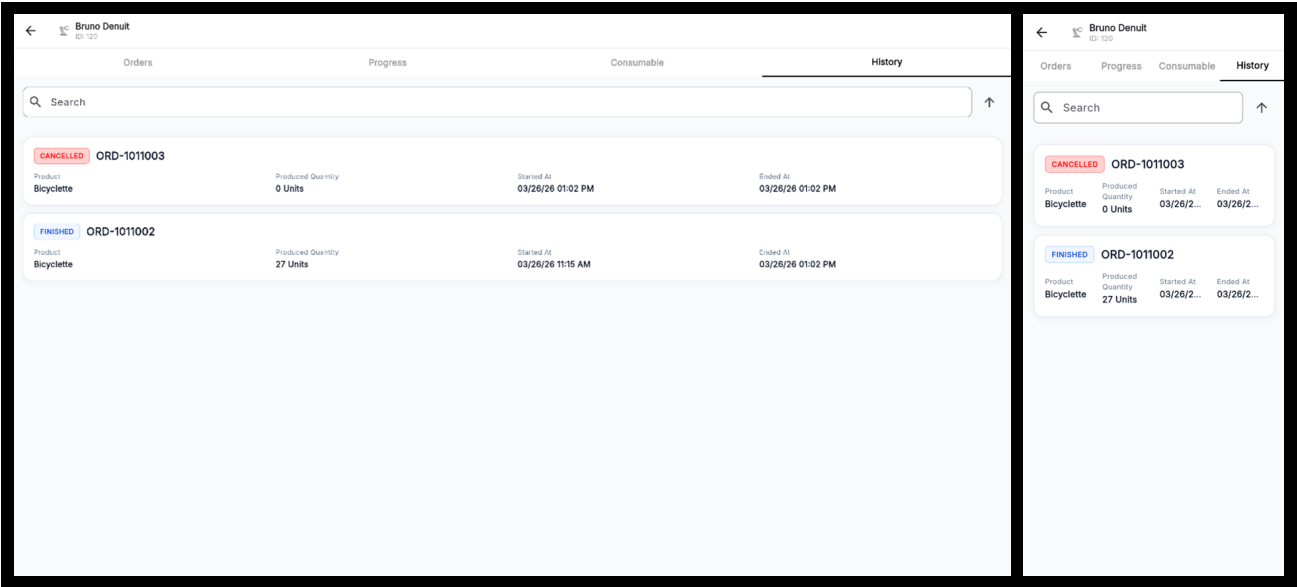


Figure 4.12. Machine History Page — desktop and mobile views.

4.5 Testing and Validation

4.5.1 Integrations tests

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 4.13 shows a test case for the `/fetchmachines` endpoint.

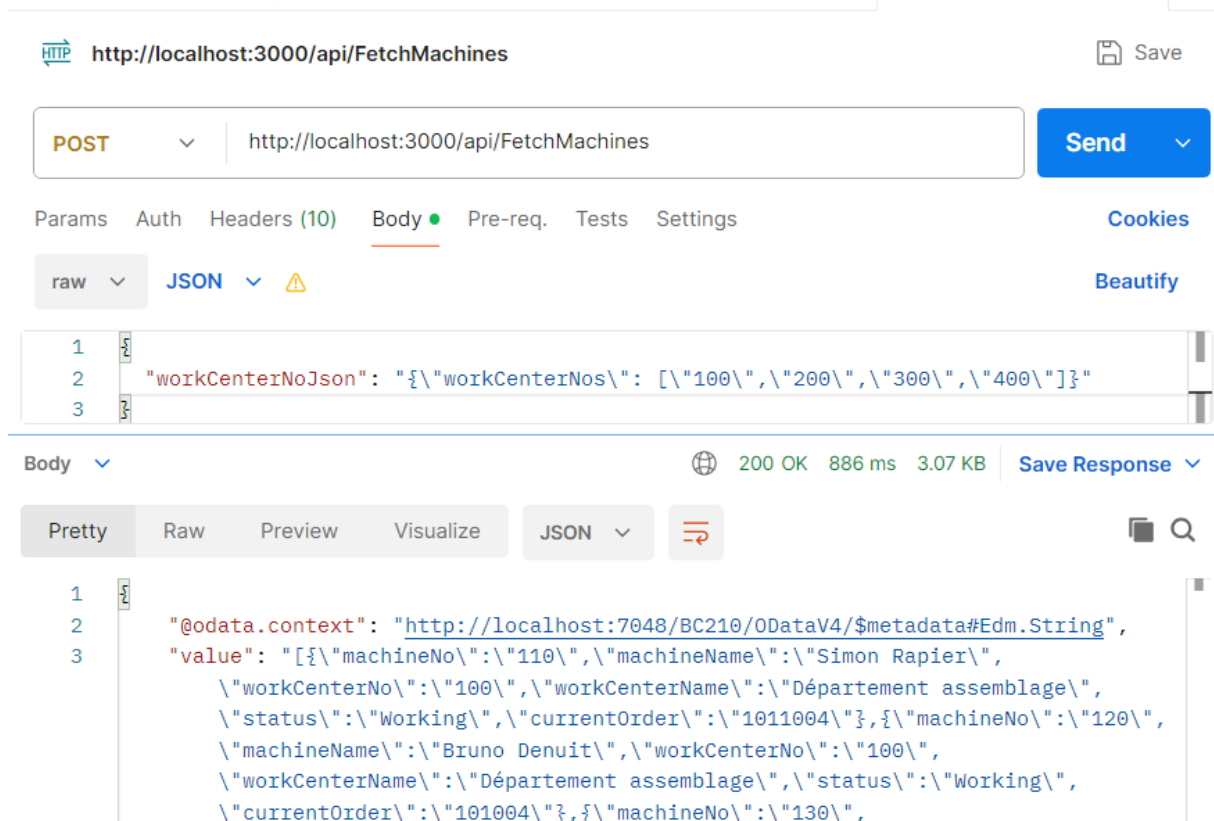


Figure 4.13. Postman endpoint test — `fetchMachines`.

Figure 4.14 shows a test case for the `/getMachinesOrders` endpoint.

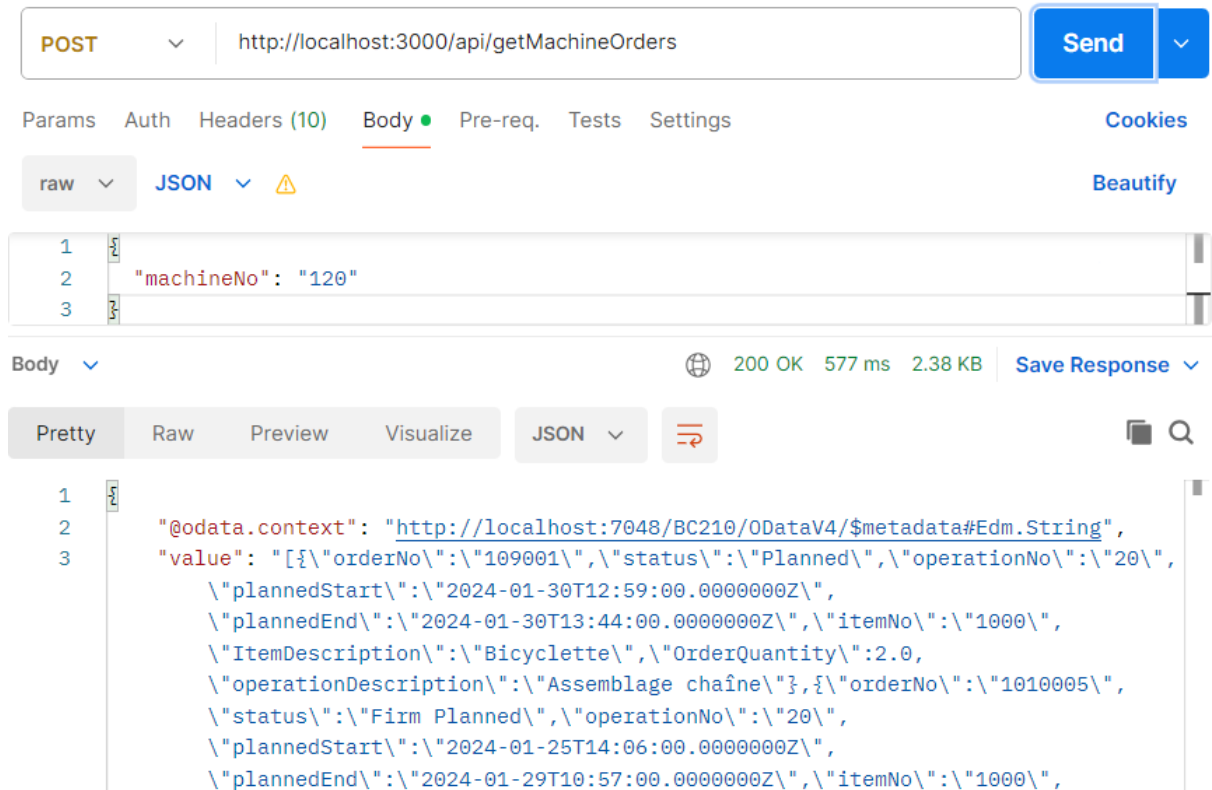


Figure 4.14. Postman endpoint test — getMachinesOrders.

Conclusion

This sprint successfully delivered the machine monitoring and production order management foundation of the *MES* application. On the backend, two new append-only tables (MES Machine Status and MES Operation Status) and a dedicated codeunit (MES Machine Actions) were implemented in Business Central AL, exposing four *OData V4* endpoints that cover the full machine and operation lifecycle. The append-only design ensures a complete, tamper-evident audit trail of every state transition without requiring any record modification.

All business rules defined at the start of the sprint have been enforced: work centre scoping (BR-MCH-002), start operation validation (BR-OPS-001), immutable history (BR-OPS-002), active operation querying (BR-OPS-003), and history scope (BR-HIST-001). The Flutter frontend delivers a responsive, real-time interface with 5-second polling streams for the machine list and operations progress view, and a one-shot history load for the completed operation audit log. The codebase is now ready to extend in Sprint 3 with consumable tracking, operation finishing, and production declaration.

Chapter 5

Sprint 3 Production Execution & Traceability

Contents

Introduction	74
5.1 Sprint Backlog	74
5.2 Business Rules	80
5.3 Design	83
5.3.1 Use Case Diagram	83
5.3.2 Textual Use Case Description	86
5.3.3 Sequence Diagrams	87
5.3.4 Class Diagram	96
5.4 Implementation	97
5.4.1 Backend Implementation	97
Tables and Entities	97
REST API and Web Services Implementation	98
5.4.2 Frontend Implementation	99
Provider Architecture	100
User Interface	100
5.5 Test and Validation	102
5.5.1 Integration test	102
Conclusion	103

Introduction

This sprint report covers **Domain 3: Production Execution & Traceability**. Building upon the authentication and machine monitoring infrastructure established in the previous sprints, this sprint delivers the core production execution features: declaring production output, pausing and resuming operations, finishing or cancelling orders, tracking production cycles over time, monitoring Bill of Materials consumption, component barcode scanning, scrap declaration, supervisor proxy declaration, and label printing. The result is a fully operational shop-floor execution loop from order assignment through to order closure, material traceability, and printed part identification.

5.1 Sprint Backlog

This sprint covers **Domain 3: Production Execution & Traceability**.

Table 5.1. US-3.1 — Declare Production

ID	User Story	Acceptance Criteria
US-3.1	As an Operator or Supervisor, I need to declare the quantity of parts produced in a cycle so that the system records my production progress.	<i>Business Central (AL):</i> • Create table MES Operation Progression. • Create the functions: – TryDeclareProduction() in codeunit MESMachineValidation. – InsertNewProgressionCycle() in codeunit MESMachineInsert. – declareProduction() in codeunit MESMachineWrite. • Expose declareProduction() through the codeunit MES Web Service. • Create page MES Operation Progression list page. <i>Flutter:</i> • Create ErpMachineOrdersService to consume the declareProduction API and MachineordersProvider to manage it. • Create widget DeclareProductionDialog. • Add the Declare Production action in ActionButtonsContainer.

Continued on next page

ID	User Story	Acceptance Criteria
US-3.2	As an Operator or Supervisor, I need to pause and resume an active operation so that production interruptions are tracked accurately.	<i>Business Central (AL):</i> • Create the functions: – TryPauseOperation() and TryResumeOperation() in codeunit MESMachineValidation. – ExecuteOperationTransition(), pauseOperation(), and resumeOperation() in codeunit MES-MachineWrite. • Expose pauseOperation() and resumeOperation() through codeunit MES Web Service. <i>Flutter:</i> • Implement ErpMachineOrdersService to consume the new endpoints • Implement MachineordersProvider to manage its state. • Create widget OperationActionButtons. • Implement pause/resume operation toggle handling in OrdersProgressionPage.
US-3.3	As a Supervisor, I need to finish or cancel a production operation so that the machine is released and the order is closed in the system.	<i>Business Central (AL):</i> • Create the functions: – TryCloseOperation() in codeunit MESMachineValidation. – InsertOperationStatus() and InsertIdleMachineStatus() in codeunit MESMachineInsert. – finishOperation() and cancelOperation() in codeunit MES-MachineWrite. • Expose finishOperation() and cancelOperation() in codeunit MES Web Service.

Continued on next page

ID	User Story	Acceptance Criteria
US-3.4	As an Operator or Supervisor, I need a dedicated detail page for an operation so that I can see all real-time information in one place.	<i>Flutter:</i> • Update <code>ErpMachineOrdersService</code> to consume the new endpoints. • Update <code>MachineordersProvider</code> to manage its state. • Update <code>ActionButtonsContainer</code> to handle finish/cancel operation flow. • Implement finish/cancel button state logic in <code>ActionButtonsContainer</code>.
		<i>Business Central (AL):</i> • Create table MES Operation Execution. • Create function <code>InsertMESOperationExecution()</code> in codeunit <code>MESMachineInsert</code>. • Create function <code>fetchOperationLiveData()</code> in codeunit <code>MESMachineFetch</code>. • Expose <code>fetchOperationLiveData()</code> through codeunit MES Web Service. • Create page MES Operation Execution. <i>Flutter:</i> • Create model <code>OperationStatusAndProgressModel</code>. • Update <code>ErpMachineOrdersService</code> to consume the new endpoints. • Update <code>MachineordersProvider</code> to manage its state. • Create page <code>OperationDetailPage</code>. • Create widget <code>OperationAppBar</code>. • Create widget <code>CurrentOrderInfoContainer</code>.
US-3.5	As an Operator or Supervisor, I need to see all production cycles declared for an operation so that I can review each declaration's quantity, operator, and timestamp.	<i>Business Central (AL):</i> • Create function <code>fetchProductionCycles()</code> in codeunit <code>MESMachineFetch</code>. • Expose <code>fetchProductionCycles()</code> through codeunit MES Web Service.

Continued on next page

ID	User Story	Acceptance Criteria
US-3.6	As an Operator or Supervisor, I need to see a visual chart of production declarations over time so that I can identify trends and performance patterns.	<i>Flutter:</i> • Create model ProductionCycleModel. • Add fromJson() and computed getters to ProductionCycleModel. • Update ErpMachineOrdersService to consume the new endpoints. • Update MachineordersProvider to manage its state. • Create widget ProductionCycle.
		<i>Flutter:</i> • Create widget ProductionChart.
US-3.7	As an Operator or Supervisor, I need to see the Bill of Materials for a production operation so that I know which components are required and how much has already been consumed.	<i>Business Central (AL):</i> • Create table MES Component Consumption. • Create function fetchBom() in codeunit MESMachineFetch. • Expose fetchBom() through codeunit MES Web Service. • Create page MES Component Consumption.

Continued on next page

ID	User Story	Acceptance Criteria
US-3.8	As an Operator or Supervisor, I need to report scrapped units with a reason code so that waste is accurately tracked.	<i>Flutter:</i> • Create model ComponentConsumption-Model. • Create MesComponentConsumptionService to consume the fetchBom() API. • Create MesComponentConsumption-Provider to manage BOM stream and refresh state. • Create widget RequiredComponent. • Implement BOM streaming and refresh flow in MesComponentConsumptionService and MesComponentConsumptionProvider. • Implement component status display and operation-based filtering in RequiredComponent.
		<i>Business Central (AL):</i> • Create table MES Operation Scrap. • Create function declareScrap() in codeunit MES Machine Write. • Expose declareScrap() through codeunit MES Web Service. • Create MES scrap Code API to fetch scrap codes from the standard Business Central Scrap Code. <i>Flutter:</i> • Create model MesScrapCode. • Create MesScrapService to consume the scrap code fetch and declareScrap() APIs. • Create MesScrapProvider to manage scrap code and scrap declaration state. • Create dialog DeclareScrapDialog.
US-3.9	As a Supervisor, I need to declare production on behalf of an operator so that output is attributed to the correct person.	<i>Business Central (AL):</i> • Add field Declared By to transactional tables MES Operation Progression and MES Component Consumption.

Continued on next page

ID	User Story	Acceptance Criteria
US-3.10	As an Operator or Supervisor, I need to scan components into an operation so that consumption is recorded against the BOM.	<i>Flutter:</i> • Create widget OperatorSelector. • Update DeclareProductionDialog to support supervisor declaration on behalf of an operator. • Connect OperatorSelector to MesUserProvider and pass onBehalfOfUserId to Machineorder-sProvider.declareProduction().
		<i>Business Central (AL):</i> • Create the function insertScans() in codeunit MES Machine Write. • Create the functions fetchAllItemBarcodes() and resolveBarcode() in codeunit MES Machine Fetch. • Expose the three function through codeunit MES Web Service. <i>Flutter:</i> • Create model ItemBarcodeModel. • Create MesBarcodeService to consume the APIs fetchAllBarcodes(), insertScans(), and resolveBarcode(). • Create MesBarcodeProvider to manage barcode state and API calls. • Create widget ScannerWidget. • Create page BarcodeListPage. • Update widget RequiredComponent to integrate item scanning, search, filtering, and BOM refresh after scan submission. • Implement ondevice camera scanning via mobile_scanner in ScannerWidget

Continued on next page

ID	User Story	Acceptance Criteria
US-3.11	As an Operator or Supervisor, I need to print a label for the active production order so that produced parts are correctly identified.	<i>Flutter:</i> • Create page PrintLabelPage. • Update ActionButtonsContainer to integrate the <i>Print Label</i> action. • Implement label generation, preview, print flow, and orientation toggle in PrintLabelPage.
US-3.12	As an Operator or Supervisor, I need a label printed for each declared unit so that individual pieces carry traceable information.	<i>Flutter:</i> • Create page DeclarationLabelPage. • Implement declaration label generation, preview, orientation toggle, and per-unit PDF export in DeclarationLabelPage.

5.2 Business Rules

Table 5.2. Business Rules

ID: Business Rule	Description
Production Execution Rules	
BR-EXEC-001: Single Running Operation per Machine	A machine may only have one operation in the <i>Running</i> state at any given time. Before starting or resuming an operation, <code>EnsureNoRunningOperation()</code> queries all MES Operation Execution records for the machine and verifies that none of their latest MES Operation Status records has the status <i>Running</i> . A violation triggers an error with the conflicting order and operation numbers included in the message.
BR-EXEC-002: Declared Quantity Cannot Exceed Remaining Quantity	During <code>TryDeclareProduction()</code> , the sum of <code>TotalProducedQuantity</code> from the latest progression record and the new input is compared to <code>MESExecution."OrderQuantity"</code> . If the sum exceeds the order quantity, an error is raised and no record is inserted.

Continued on next page

ID: Business Rule	Description
BR-EXEC-003: State Machine for Operation Status	Operation status transitions are strictly enforced: • <i>Running</i> → <i>Paused</i> (via <code>TryPauseOperation</code>). • <i>Paused</i> → <i>Running</i> (via <code>TryResumeOperation</code>). • <i>Running</i> or <i>Paused</i> → <i>Finished</i> or <i>Cancelled</i> (via <code>TryCloseOperation</code>). • <i>Finished</i> or <i>Cancelled</i> → any other state (blocked: <code>TryCloseOperation</code> raises an error). Each transition inserts a new immutable MES Operation Status record; no records are ever updated or deleted, providing a full audit trail.
BR-EXEC-004: Machine Status is Derived from Operation Status	MES Machine Status records are inserted (never updated) whenever an operation transition occurs: • Start / Resume → insert <i>Working</i> status with the current production order number. • Pause / Finish / Cancel → insert <i>Idle</i> status with no production order. The machine list display reads the most recent record for each machine (sorted by Last Updated At descending).
BR-EXEC-005: Only Released Orders Can Be Started	<code>TryStartOperation()</code> filters Prod. Order Routing Line with <code>Status = Released</code>. Planned and Firm Planned lines are visible in the Flutter Orders tab but their Start Order button is disabled (<code>canStart = order.status == 'Released'</code>).
Scrap Declaration Rules	
BR-SCRAP-001: Scrap Requires Active Operation	• Scrap can only be declared when the operation status is <i>Running</i>. • The <i>Report Reject</i> button is disabled for <i>Paused</i>, <i>Finished</i>, and <i>Cancelled</i> states.
BR-SCRAP-002: Reason Code Mandatory	• A scrap reason code must be selected before submission. • Codes are fetched from the standard Business Central Scrap Code table and cached in <code>MesScrapProvider</code> after the first load.
Proxy Declaration Rules	

Continued on next page

ID: Business Rule	Description
BR-PROXY-001: Supervisor Attribution	• Only users with the Supervisor role see the operator selector in the declare production dialog. • The selected operator's User Id is written to the Declared By field; the session User Id is written to Operator Id as the executing party. • If no operator is selected, both fields contain the supervisor's User Id.
Barcode Scanning Rules	
BR-SCAN-001: Internal Barcode Priority	• If a scanned DataMatrix string matches the proprietary pipe-separated format, it is parsed locally without a server round-trip. • External barcodes that do not match this format are resolved via <code>resolveBarcode()</code> using the Item Identifier table.
BR-SCAN-002: Duplicate Scan Merging	• Scanning the same item twice increments the quantity on the existing row in the pending scan list rather than adding a duplicate entry. • The operator can adjust the quantity further using the +/- controls before confirming.
Bill of Materials Rules	

Continued on next page

ID: Business Rule	Description
BR-BOM-001: Routing Link Code Scoping	When an order's components have Routing Link Codes assigned, <code>fetchBom()</code> uses the following three-condition filter to determine which components to show for a given operation: • Include: components whose Routing Link Code matches the current operation's code (<code>belongsToThisOperation = true</code>). • Include: components with an empty Routing Link Code when at least one component in the order has a routing link (shown as shared, <code>belongsToThisOperation = false</code>). • Exclude: components whose Routing Link Code is non-empty and differs from the current operation's code (they belong to a different operation). When no component in the order has a Routing Link Code, all components are shown as operation-specific.
Traceability Rules	
BR-TRACE-001: Immutable Audit Records	MES Operation Status, MES Operation Progression, MES Machine Status, and MES Component Consumption records are insert-only. No update or delete operations are performed on these tables outside of testing pages, ensuring a complete chronological audit trail for every machine, order, and operator action.
BR-TRACE-002: Operator Identity on Every Record	All progression, status, and consumption records stamp the <code>Operator Id</code> using the BC session <code>UserId</code> at insert time. The <code>fetchProductionCycles()</code> endpoint resolves first and last name from the <code>Employee</code> table for display.

5.3 Design

5.3.1 Use Case Diagram

This sprint adds production execution actors and interactions centred on the Operator and Supervisor roles. The primary use cases are: Declare Production, Pause Operation, Resume Operation, Finish Operation, Cancel Operation, View BOM Components, View Operation Detail, View Machine History, and Change Password.

The Supervisor role is a superset of Operator: both roles can declare production, pause, resume, and view detail; only the Supervisor may close (finish or cancel) a production order in the in-

tended business flow, although the current implementation enforces this at the UI layer through the `canCloseProductionOrder` flag rather than at the API level. The resulting use case diagram is shown in Figure 5.1.



Figure 5.1. UML Use Case Diagram — Sprint 3

5.3.2 Textual Use Case Description

Table 5.3. Textual Use Case Description — Declare Production

Use Case Name	Declare Production
Primary Actor	Operator / Supervisor
Preconditions	• The user is authenticated and navigated to an Operation Detail page. • The operation status is <i>Running</i> or <i>Paused</i>. • The produced quantity is below the order quantity.
Postconditions	• A new MES Operation Progression record is inserted. • The production chart and cycle table update via the refresh stream. • The progress percentage and produced quantity in the info container update.
Main Scenario	1. The user taps <i>Declare Production</i> in the Quick Actions panel. 2. The DeclareProductionDialog opens showing the remaining quantity. 3. The user enters a positive integer not exceeding the remaining quantity. 4. The user taps <i>Submit</i>. 5. The system POSTs the declaration to <code>/MESWebService_declareProduction</code>. 6. The ERP validates the quantity and inserts the progression record. 7. The dialog closes and the refresh stream triggers a live data update.
Alternative Flow	• <i>Step 3, invalid quantity</i>: client-side validation shows an inline error; submission is blocked. • <i>Step 6, ERP error</i>: the server error message is displayed in the dialog; the record is not inserted.

Table 5.4. Textual Use Case Description — Finish / Cancel Operation

Use Case Name	Finish / Cancel Operation
Primary Actor	Supervisor
Preconditions	• The user is authenticated with an active session. • The operation is in <i>Running</i> or <i>Paused</i> state.
Postconditions	• A <i>Finished</i> or <i>Cancelled</i> status record is inserted. • The execution end time is stamped. • An Idle machine status record is inserted. • The user is navigated back to the machine page.
Main Scenario	1. The user taps <i>Finish Production Order</i> (if complete) or <i>Cancel Production Order</i> (if incomplete). 2. A confirmation dialog is displayed; the user confirms. 3. The system POSTs to <code>/MESWebService_finishOperation</code> or <code>cancelOperation</code>. 4. The ERP validates, inserts the status record, stamps the end time, and inserts an Idle machine status. 5. Flutter calls <code>Navigator.pop()</code> to return to the machine page.

5.3.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for the production and operation-lifecycle flows introduced in Sprint 3.

Figure 5.2 depicts the production declaration flow. The operator submits a quantity; the Flutter client validates the input locally, then forwards the request to the AL layer, which re-validates the token (SD-02), checks that the declared amount is positive and does not exceed the remaining order quantity, and — on success — inserts a new immutable *MES Operation Progression* record and records the relevant user interaction entries. When a Supervisor declares production on behalf of an operator, the AL layer additionally verifies that the two users share at least one work centre before proceeding.

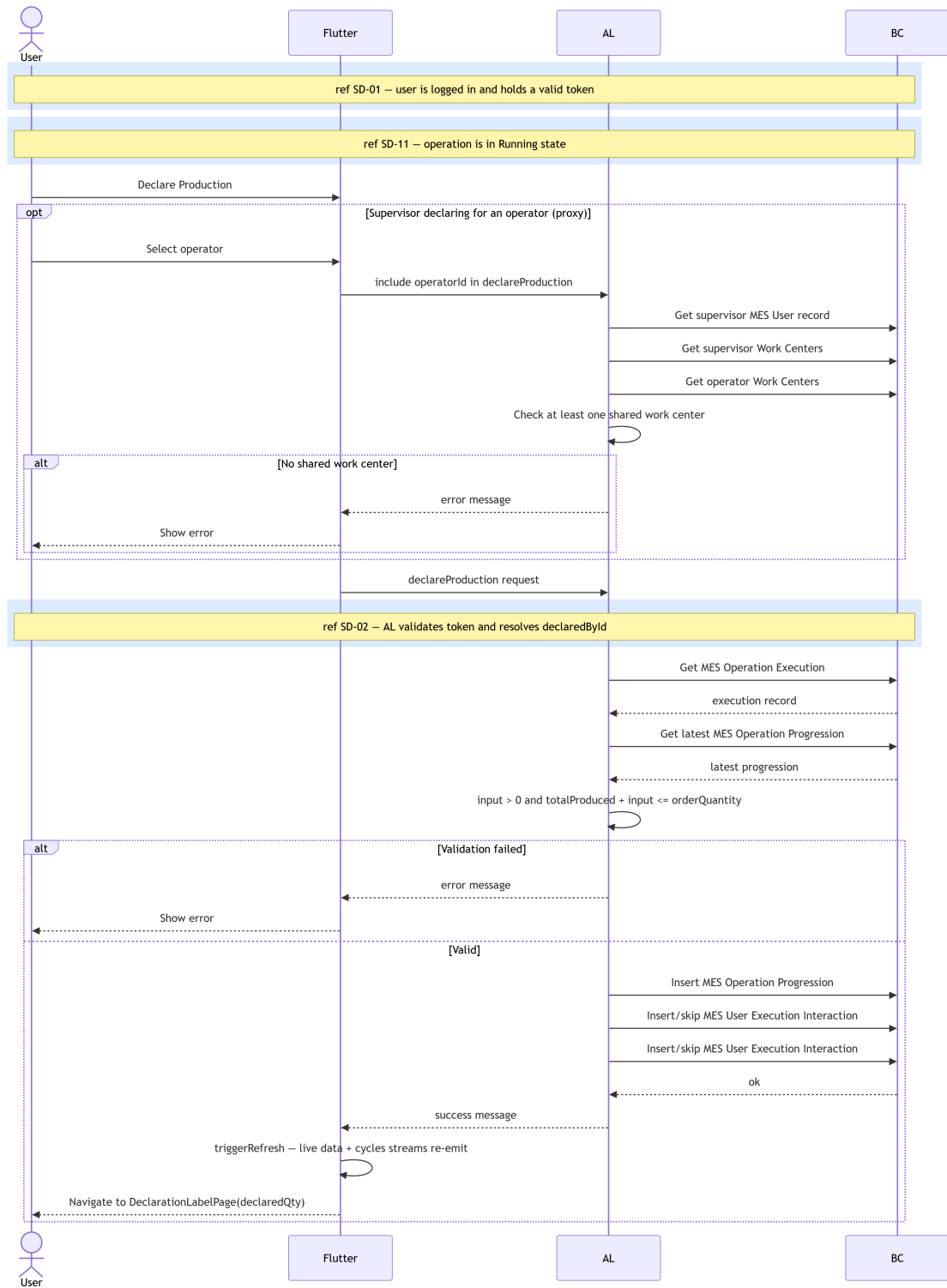


Figure 5.2. SD-12: Declare Production.

Figures 5.3 and 5.4 present the two supplementary data-retrieval flows that run concurrently alongside the main operation detail view. Both are polling streams that re-emit on demand or when a state-change trigger is received from the Flutter client.

Figure 5.3 describes the bill-of-materials retrieval flow. The AL layer resolves the routing link

code from the active *ProdOrderRoutingLine*, filters the production order components by that link, aggregates the quantity already scanned per item, and resolves per-unit quantities from the *ItemUnitOfMeasure* table before returning the assembled BOM rows to the Flutter client.

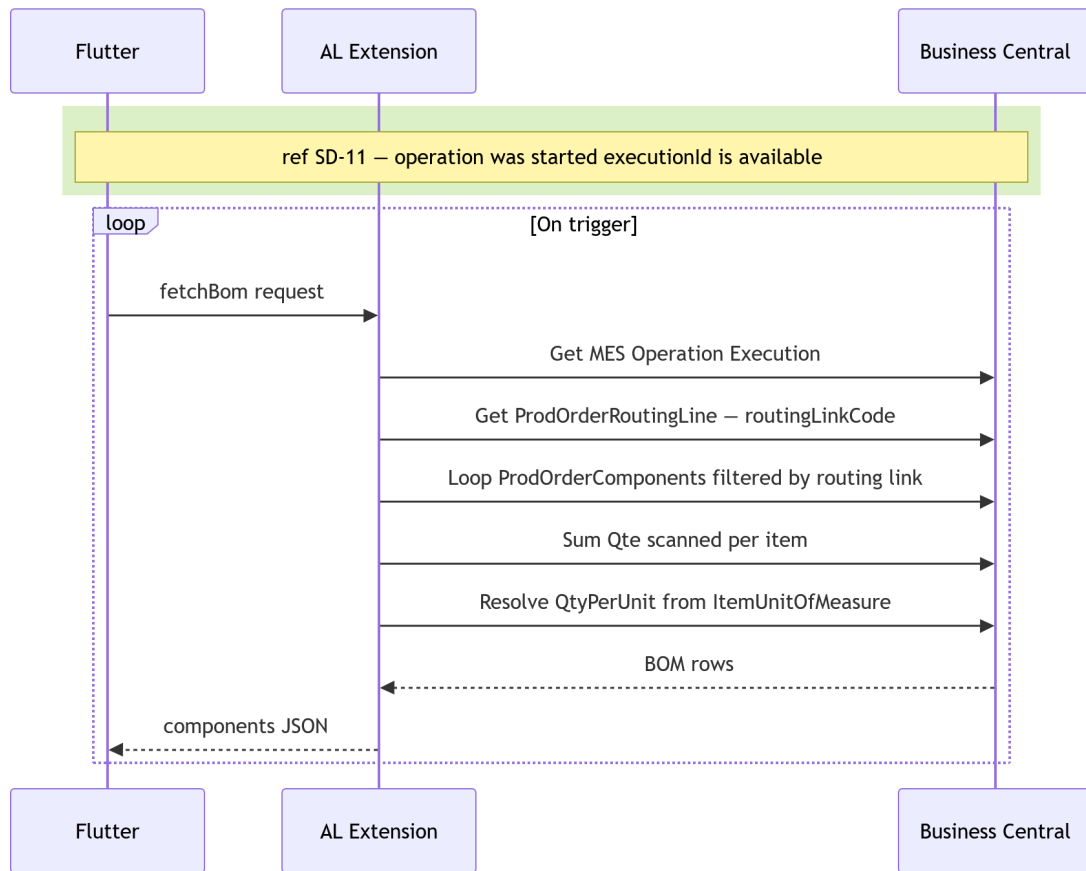


Figure 5.3. SD-21: Fetch Bill of Materials.

Figure 5.4 illustrates the production-cycle retrieval flow. The AL layer fetches the full progression history for the operation, enriches each entry with the name of the user who declared it, and returns the assembled cycle list to the Flutter client.

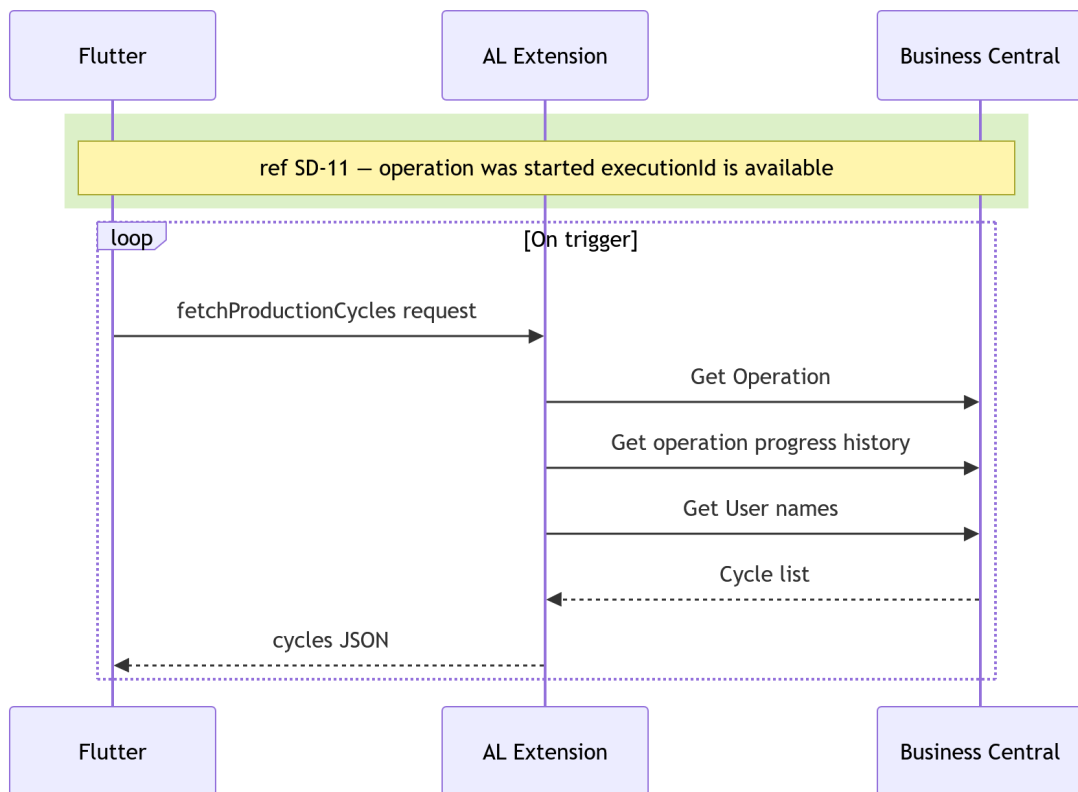


Figure 5.4. SD-22: Fetch Production Cycles.

Figures 5.5 through 5.7 illustrate the remaining operation lifecycle transition flows: pause, resume, finish, and cancel. All four follow the same structural pattern as the start operation sequence (SD-11): the Flutter frontend POSTs to the corresponding MES Web Service endpoint; the AL backend validates the current state via the Try* family of functions; inserts a new immutable status record in Business Central; and returns a structured success or error response to the client.

Figure 5.5 describes the pause flow. The AL layer confirms that the operation is currently in the *Running* state before inserting a *Paused MES Operation State* record and setting the machine status back to *Idle*.

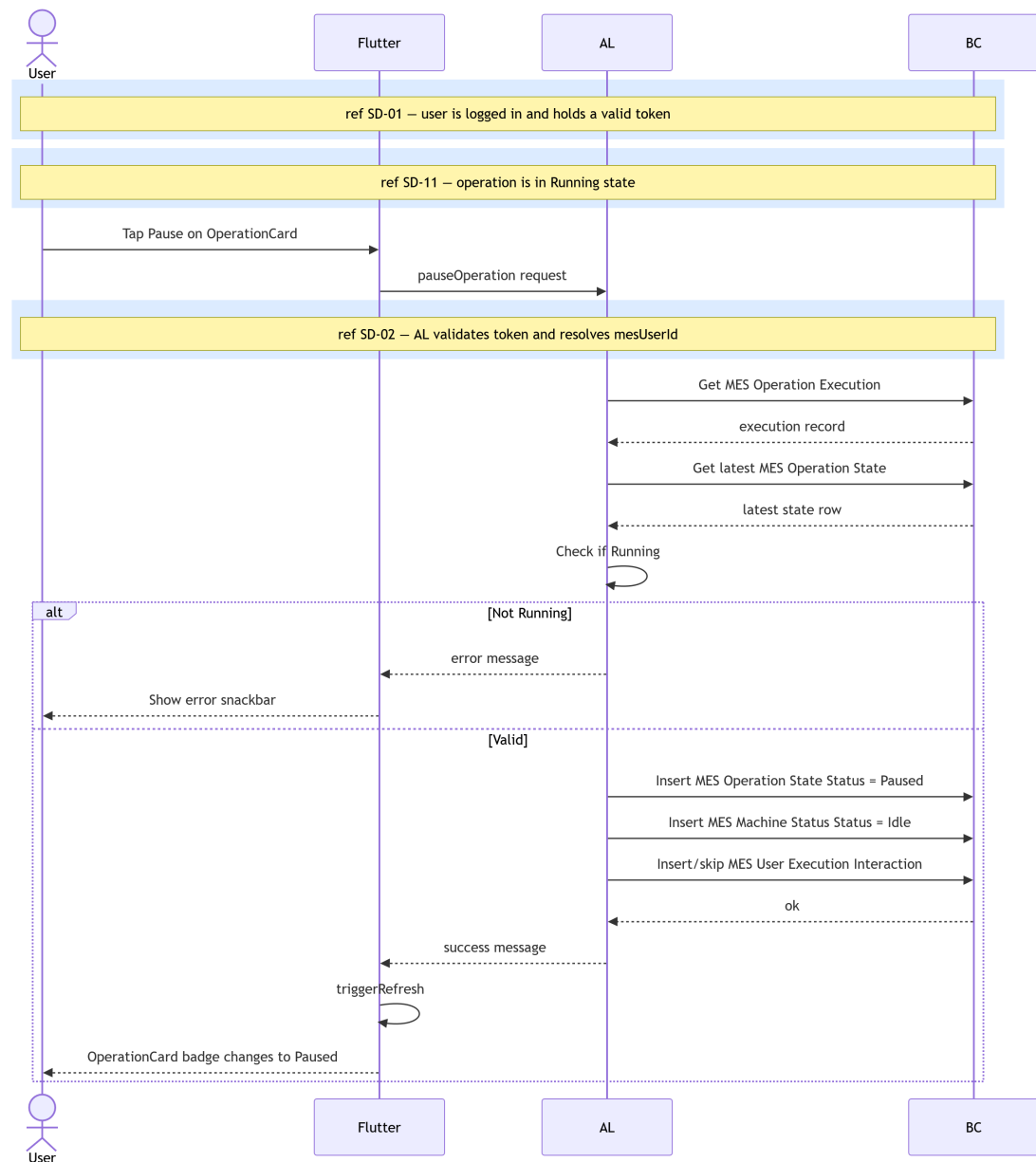


Figure 5.5. SD-13: Pause Operation.

Figure 5.6 presents the resume flow. In addition to confirming that the operation is in the *Paused* state, the AL layer verifies that no other operation is currently running on the same machine before inserting a *Running* state record and marking the machine as *Working*.

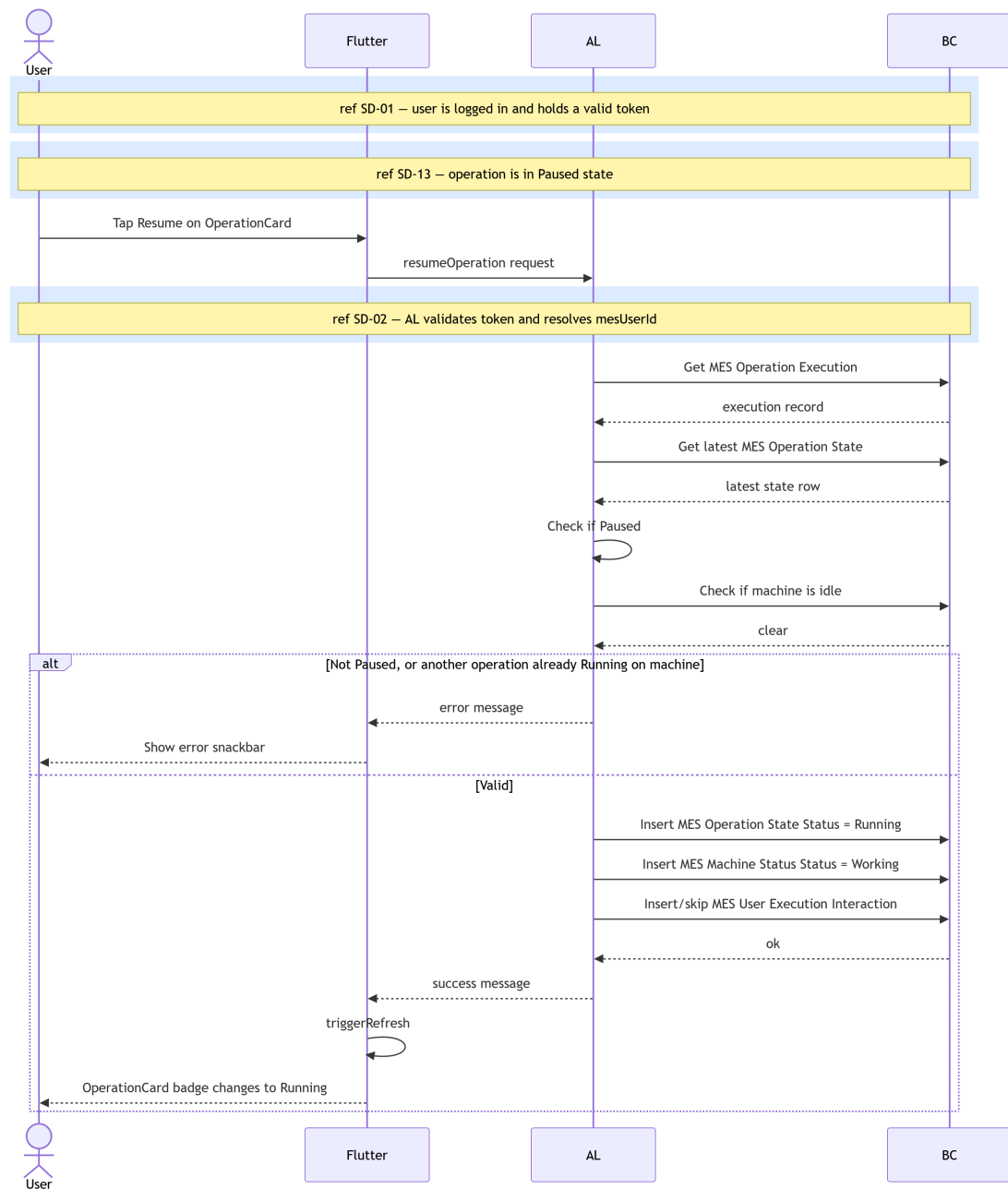


Figure 5.6. SD-14: Resume Operation.

Figure 5.7 covers the finish and cancel flows, which share a single endpoint family. The AL layer rejects the request if the operation is already in a terminal state; otherwise it inserts the appropriate closed-state record, stamps the execution end time, and resets the machine status to *Idle*. The closed operation then appears in the history view rather than the active operations list.

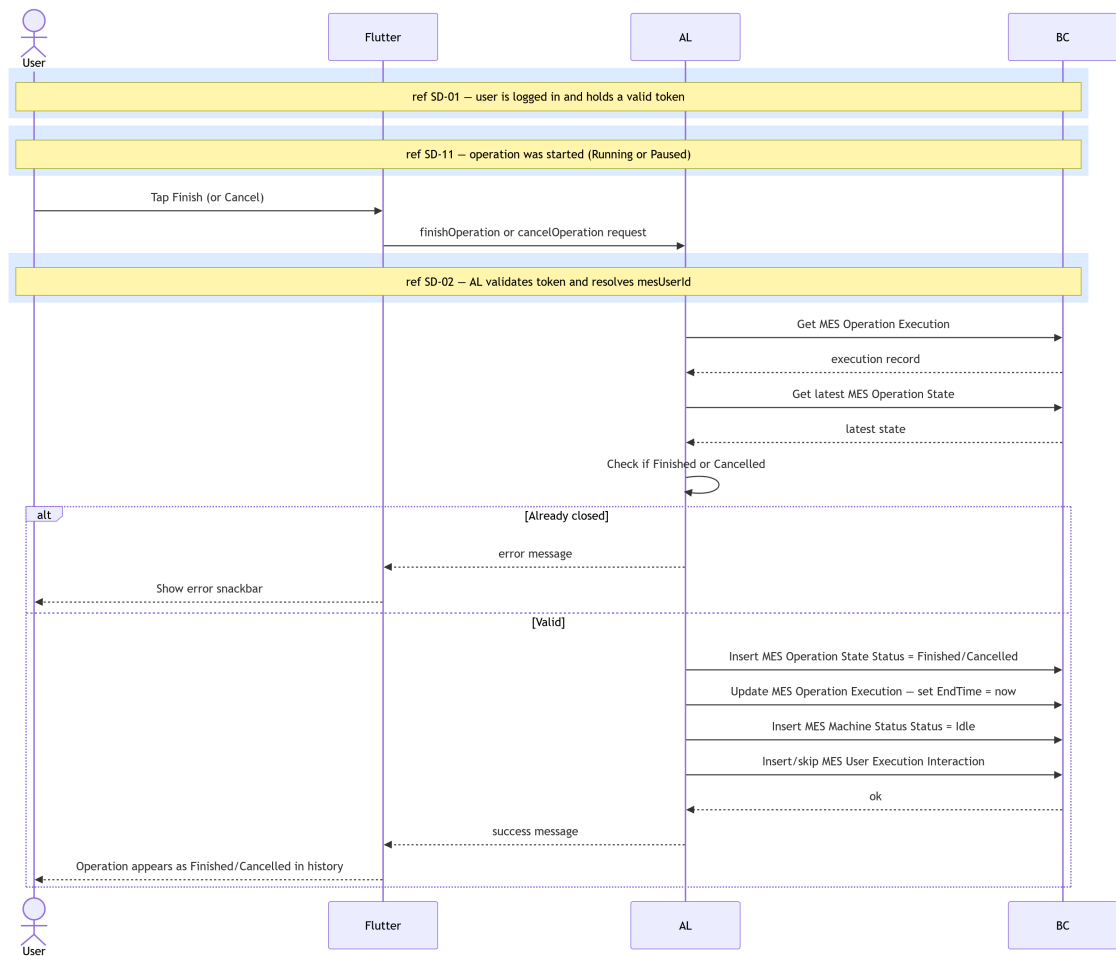


Figure 5.7. SD-15: Finish/Cancel Operation.

Figure 5.8 describes the scrap declaration flow. Before inserting the scrap record, the AL layer verifies that the operation has not already been finished, cancelled, or paused, and that the referenced scrap code exists in Business Central. On success, it inserts a *MES Operation Scrap* entry, records user interaction, and appends a new *MES Operation Progression* row so that the scrap quantity is reflected in the live data stream.

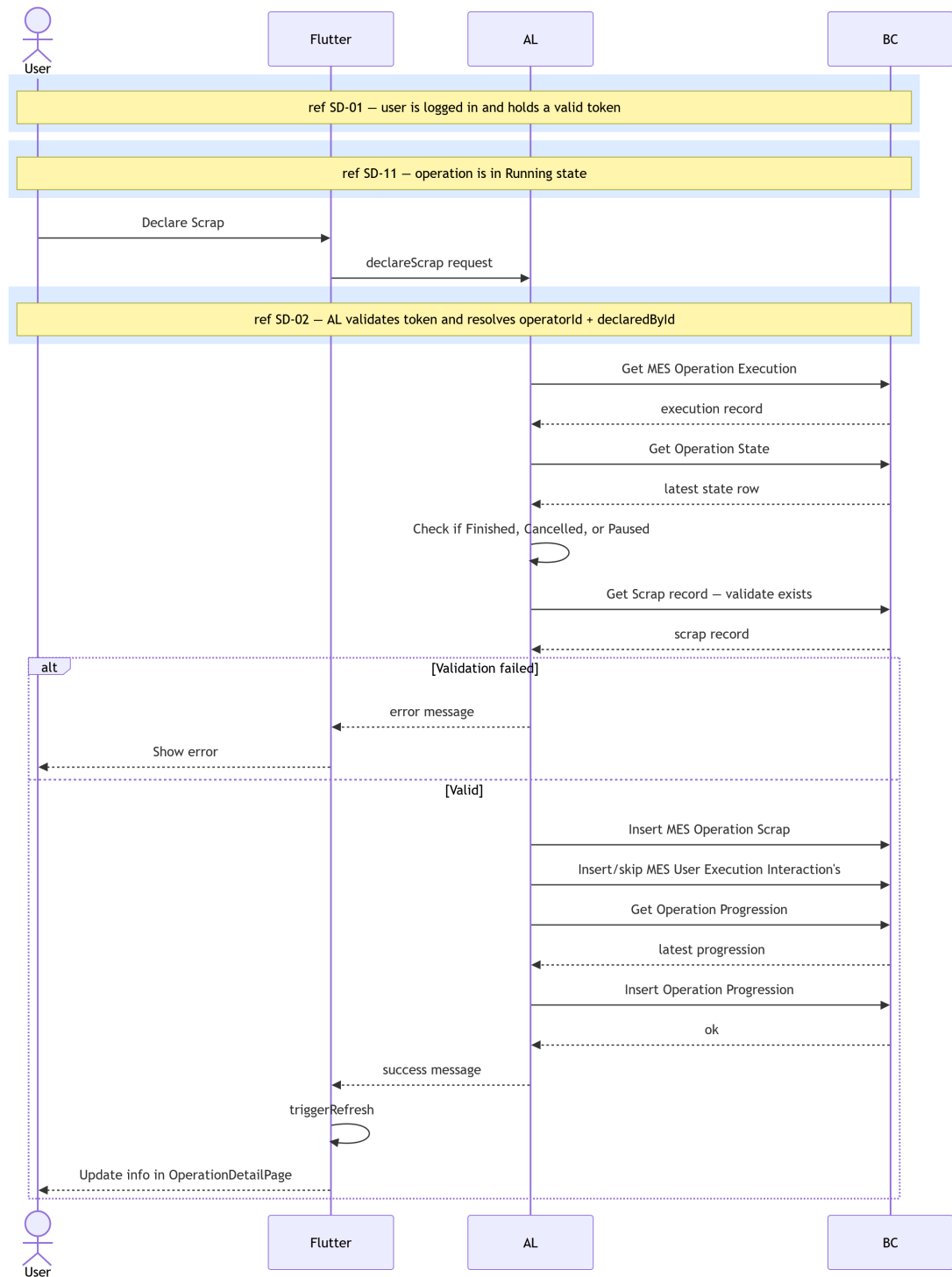


Figure 5.8. SD-16: declare scrap.

Figure 5.9 illustrates the component scanning flow, which operates in two steps. First, the Flutter client resolves each scanned barcode into an item record via a lightweight `resolveBarcode` call and accumulates the results locally. Once the operator confirms the batch, a single `insertScans` request is sent to the AL layer, which inserts one *MES Component Consumption* record per scanned item and records the associated user interaction entries.

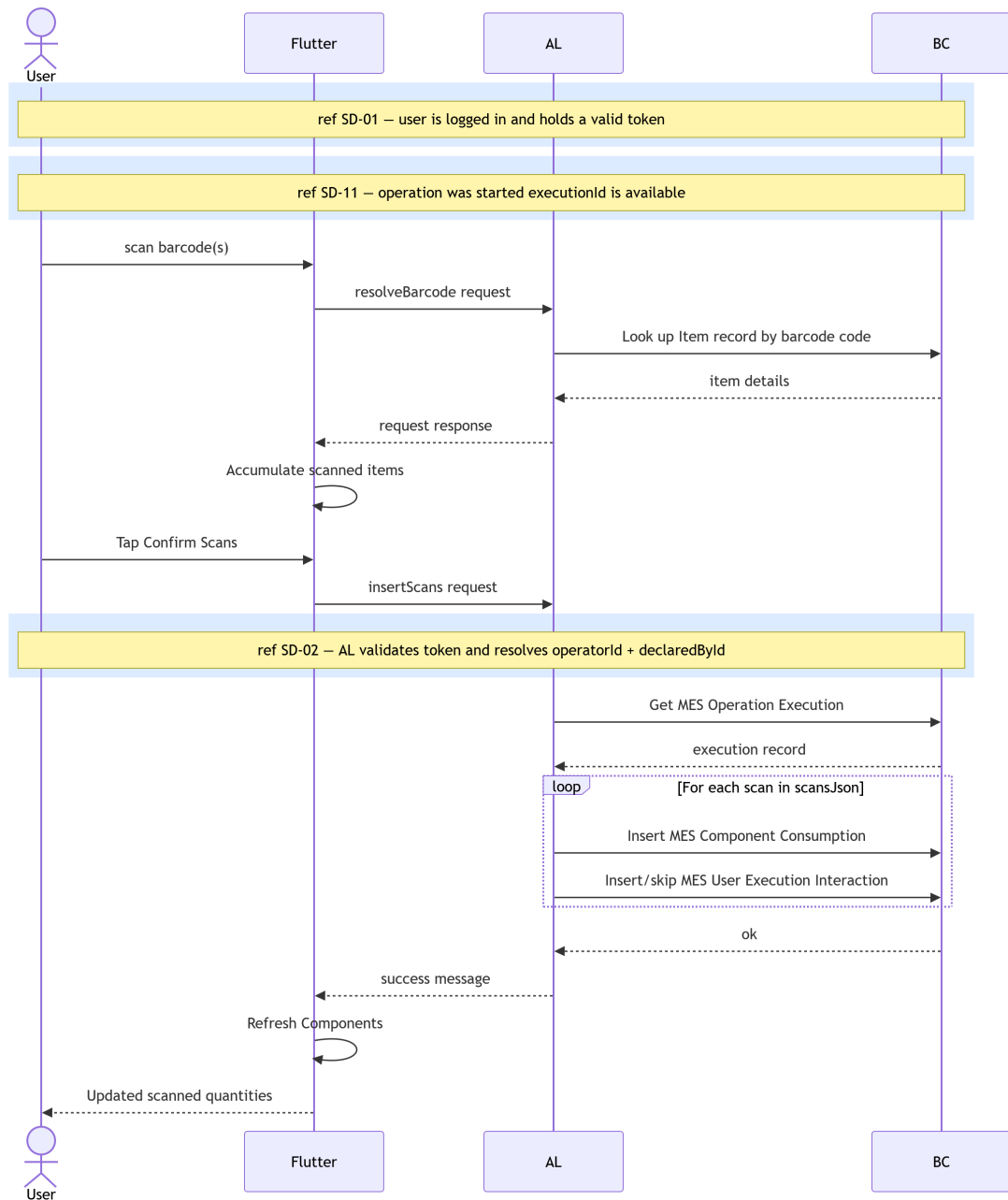


Figure 5.9. SD-17: scan component.

Figure 5.10 presents the live operation data retrieval sequence. When the user navigates to the *OperationDetailPage*, three *StreamBuilder* instances mount simultaneously: one polling the current operation status and quantities from Business Central (SD-18), one consuming the production-cycle stream (SD-22), and one consuming the BOM stream (SD-21). The Flutter client merges all three data sources and renders the full detail view. If the operation is finished, cancelled, or not yet started, the live-data stream returns an empty response and the client falls back to the history view.

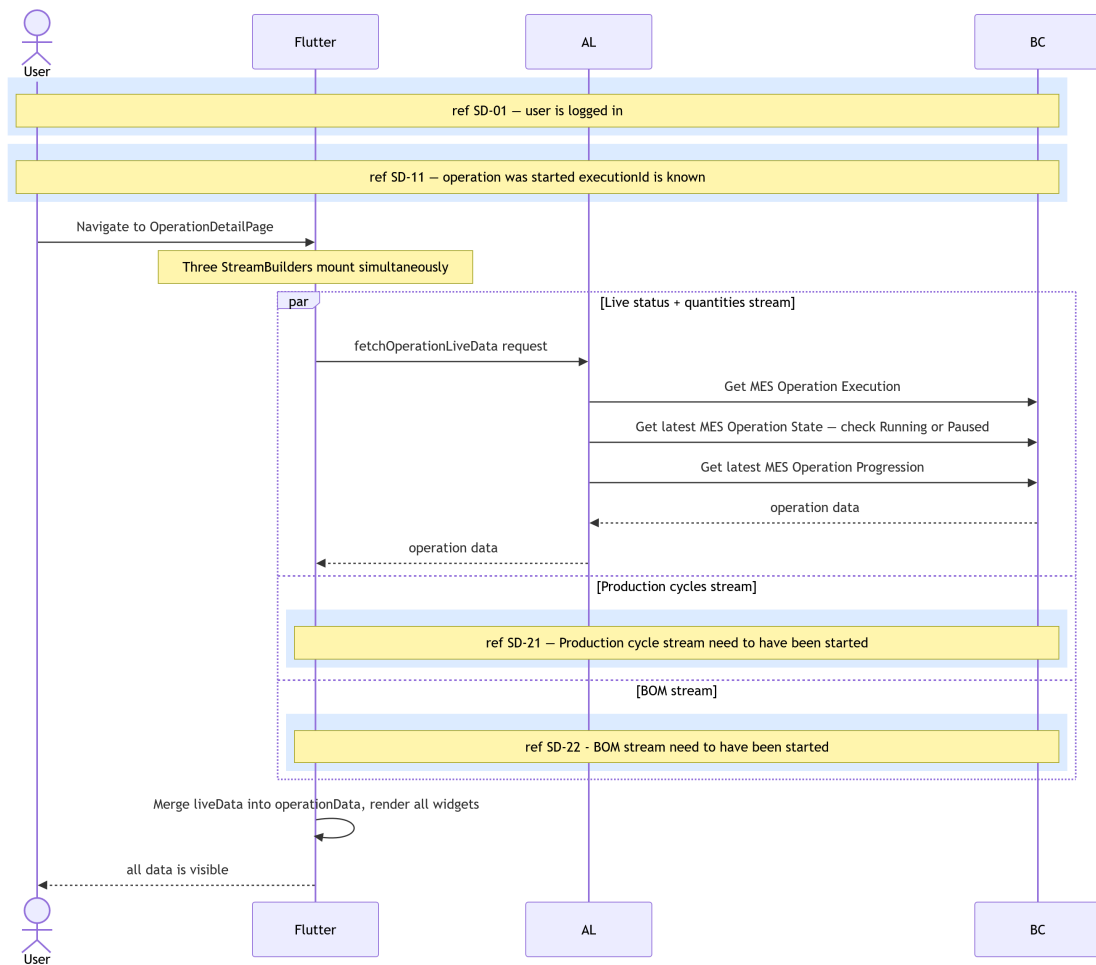


Figure 5.10. SD-18: Fetch Operation Live Data.

5.3.4 Class Diagram

The UML Class Diagram (Figure 5.11) extends the Sprint 2 domain model with three new tables. MES Operation Execution (50110) is the central anchor record linking a machine, production order, and operation to its full lifecycle; it holds item description, order quantity, and start/end timestamps. MES Operation Progression (50109) is an append-only table of production cycle declarations, each referencing an execution and accumulating TotalProducedQuantity from the previous cycle. MES Component Consumption (50111) records scrap and barcode scan events, also referencing an execution. Both progression and consumption records are child tables of the execution record, forming a hub-and-spoke traceability model.

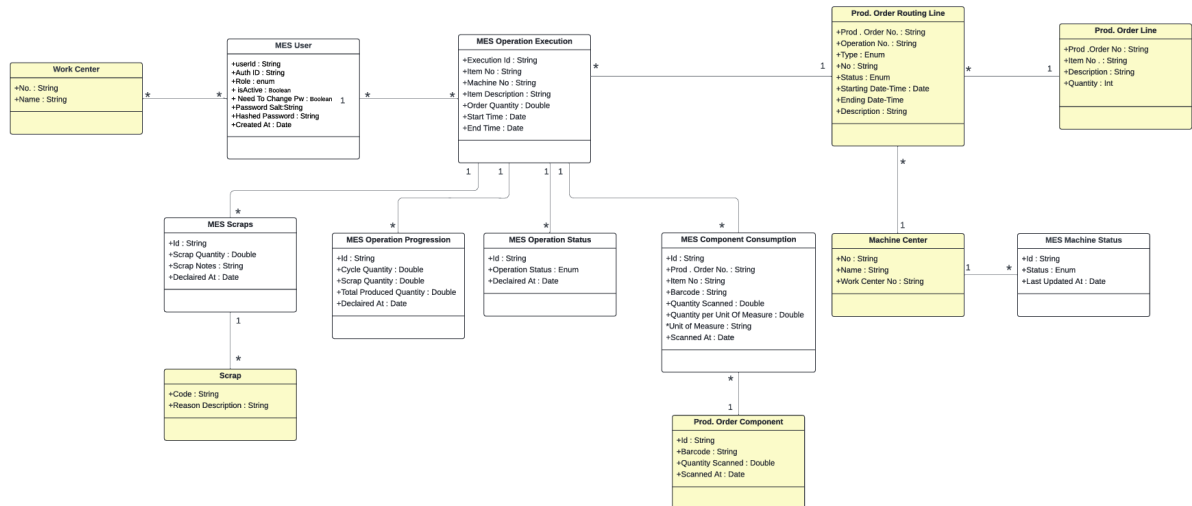


Figure 5.11. UML Class Diagram — Production Execution & Traceability.

5.4 Implementation

5.4.1 Backend Implementation

Tables and Entities

Table 5.5. MES Operation Execution (Table 50110) — Field Dictionary

Name	Type	Description
Execution Id	Code[50]	GUID-based primary key; auto-generated on insert.
Machine No	Code[20]	Foreign key to Machine Center.
Prod Order No	Code[20]	Foreign key to Prod. Order Routing Line.
Operation No	Code[10]	Operation identifier from the routing line.
Item No	Code[20]	Copied from Prod. Order Line on execution creation.
Item Description	Text[100]	Copied from Prod. Order Line for denormalized display.
Order Quantity	Decimal	Copied from Prod. Order Line; used for progress calculation.
Start Time	DateTime	Set by OnInsert trigger to CurrentDateTime().
End Time	DateTime	Stamped when status transitions to Finished or Cancelled.

Table 5.6. MES Operation Progression (Table 50109) — Field Dictionary

Name	Type	Description
Id	Code[50]	GUID-based primary key; auto-generated on insert.
Execution Id	Code[50]	Foreign key to MES Operation Execution.
Cycle Quantity	Decimal	Units declared in this single production cycle.
Scrap Quantity	Decimal	Units scrapped in this cycle (currently always 0).
Total Produced Quantity	Decimal	Running total: previous total + Cycle Quantity.
Operator Id	Code[50]	BC session UserId of the operator who declared.
Last Updated At	DateTime	Set by OnInsert trigger to CurrentDateTime().

Table 5.7. MES Component Consumption (Table 50111) — Field Dictionary

Name	Type	Description
Id	Code[50]	GUID-based primary key; auto-generated on insert.
Execution Id	Code[50]	Foreign key to MES Operation Execution.
Prod Order No	Code[50]	Foreign key to Prod. Order Component.
Item No	Code[50]	Item identifier of the consumed component.
Barcode	Code[50]	Barcode scanned to register the component.
Unit of Measure	Code[50]	Unit of measure code from the component definition.
Quantity Scanned	Decimal	Raw quantity read from the barcode scan.
Quantity Consumed	Decimal	Quantity actually applied to the production order.
Operator Id	Code[50]	BC session UserId of the operator who scanned.
Scanned At	DateTime	Set by OnInsert trigger to CurrentDateTime().

REST API and Web Services Implementation

All endpoints are exposed as ODataV4 unbound actions through MES Web Service (codeunit 50126), reachable at:

POST <baseUrl>/ODataV4/MESWebService_<ProcedureName>?company=<companyId>

Table 5.8 lists all endpoints added in this sprint.

Table 5.8. New API Endpoints — Sprint 3

Endpoint	Key Parameters	Response
startOperation	prodOrderNo, operationNo, machineNo	{value: bool, message?}
declareProduction	machineNo, prodOrderNo, operationNo, input	{value: bool, message?}
pauseOperation	machineNo, prodOrderNo, operationNo	{value: bool, message?}
resumeOperation	machineNo, prodOrderNo, operationNo	{value: bool, message?}
finishOperation	machineNo, prodOrderNo, operationNo	{value: bool, message?}
cancelOperation	machineNo, prodOrderNo, operationNo	{value: bool, message?}
fetchOperationsStatusAndProgress	machineNo, fetchFinished	JSON array of operation progress objects
fetchOperationLiveData	machineNo, prodOrderNo, operationNo	JSON array (0 or 1 element)
fetchProductionCycles	machineNo, prodOrderNo, operationNo	JSON array of cycle records
fetchBom	prodOrderNo, operationNo	JSON array of BOM component records
ChangePassword	token, oldPassword, newPassword	{success: bool, message}

5.4.2 Frontend Implementation

This sprint introduces a key reactive pattern: a shared `StreamController<void>.broadcast()` in `MachineordersProvider` and `MesComponentConsumptionProvider`. When any write action (declare, pause, resume, finish, cancel) succeeds, `triggerRefresh()` adds an event to the controller. All live data streams (`fetchOperationLiveDataStream`, `streamProductionCycles`, `streamBom`) are implemented as `async*` generators that yield on both the initial fetch and on each event from the shared broadcast stream, enabling coordinated, push-driven UI updates without polling.

Provider Architecture

Table 5.9. Provider Responsibilities — Sprint 3 Additions

Provider	Responsibility
MachineordersProvider	Owns write operations (start, declare, pause, resume, finish, cancel) and their refresh controller; exposes all live data streams.
MesComponentConsumption Provider	Exposes <code>getBomStream()</code> combining service fetch with an independent refresh controller.
AuthProvider	Adds <code>changePassword()</code> and <code>adminSetPassword()</code> ; manages <code>needsPasswordChange</code> flag in the in-memory session.

User Interface

Figures 5.12 through 5.15 illustrate the *Operation Detail Page* across its four possible states: Running, Paused, Finished, and Cancelled. Each view adapts its action buttons and status badge to the current state, while the production chart, cycle table, and BOM list remain consistently visible. On desktop, the page uses a two-column layout with order information and production data on the left, and the *Quick Actions* panel on the right; on mobile, these sections stack vertically. Action button availability is state-dependent: *Declare Production* and *Report Reject* are disabled in the Paused, Finished, and Cancelled states; the close button triggers a *Finish Production Order* or *Cancel Production Order* action depending on whether the progress has reached 100%; and *Print Label* is enabled only once the operation is finished.

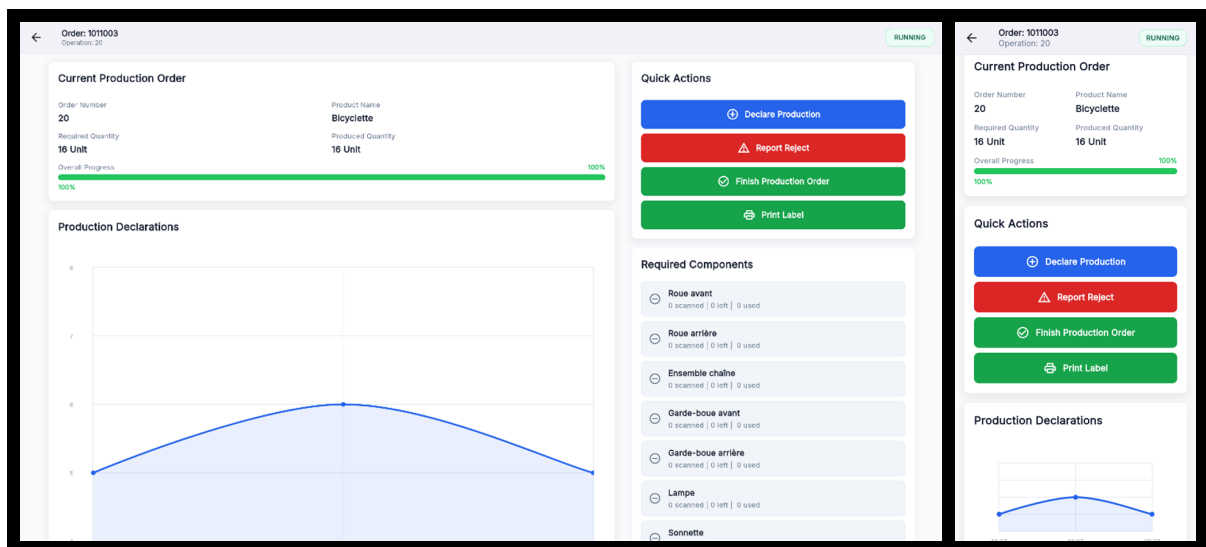


Figure 5.12. *Operation Detail Page* (Running state) — desktop and mobile views.

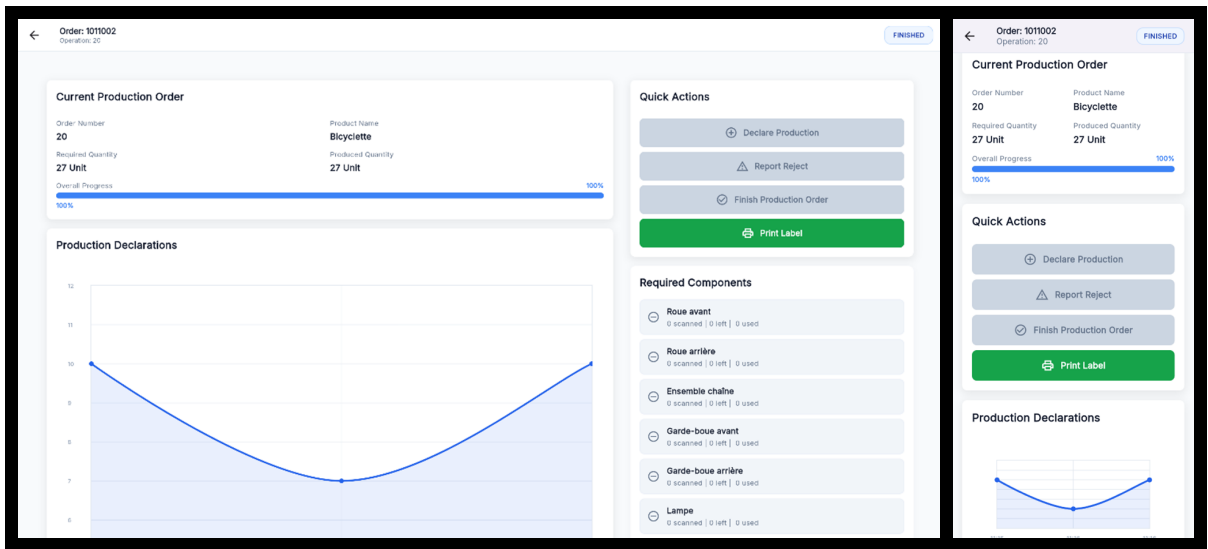


Figure 5.13. Operation Detail Page (Finished state) — desktop and mobile views.

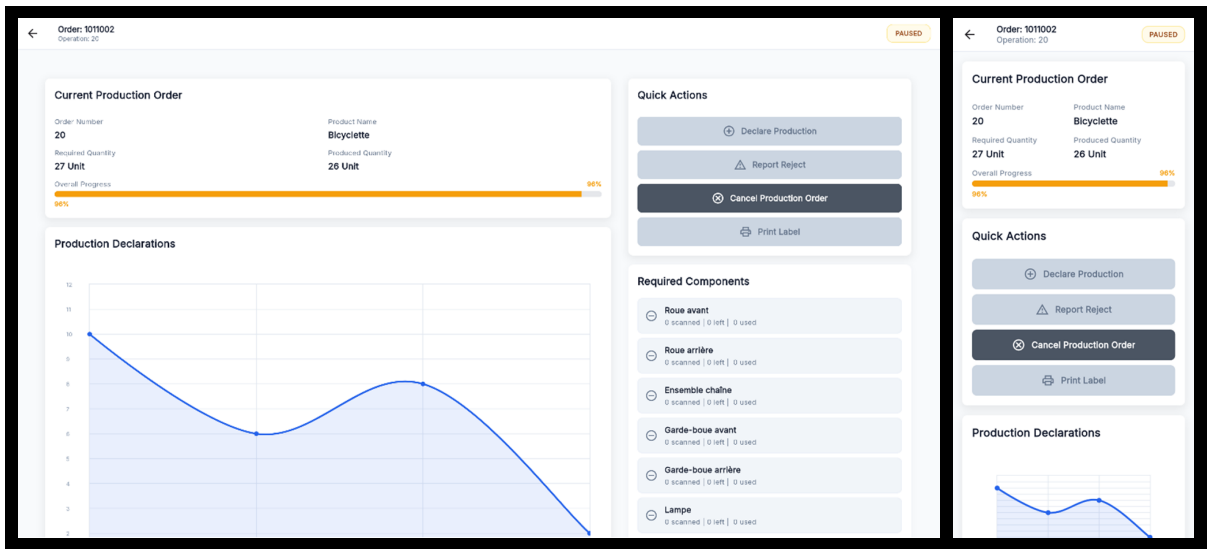


Figure 5.14. Operation Detail Page (Paused state) — desktop and mobile views.

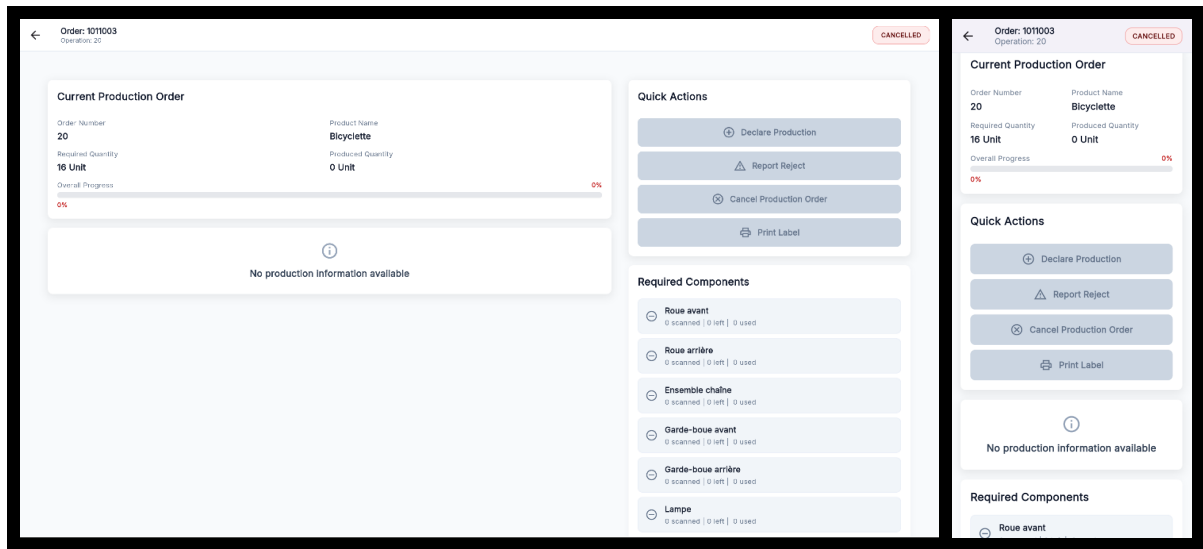


Figure 5.15. Operation Detail Page (Cancelled state) — desktop and mobile views.

5.5 Test and Validation

5.5.1 Integration test

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 5.16 shows a test case for the `/fetchBom` endpoint.

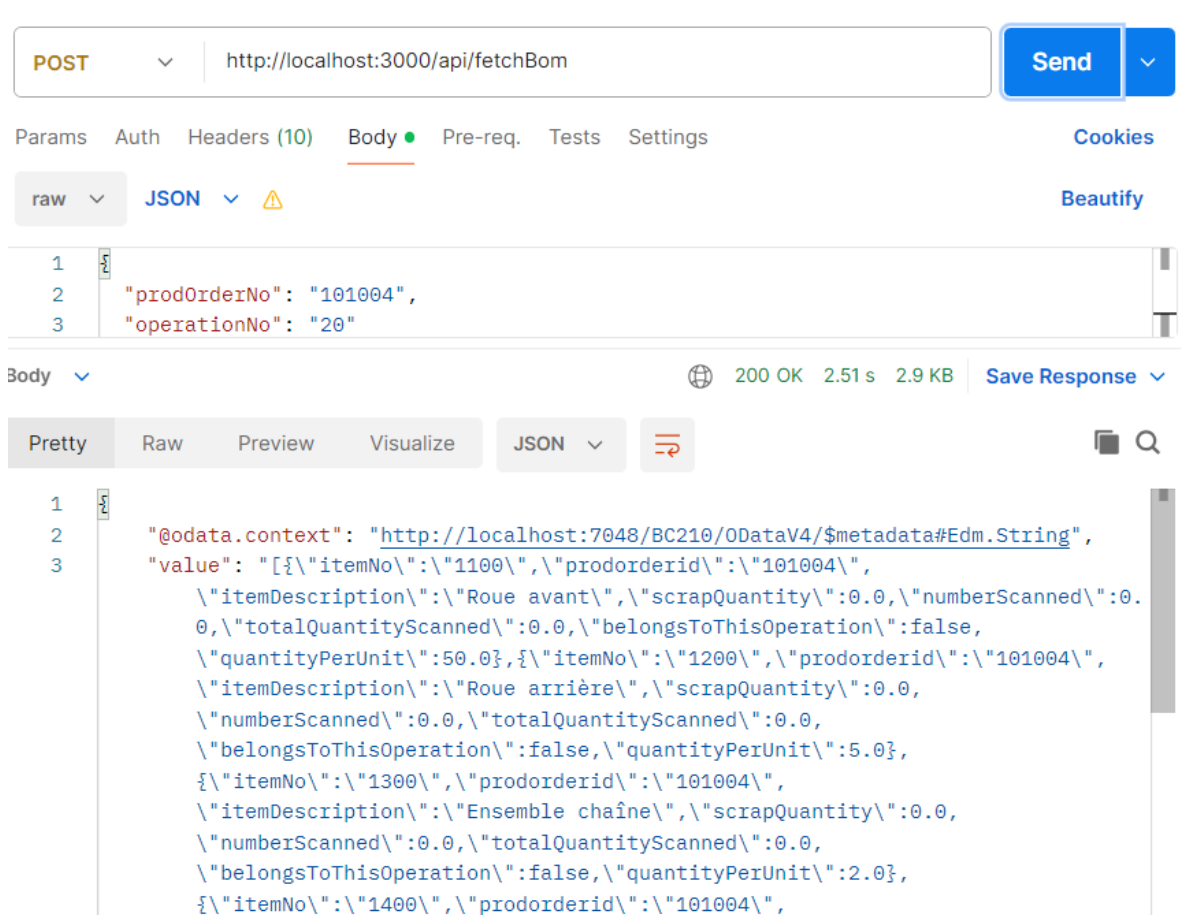


Figure 5.16. Postman endpoint test — fetchBom.

Conclusion

This sprint delivered the complete shop-floor execution loop for the MES system. Operators can now start, pause, resume, and close production operations, declare production output cycle by cycle, track cumulative progress through real-time charts and paginated tables, inspect the Bill of Materials, scan physical components via DataMatrix barcodes, declare scrapped units with reason codes, and print traceable labels for both production orders and individual declared units. Supervisors gain the ability to declare production on behalf of a specific operator, with a full attribution audit trail. The reactive stream architecture ensures that all UI panels update in concert after every write action, providing a consistent and accurate view of the production floor without manual refreshes.

Chapter 6

Sprint 4 Administration, Printing & Scanning

Contents

Introduction	105
6.1 Sprint Backlog	105
6.2 Business Rules	106
6.3 Design	107
6.3.1 Use Case Diagram	107
6.3.2 Textual Use Case Description	108
6.3.3 Sequence Diagrams	109
6.3.4 Class Diagram	111
6.4 Implementation	112
6.4.1 Backend Implementation	112
Dashboard Aggregation	112
Activity Log Aggregation	112
API Endpoints Added in Sprint 4	112
6.4.2 Frontend Implementation	112
Shared HTTP Infrastructure	112
Provider Architecture	113
User Interface	113
6.5 Test and validation	115
6.5.1 Integration test	115
Conclusion	116

Introduction

This sprint delivers the administration layer, multi-language support, and in-app onboarding tutorials that complete the *MES* solution. Administrators receive a persistent navigation shell giving access to a machine performance dashboard, a paginated activity log, and a user role management interface. Cross-cutting quality-of-life features include full internationalisation with English, French, and Arabic support, automatic right-to-left layout for Arabic, and guided coach-mark tutorials shown once per screen on first use. No new Business Central tables are introduced in this sprint; all data is derived by querying and aggregating the tables created in Sprints 1 through 3.

6.1 Sprint Backlog

This sprint covers **Domain 4: Administration & Monitoring** and **Domain 5: UX & Accessibility**.

Table 6.1. Sprint 4 Story Backlog

ID	User Story	Tasks
US-4.1	As an Administrator, I need a performance dashboard for all machines so that I can monitor production health across the facility.	<i>Business Central (AL):</i> • Create function <code>fetchMachineDashboard()</code> in codeunit MES Machine Fetch. • Expose the function through the codeunit MES Web Service. <i>Flutter:</i> • Create model <code>MachineDashboardModel</code>. • Create <code>LogService</code> to consume the <code>fetchMachineDashboard</code> API and <code>Log-Provider</code> to manage its state. • Create page <code>MachineDashboardPage</code>. • Create widget <code>AdminMachineCard</code>. • Implement machine dashboard search and time-range filtering in <code>MachineDashboardPage</code>.
US-4.2	As an Administrator, I need a filterable, paginated log of all system actions so that I can audit operator and machine activity.	<i>Business Central (AL):</i> • Create function <code>fetchActivityLog()</code> in codeunit MES Machine Fetch. • Add the wrapper of the function to MES Web Service.

Continued on next page

ID	User Story	Tasks
		<i>Flutter:</i> • Create model ActivityLogModel. • Create LogService to consume the fetchActivityLog() API and LogProvider to manage its state. • Create page ActivityLogPage. • Implement activity log search, type filtering, time-range filtering, and pagination in ActivityLogPage.
US-4.3	As an Administrator, I need a persistent sidebar so that I can navigate between all admin sections without losing context.	<i>Flutter:</i> • Create page AdminPage. • Create widget SidebarItem. • Implement sidebar navigation and profile-page navigation in AdminPage.
US-4.4	As any user, I need the application available in English, French, and Arabic so that I can use it in my preferred language.	<i>Flutter:</i> • Integrate the easy_localization package. • Create translation JSON files for en, fr, and ar. • Create widget LanguageSelector. • Implement language selection in ProfilePage. • Add the EasyLocalization wrapper in main(). • Implement right-to-left layout support through MaterialApp.
US-4.5	As a first-time user, I need guided coach-mark tutorials so that I can learn the interface without external documentation.	<i>Flutter:</i> • Integrate the tutorial_coach_mark package. • Implement tutorial persistence using SharedPreferences. • Create MachineListTutorial, MachineDetailTabsTutorial, OperationDetailTutorial, and AdminDashboardTutorial. • Implement tutorial target styling and localisation.

6.2 Business Rules

Table 6.2. Business Rules

ID: Business Rule	Description
Dashboard and Log Rules	
BR-ADMIN-001: Dashboard Time-Range Scoping	• The hoursBack parameter filters all dashboard and activity log queries; it is applied server-side when querying the MES tables. • The default value is 24 hours; available options are exposed as hourOptions in LogProvider (e.g., 8 h, 24 h, 48 h, 7 days). • Changing the time-range selector triggers a new API call and replaces the displayed data.
Internationalisation Rules	
BR-I18N-001: RTL Support	• Selecting the Arabic locale (ar) sets the MaterialApp locale to ar; Flutter's Directionality widget handles right-to-left layout automatically without any per-widget configuration. • All UI strings are externalised in JSON translation files; no hard-coded string literals remain in the codebase. • The language preference is persisted between app restarts via the EasyLocalization storage layer.
Tutorial Rules	
BR-TUTORIAL-001: One-Time Display	• Each tutorial is shown exactly once per application install, gated by a SharedPreferences boolean flag keyed to the tutorial identifier (e.g., tutorial_machine_list_shown). • Once dismissed or completed, the flag is set to true and the tutorial is never shown again, even after the application is restarted.

6.3 Design

6.3.1 Use Case Diagram

The UML Use Case Diagram (Figure 6.1) covers the actors and interactions introduced in this sprint. The Administrator gains three new interactions: *View Machine Dashboard*, *View Activity Log*, and *Navigate Admin Shell*. All three actors (Operator, Supervisor, Administrator) share the *Select Language* and *View Tutorial* use cases, which are cross-cutting accessibility features

available on every screen.

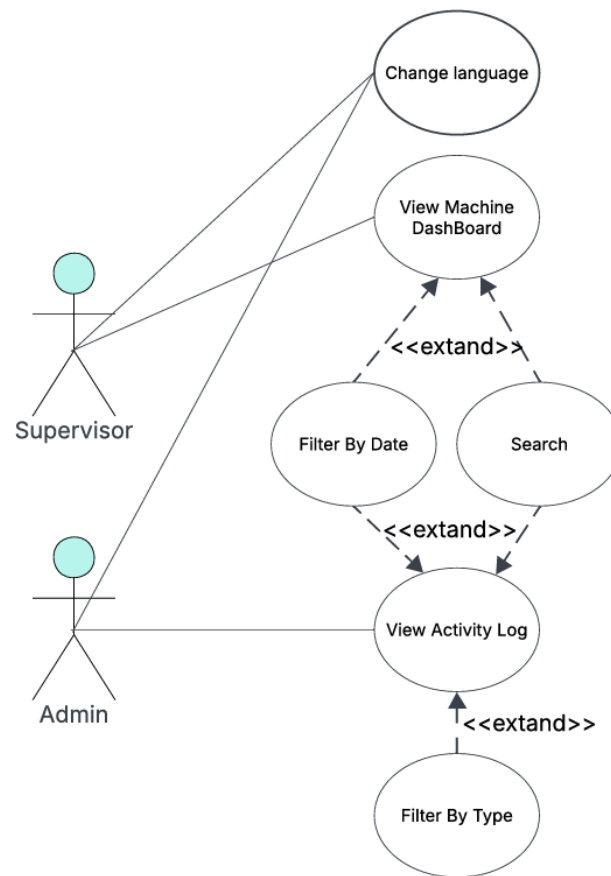


Figure 6.1. UML Use Case Diagram — Administration & UX.

6.3.2 Textual Use Case Description

The following table (Table 6.3) provides the detailed textual description of the *View Machine Dashboard* use case, which represents the most significant new administrator workflow in this sprint.

Table 6.3. Textual Use Case Description — View Machine Dashboard

Use Case Name	View Machine Dashboard
Primary Actor	Administrator
Preconditions	• The user is authenticated with the <i>Admin</i> role. • At least one MES Operation Execution record exists.

Continued on next page

Postconditions

- A grid of machine cards is displayed, each showing uptime, production totals, and scrap for the selected time window.
 - Changing the time-range selector refreshes all card metrics.
-

Main Scenario

1. The administrator logs in and is routed to AdminPage.
 2. The administrator selects *Machine Dashboard* in the sidebar.
 3. The system POSTs to /MESWebService_fetchMachineDashboard with the default hoursBack = 24.
 4. The backend aggregates execution and progression records and returns per-machine metrics.
 5. The MachineDashboardPage renders one AdminMachineCard per machine.
 6. The administrator changes the time range to 48 h via the AppBar dropdown; the page re-fetches and updates.
-

6.3.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for the administrator analytics flows introduced in Sprint 4.

Figure 6.2 depicts the machine dashboard retrieval flow. The AL layer iterates over all *Machine Center* records and, for each machine, computes the total operation count, cumulative produced and scrap quantities, and uptime percentage from the corresponding Business Central records before assembling the per-machine KPI rows returned to the Flutter client.

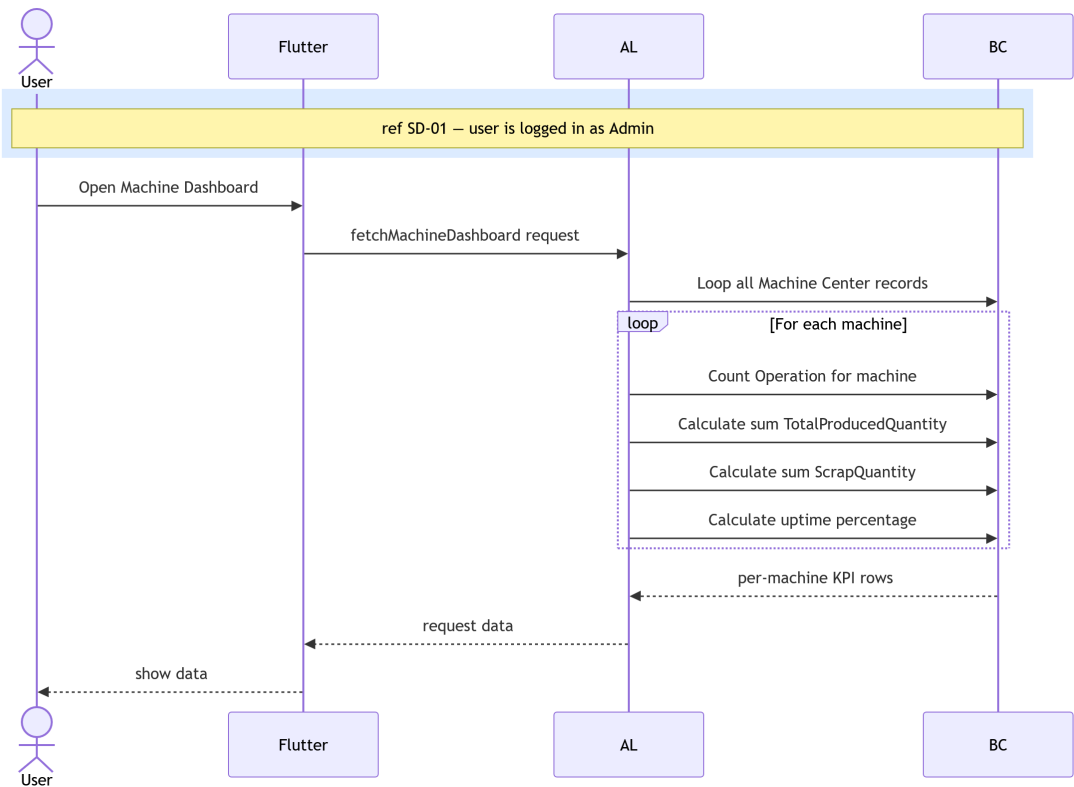


Figure 6.2. SD-19: Fetch Machine Dashboard.

Figure 6.3 presents the activity log retrieval flow. The AL layer queries four event tables in parallel — *MES Operation State*, *MES Operation Progression*, *MES Operation Scrap*, and *MES Component Consumption* — filtering each by the requested time window. For every event row it joins the associated *MES User* and *MES Operation Execution* records to enrich the entry with operator identity and execution context. The resulting flat event list is returned to the Flutter client, which renders it as a chronological activity table.

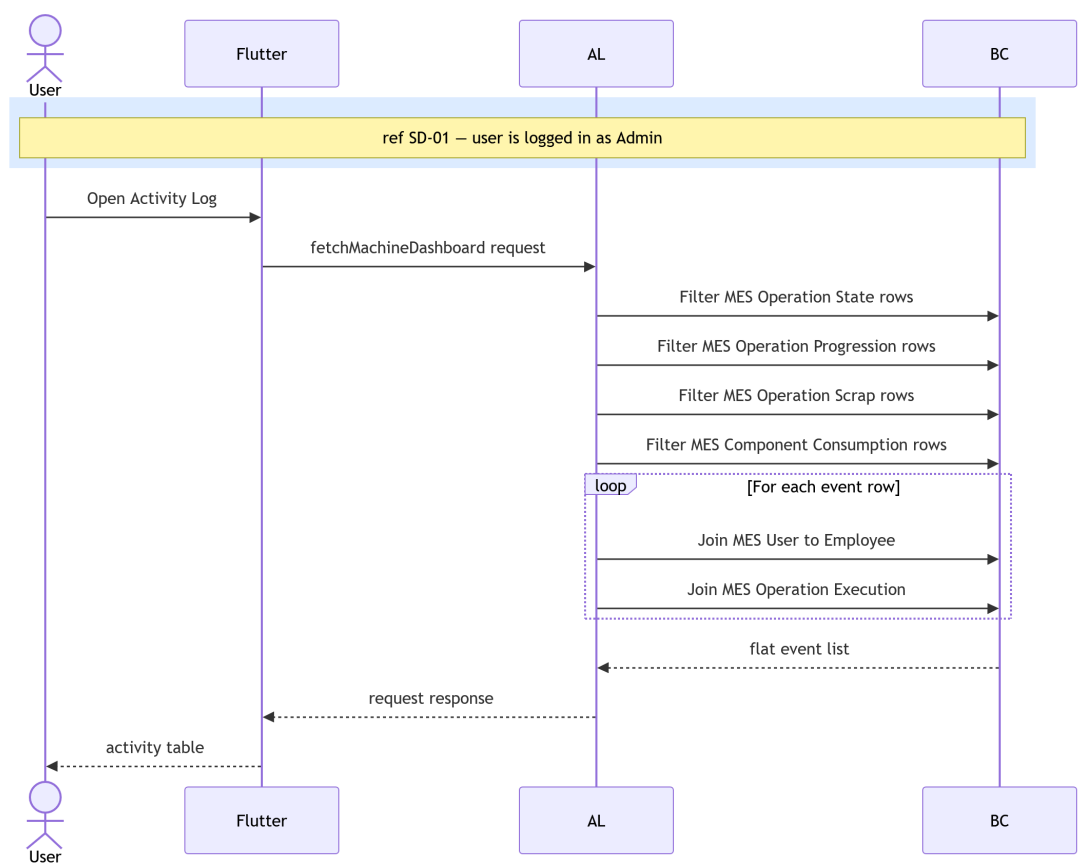


Figure 6.3. SD-20: Fetch Activity Log.

6.3.4 Class Diagram

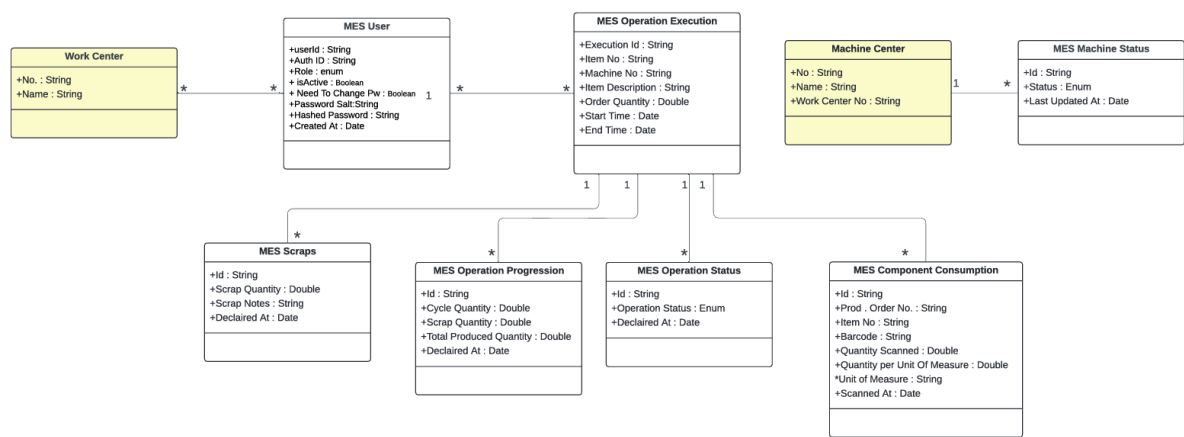


Figure 6.4. UML Class Diagram.

6.4 Implementation

6.4.1 Backend Implementation

No new Business Central tables are created in this sprint. All backend work consists of new query and aggregation procedures added to the existing MES Web Service codeunit (50126), reading from the tables defined in Sprints 1 through 3.

Dashboard Aggregation

`fetchMachineDashboard()` accepts a validated admin token and a `hoursBack` integer. It iterates all Machine Center records, and for each machine it filters MES Operation Execution records whose Start Time falls within the requested window. For each matched execution it sums the Cycle Quantity fields from linked MES Operation Progression records to obtain `totalProduced`, and counts records marked as scrap to obtain `totalScrap`. Running minutes are derived from the difference between End Time (or the current datetime for open executions) and Start Time. Uptime percentage is computed as $\text{runningMinutes} / (\text{hoursBack} \times 60) \times 100$, capped at 100.

Activity Log Aggregation

`fetchActivityLog()` queries three tables in descending timestamp order, filtered to the `hoursBack` window: MES Operation Status (type `status_change`), MES Operation Progression (type `production`), and MES Component Consumption (type `scan` or `scrap` depending on whether a scrap code is present). For each record the operator's first and last name are resolved from the Employee table via the Operator Id foreign key. The results are merged into a single chronological list before being returned as a JSON array.

API Endpoints Added in Sprint 4

Table 6.4. New API Endpoints — Sprint 4

Endpoint	Key Parameters	Response
<code>fetchMachineDashboard</code>	<code>token</code> , <code>hoursBack</code>	JSON array of MachineDashboardModel objects
<code>fetchActivityLog</code>	<code>token</code> , <code>hoursBack</code>	JSON array of ActivityLogModel objects

6.4.2 Frontend Implementation

Shared HTTP Infrastructure

All services in this sprint (and retroactively migrated services from earlier sprints) use a shared `HttpClient` and `HttpResponseParser` layer, replacing the inline `http.post` calls used in Sprint 1.

`HttpClient.post()`: centralised `http.post` wrapper that injects JSON content-type headers and encodes the request body. All services call this method rather than the raw `http` package directly.

`HttpResponseParser`: provides four static methods. `parseList()` decodes Business Central OData list responses (nested value string). `parseObject()` decodes single-object responses. `parseWriteResult()` decodes write-result responses and throws with the backend error message on failure. `_assertSuccess()` validates HTTP status codes and throws a structured exception for non-2xx responses.

Provider Architecture

Table 6.5. Provider Responsibilities — Sprint 4 Additions

Provider	Responsibility
LogProvider	Owns dashboard and activity log fetch calls; exposes <code>selectedHours</code> , <code>hourOptions</code> , and <code>setHours()</code> shared between the two admin pages.
MesBarcodeProvider	Exposes <code>fetchAllBarcodes()</code> , <code>insertScans()</code> , and <code>resolveBarcode()</code> ; resolves session token automatically from <code>AuthProvider</code> .

User Interface

Admin Navigation Shell (Figure 6.5) provides the persistent sidebar that frames all administrator pages. A 250 px dark panel on the left lists the four sections (Users & Roles, Machine Dashboard, Activity Logs, Settings), with the active section highlighted in blue. At the bottom, a user-info strip shows the administrator’s avatar, full name, and Auth ID, and navigates to `ProfilePage` on tap. The `IndexedStack` to the right keeps all pages alive simultaneously, preserving scroll position and loaded data as the administrator switches sections.



Figure 6.5. *Admin Navigation Shell* — desktop view.

Figures 6.6 and 6.7 illustrate the two main administration pages. The *Machine Dashboard Page* renders one *AdminMachineCard* per machine, each displaying a circular uptime indicator and four metric rows (operations, produced, scrap, running minutes); the *AppBar* hosts a time-range dropdown that re-fetches data on change. The *Activity Log Page* presents a paginated list of all system events ordered newest-first; each row shows a type icon, operator name, machine, action text (expandable for long entries), and timestamp; type and time-range dropdowns in the *AppBar* narrow the view.

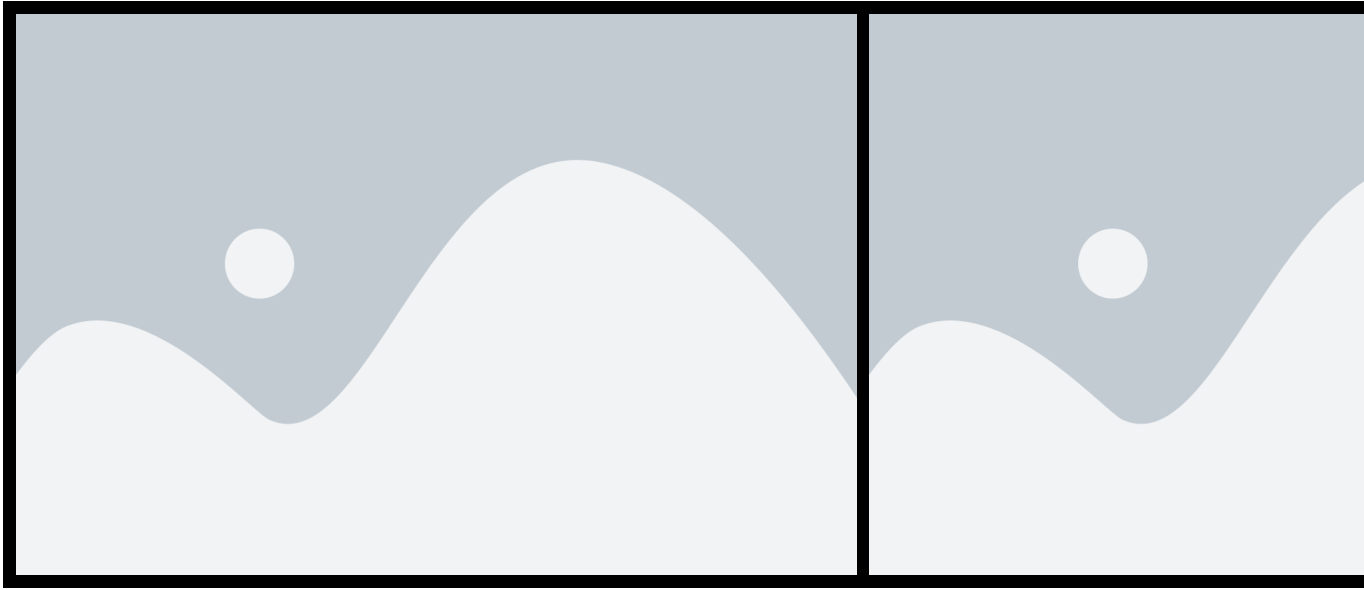


Figure 6.6. *Machine Dashboard Page* — desktop and mobile views.

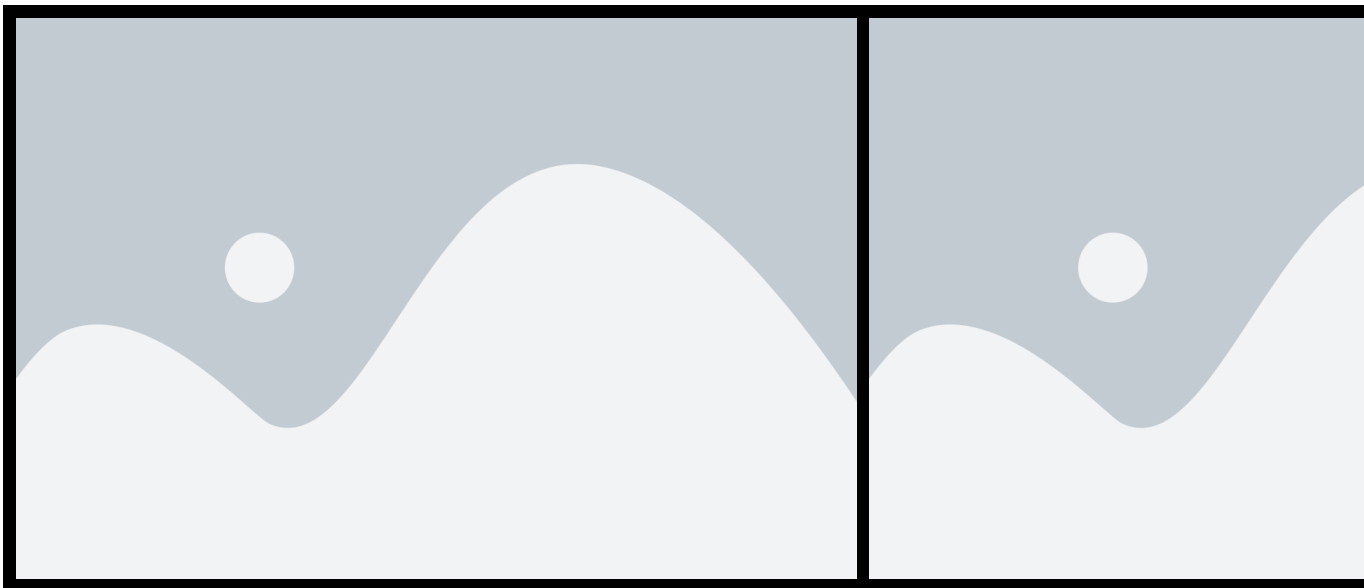


Figure 6.7. *Activity Log Page* — desktop and mobile views.

6.5 Test and validation

6.5.1 Integration test

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 6.8 shows a test case for the `/activityLog` endpoint.

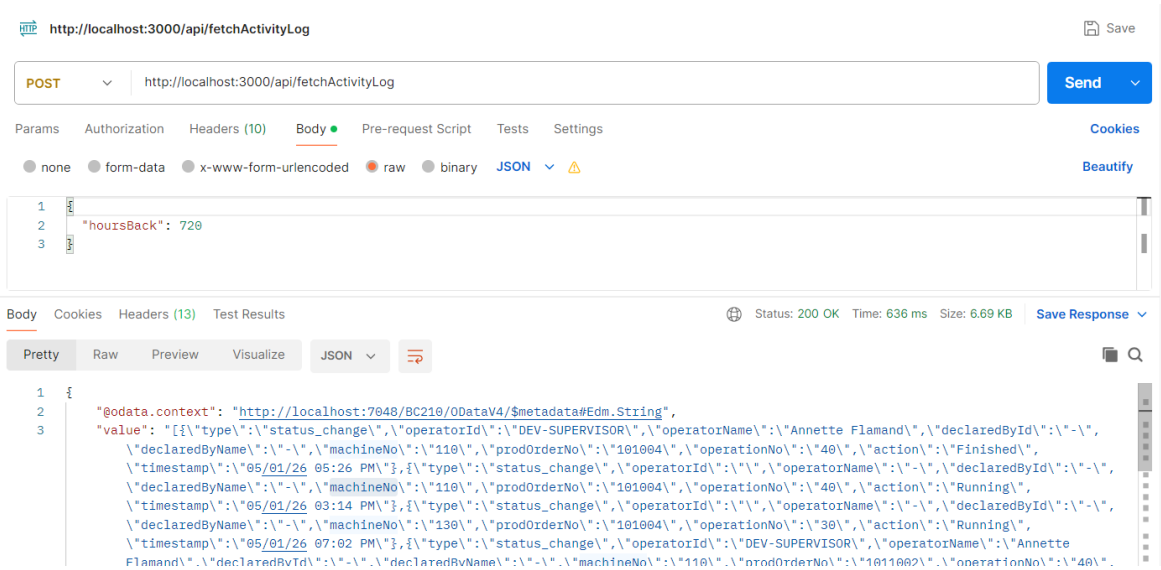
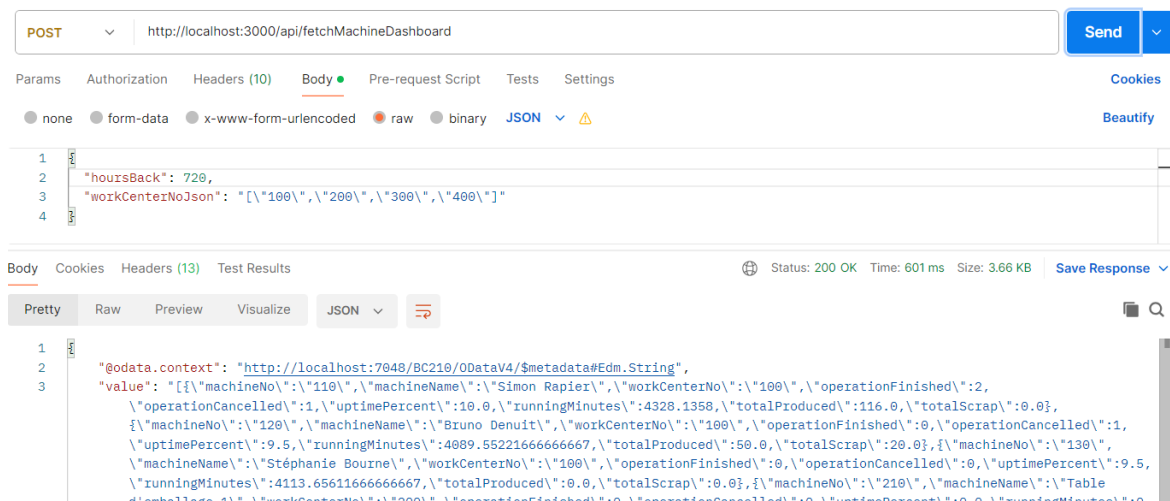
Figure 6.8. Postman endpoint test — `fetchActivityLog`.

Figure 6.9 shows a test case for the `/MachineDashboard` endpoint.

Figure 6.9. Postman endpoint test — `fetchMachineDashboard`.

Conclusion

This sprint completed the *MES* solution by delivering the administration layer and cross-cutting quality-of-life features. Administrators can now monitor the production health of the entire facility through a configurable machine performance dashboard, audit all operator and machine actions through a paginated activity log, and manage users from a persistent navigation shell without losing context between sections. All users benefit from full internationalisation covering English, French, and Arabic with automatic right-to-left layout, and from guided coach-mark tutorials that ease onboarding for first-time users.

All business rules defined at the start of the sprint have been enforced: time-range scoping

(BR-ADMIN-001), right-to-left layout and string externalisation (BR-I18N-001), and one-time tutorial display (BR-TUTORIAL-001). No new Business Central tables were required; all data is derived from the tables created in Sprints 1 through 3, demonstrating the extensibility of the append-only architecture established from the first sprint.

Chapter 7

Sprint 5 AI Assistant & Analytical Intelligence.

Contents

- Introduction 119
- 7.1 Sprint Backlog 119
- 7.2 Business Rules 122
- 7.3 Design 125
 - 7.3.1 Use Case Diagram 125
 - 7.3.2 Textual Use Case Description 126
 - 7.3.3 Sequence Diagrams 128
 - 7.3.4 Class Diagram 130
- 7.4 Implementation 131
 - 7.4.1 Python AI Agent 131
- Data Models 131
- Intent Classification 132
- Tool Catalogue 133
- Orchestration Pipeline 134
- Multi-Provider LLM Client 134
- 7.4.2 Node.js Middleware 135
- Endpoint Routing 135
- 7.4.3 Flutter Frontend 135
- AI Data Layer 135
- AI Domain Layer 135
- User Interface 136
- Conclusion 136

Introduction

This sprint report covers **Domain 5: AI Assistant & Analytical Intelligence**. Building upon the complete shop-floor execution loop established in the previous sprints, this sprint delivers the AI layer of the MES system: a natural-language chat assistant embedded in the Flutter frontend, a Node.js reverse proxy middleware that bridges the Flutter app to both Business Central and a locally-hosted Python AI agent, and the Python agent itself. The agent provides deterministic tool orchestration, multi-provider LLM synthesis, fuzzy machine name resolution, multi-machine fan-out, production data enrichment, and contextual redirect actions. The result is a conversational interface that operators and supervisors can use to query real-time machine status, production orders, scrap rates, BOM consumption, delays, and operator performance without leaving the MES application.

7.1 Sprint Backlog

This sprint covers **Domain 5: AI Assistant & Analytical Intelligence**.

Table 7.1. US-5.1 — AI Chat Interface

ID	User Story	Acceptance Criteria
US-5.1	As an Operator or Supervisor, I need an embedded chat assistant in the MES app so that I can query production data using natural language without switching tools.	<i>Flutter:</i> • Create model ConversationTurn, AiRedirectAction, and AiChatResponse. • Create AiChatService to POST to the /api/ai/chat endpoint. • Create AiChatProvider to manage conversation history, loading state, and the last response. • Create page AiChatPage supporting both modal dialog and embedded side-panel modes. • Integrate the chat panel into MachineListPage as a collapsible side panel on desktop and a modal dialog on mobile. • Implement <code>_handleAction()</code> to process AiRedirectAction objects (redirect to machine, operation, or machine list).

Continued on next page

ID	User Story	Acceptance Criteria
US-5.2	As a Supervisor, I need the AI to understand questions about multiple machines simultaneously so that I can get fleet-wide answers from a single query.	<i>Python AI Agent:</i> • Implement composite query detection in <code>composite.py</code>. • Implement fan-out batch execution across machines using <code>asyncio.gather</code> in batches of 4. • Implement <code>build_composite_context()</code> to produce a compact markdown context for the LLM. • Cap fan-out at 12 machines to prevent timeout. • Integrate the composite path into <code>MESAgentOrchestrator</code>.
US-5.3	As an Operator or Supervisor, I need to refer to machines by colloquial names or partial identifiers so that I do not need to memorise exact machine codes.	<i>Python AI Agent:</i> • Implement <code>build_machine_index()</code> to build a lookup structure from the machine list. • Implement a five-tier fuzzy resolution cascade in <code>resolve_machine()</code> (exact canonical, normalised exact, digit suffix, ID prefix/suffix, token overlap, edit distance). • Define <code>MIN_CONFIDENCE = 0.45</code> and <code>HIGH_CONFIDENCE = 0.85</code> thresholds. • Implement <code>format_candidates_for_llm()</code> to surface ambiguous matches to the user. • Integrate resolver into orchestrator when <code>unresolved_machine_ref</code> is set.

Continued on next page

ID	User Story	Acceptance Criteria
US-5.4	As a Supervisor, I need the AI to alert me to overdue orders, excessive pauses, and high scrap rates so that I can take corrective action without manually reviewing dashboards.	<i>Python AI Agent:</i> • Implement <code>enrich_deadlines()</code> in <code>data_analysis.py</code> to compute <code>hours_until_deadline</code>, <code>is_overdue</code>, and <code>risk_level</code> per order. • Implement <code>analyse_scrap()</code> to compute <code>scrap_rate</code>, <code>severity</code> (ok / warning / critical), <code>spike_detected</code>, and <code>interpretation</code>. • Implement <code>analyse_velocity()</code> to compute <code>units_per_hour</code>, <code>required_rate</code>, <code>projected_end</code>, and <code>velocity_status</code>. • Wire enrichment results into the LLM context block before synthesis.
US-5.5	As an Operator or Supervisor, I need intent classification to work reliably without requiring a network call to an LLM so that the assistant remains responsive even when the LLM is slow.	<i>Python AI Agent:</i> • Implement <code>intent.py</code> with 12 intent classes and a priority-ordered regex rule cascade. • Implement <code>extraction_helpers</code>: <code>_extract_machine_no()</code>, <code>_extract_machine_ref()</code>, <code>_extract_order_no()</code>, <code>_extract_op_no()</code>, <code>_extract_hours()</code>. • Define the <code>ToolStep</code> and <code>ToolChain</code> data types. • Implement <code>llm_intent.py</code> as the primary LLM-based classifier with automatic fallback to regex.

Continued on next page

ID	User Story	Acceptance Criteria
US-5.6	As a Supervisor, I need the AI to provide clickable navigation actions in its replies so that I can jump directly to a relevant machine or operation.	<i>Python AI Agent:</i> • Define the ActionType enum with 5 redirect types in <code>models.py</code>. • Implement <code>_parse_actions()</code> in the orchestrator to extract a <code>``actions``</code> JSON block from the LLM reply. • Return up to 4 RedirectAction objects in ChatResponse. <i>Flutter:</i> • Render <code>_ActionButtons</code> in <code>AiChatPage</code> as a Wrap of tappable ElevatedButtons below each assistant reply. • Implement <code>_handleAction()</code> routing per action type.

7.2 Business Rules

Table 7.2. Business Rules

ID: Business Rule	Description
AI Intent Classification Rules	
BR-AI-001: Two-Tier Classification with Mandatory Fallback	The primary classifier (<code>llm_intent.py</code>) sends a structured prompt to the LLM and validates the returned JSON plan against a schema. If the LLM response is malformed, missing required fields, references an unknown tool name, or contains duplicate <code>result_key</code> values, the system falls back unconditionally to the deterministic regex classifier (<code>intent.py</code>). The fallback reason is recorded in the <i>ThinkingBlock</i> but never surfaced to the end user.

Continued on next page

ID: Business Rule	Description
BR-AI-002: LLM Does Not Control Tool Selection	The LLM's role is limited to producing a <i>plan</i> (a list of tool names and arguments). Actual tool execution, access-control checks, slot resolution, and enrichment are performed entirely in Python. No tool is ever called solely on the LLM's instruction without passing through Python validation. This prevents prompt-injection attacks from influencing which data is fetched.
BR-AI-003: Exactly Two LLM Calls Per Request	Each /chat request triggers at most two LLM calls: one optional call in <code>LLMIntentIdentifier.classify()</code> for intent planning, and one mandatory call in <code>_synthesise()</code> for response generation. The regex fallback path uses zero LLM calls for classification. Tool execution never triggers additional LLM calls.
Access Control Rules	
BR-AI-004: Work-Center-Scoped Machine Access	The orchestrator builds an <code>allowed_machines</code> dictionary by calling <code>ListMachinesTool</code> with the user's work centres extracted from the session token. Every subsequent call to a machine-scoped tool is checked against this dictionary before execution. If the requested machine is not in the allowed set, the tool call is skipped and an access-denial message is included in the context. The LLM never influences this check.
BR-AI-005: Token Forwarded to BC; Stripped from AI Calls	The session bearer token is included in the <code>user_context</code> forwarded to the Python agent and is passed on to BC web service calls that require it (e.g., <code>fetchMyData</code>). The token is <i>never</i> sent to the external LLM provider; it is only used for BC HTTP calls inside <code>mes_tools.py</code> .
Machine Name Resolution Rules	
BR-AI-006: Five-Tier Fuzzy Resolution with Confidence Thresholds	When a user refers to a machine by a colloquial name or partial ID, <code>resolver.py</code> applies a five-tier match cascade. A confidence score below <code>MIN_CONFIDENCE = 0.45</code> causes the assistant to ask the user to clarify.

Continued on next page

ID: Business Rule	Description
	A score between 0.45 and <code>HIGH_CONFIDENCE = 0.85</code> causes the assistant to proceed while noting the assumption. A score at or above 0.85 causes silent resolution. The five tiers are: (1) exact canonical ID, (2) normalised exact, (3) digit suffix match, (4) ID prefix/suffix overlap, and (5) token overlap and edit-distance scoring on the machine name.
Fan-Out Rules	
BR-AI-007: Composite Queries Capped at 12 Machines	When a query is classified as composite (targeting multiple or all machines), <code>composite.py</code> filters the working set to the relevant machines (e.g., only Working machines for a running-status query), sorts them with Working machines first, and caps the fan-out at <code>MAX_FANOUT_MACHINES = 12</code> . Tool calls for the selected machines are dispatched in parallel batches of <code>BATCH_SIZE = 4</code> using <code>asyncio.gather</code> to prevent BC from being overwhelmed.
Data Enrichment Rules	
BR-AI-008: Pre-Computed Analytics Before LLM Synthesis	Numerical analysis (scrap rates, deadline risk levels, production velocity, utilisation percentages) is computed in Python by <code>data_analysis.py</code> <i>before</i> the LLM receives any data. The LLM is presented with pre-computed facts (e.g., <code>severity: "critical", is_overdue: true</code>) rather than raw numbers. This prevents numeric hallucination and ensures that alerts are deterministic.
Middleware Security Rules	
BR-MW-001: SSPI Credential Isolation	The Node.js middleware strips the Flutter Authorization header before forwarding any request to Business Central. Authentication to BC is performed exclusively by the PowerShell child process using <code>UseDefaultCredentials = \$true</code> , which relies on the Windows session of the service account running the proxy. No BC credentials are stored in environment variables or configuration files accessible to the Node process.

Continued on next page

ID: Business Rule	Description
BR-MW-002: SSPI Challenge Header Suppression	The WWW-Authenticate header returned by Business Central during SSPI negotiation is stripped from all responses before they are forwarded to the Flutter client. This prevents the client from attempting to handle Kerberos/NTLM challenges that it has no capacity to process.
BR-MW-003: AI Route Priority	The Express middleware stack mounts aiProxy.js at /api/ai <i>before</i> the BC catch-all handler forwardRequest. This ordering guarantees that no AI chat request ever reaches the PowerShell spawner. If aiProxy.js is removed or fails to load, the AI route falls through to the BC handler and returns an error rather than silently calling BC.
Conversation Rules	
BR-AI-009: Conversation History Managed by Flutter	The AiChatProvider maintains the full conversation history in memory as a List<ConversationTurn>. Each /chat call includes the history up to but not including the current turn. History is not persisted between application sessions. When the user clears the chat, history and the last response are both nulled.
BR-AI-010: LLM Context Capped Before Synthesis	The context block assembled from tool results and enrichment data is truncated to LLM_MAX_CONTEXT_CHARS before being inserted into the synthesis prompt. This prevents token-limit errors regardless of how many machines or cycles were queried. The cap is configurable via an environment variable and defaults to a value defined in config.py.

7.3 Design

7.3.1 Use Case Diagram

This sprint adds a single new actor, the *AI Assistant*, which mediates between the human user roles and the data layer. The primary use cases are: Chat with AI Assistant, Query Machine Status, Query Production Orders, Query Scrap Report, Query Operator Summary, Query Delay Report, Query BOM Consumption, Navigate to Machine (redirect action), and Navigate to Operation (redirect action).

Both the Operator and Supervisor roles can interact with the AI assistant for read-only queries scoped to their assigned work centres. The Supervisor additionally receives fleet-wide compos-

ite query results and supervisor-specific overview data. The resulting use case diagram is shown in Figure 7.1.



Figure 7.1. UML Use Case Diagram — Sprint 5

7.3.2 Textual Use Case Description

Table 7.3. Textual Use Case Description — Chat with AI Assistant

Use Case Name	Chat with AI Assistant
Primary Actor	Operator / Supervisor
Preconditions	• The user is authenticated and holds a valid session token. • The AI chat panel is open (side panel on desktop, modal on mobile). • The Node.js middleware and Python AI agent are running.
Postconditions	• An assistant reply is appended to the conversation history. • Up to 4 <i>RedirectAction</i> buttons may be rendered below the reply. • The conversation history is updated in AiChatProvider.

Continued on next page

Main Scenario

1. The user types a message and taps the send button.
2. AiChatProvider appends a *user* turn to the history and calls `AiChatService.sendMessage()`.
3. The service POSTs to `/api/ai/chat` with the message, user context (userId, role, work centres, token), and conversation history.
4. The Node.js middleware routes the request to `/api/ai` and forwards it to the Python AI agent.
5. The AI agent classifies the intent, executes the tool chain against Business Central via the middleware, enriches the results, and synthesises a reply.
6. The reply and optional redirect actions are returned as a `ChatResponse` JSON object.
7. The Flutter UI appends the *assistant* turn and renders any action buttons.

Alternative Flows

- *LLM intent classification fails*: the regex classifier takes over; the user receives the same response with no visible difference.
- *Machine name is ambiguous*: the assistant asks the user to clarify and lists the top candidate machines.
- *AI agent timeout (60 s)*: the middleware returns a 504 error; the provider sets `errorMessage` and removes the failed user turn from history.
- *Machine not in user's work centre*: the tool call is skipped; the assistant informs the user of the access restriction.

Table 7.4. Textual Use Case Description — Query Supervisor Overview

Use Case Name	Query Supervisor Overview
Primary Actor	Supervisor
Preconditions	• The user is authenticated with the <i>Supervisor</i> role. • The AI assistant is open.

Continued on next page

Postconditions

- The assistant returns a structured summary of stopped machines, idle operators, abnormal pauses, high-scrap operations, and delayed orders.
 - Redirect actions allow the supervisor to navigate directly to flagged machines.
-

Main Scenario

1. The supervisor types a broad query such as “give me a shift overview”.
 2. The intent classifier maps the message to the `SUPERVISOR_OVERVIEW` intent.
 3. The orchestrator calls `GetSupervisorOverviewTool` which in turn calls `fetchSupervisorOverview` on the BC web service.
 4. The returned JSON (stopped machines, idle operators, abnormal pauses, high-scrap operations, delayed orders) is enriched with delay risk levels by `data_analysis.py`.
 5. The LLM synthesises a concise shift report and emits up to 4 redirect actions pointing to the most critical machines.
 6. The supervisor taps a redirect action to navigate directly to the flagged machine page.
-

7.3.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Node.js middleware, the Python AI Agent, and Business Central for the primary AI interaction flows.

Figure 7.2 depicts the main chat flow: the Flutter client sends a message with user context to the middleware, which routes it to the Python agent. The agent classifies the intent, executes the BC tool chain via the middleware’s PowerShell proxy, enriches the results, and synthesises a reply using the configured LLM.

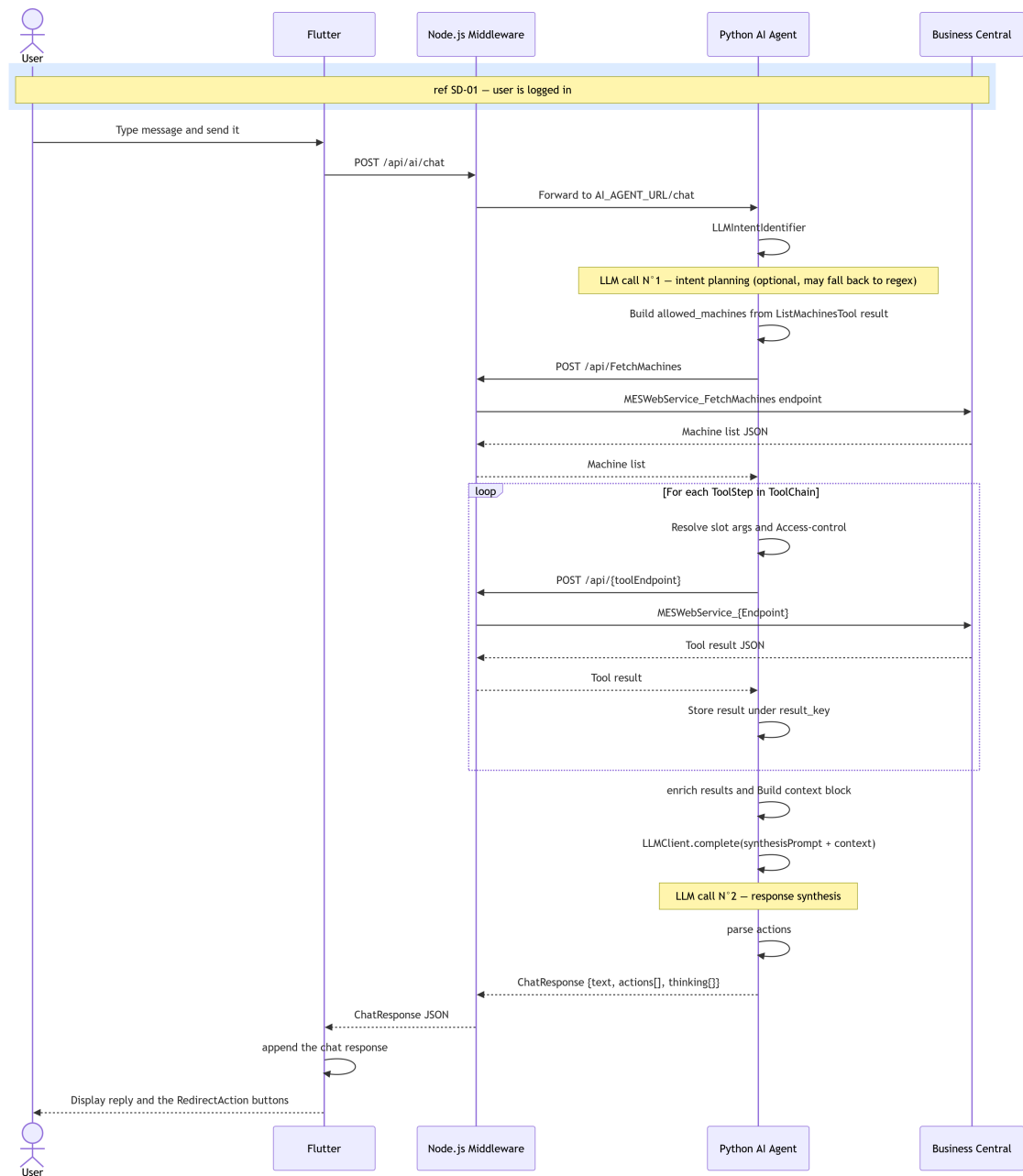


Figure 7.2. SD-30: AI Chat — Main Flow.

Figure 7.3 illustrates the two-tier intent classification flow, showing the LLM-based primary classifier, the JSON plan validation step, and the fallback path to the regex classifier. The *ThinkingBlock* capturing the classification trace is also shown.

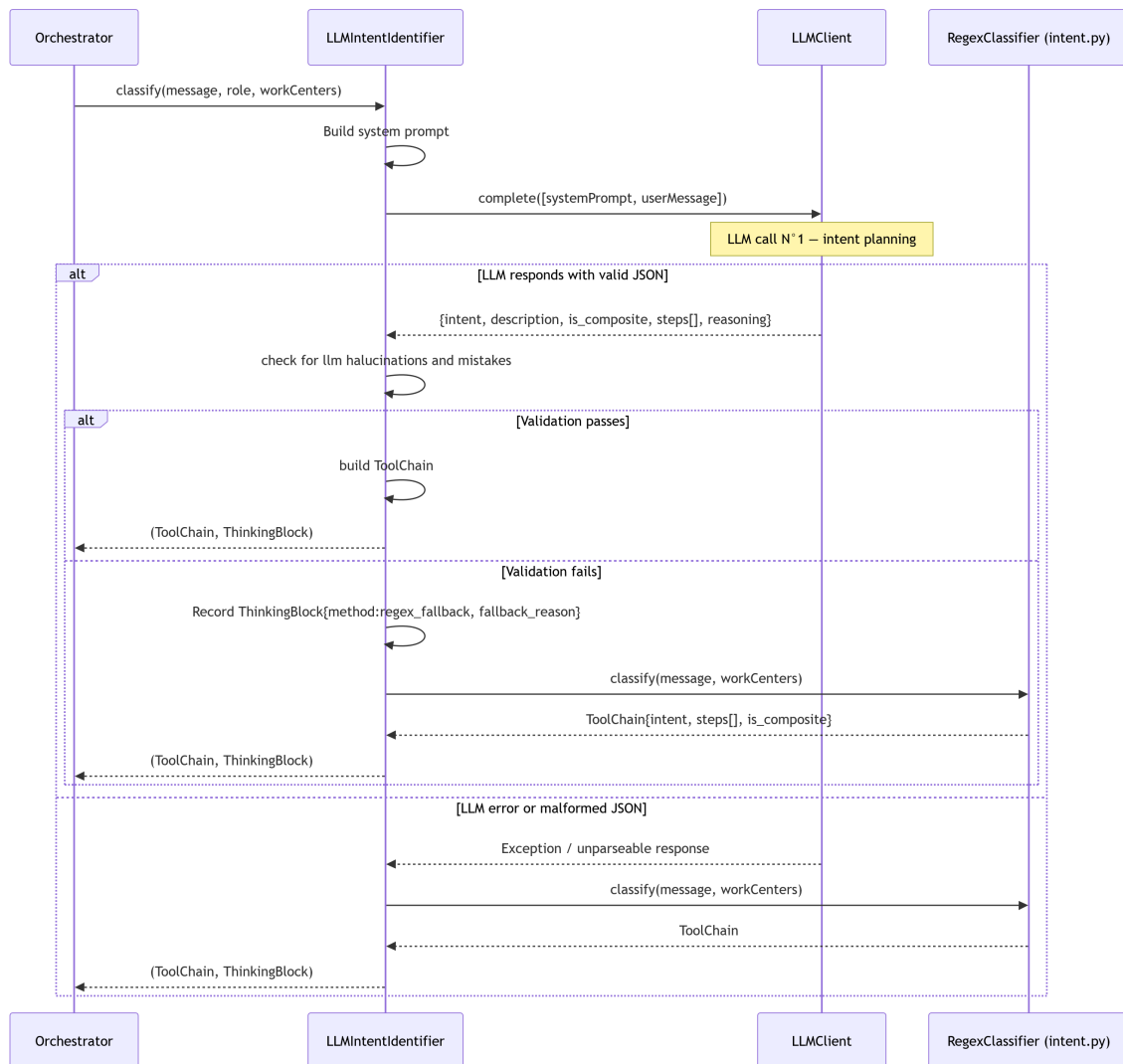


Figure 7.3. SD-31: Intent Classification — LLM with Regex Fallback.

7.3.4 Class Diagram

The UML Class Diagram (Figure 7.4) shows the Python AI agent's domain model. MES Agent Orchestrator is the central coordinator, holding references to an LLM Intent Identifier, an LLM Client (the abstract interface implemented by six concrete provider classes), and the TOOL MAP registry. Tool Chain is a value object produced by the classifiers and consumed by the orchestrator; it contains an ordered list of Tool Step records. All tool classes implement a common `execute()` interface. Chat Request and Chat Response are the inbound and outbound Pydantic contracts of the `/chat` endpoint.



Figure 7.4. UML Class Diagram — AI Assistant & Analytical Intelligence.

7.4 Implementation

7.4.1 Python AI Agent

Data Models

Table 7.5. Pydantic Models — agent/models.py

Class	Key Fields	Description
ConversationTurn	role, content	One message in the conversation history. Role is either "user" or "assistant".
UserContext	user_id, role, work_centers, token	Session identity extracted from the Flutter session storage and forwarded with every chat request.
ChatRequest	message, user_context, conversation_history	Inbound contract for the POST /chat endpoint. conversation_history defaults to an empty list.

Continued on next page

Class	Key Fields	Description
ActionType	enum: 5 values	Defines the five redirect types the LLM may emit: <code>redirect_machine</code> , <code>redirect_operation</code> , <code>redirect_machine_list</code> , and two reserved types.
RedirectAction	<code>action_type</code> , <code>label</code> , <code>payload</code>	A clickable navigation action sent back to Flutter and rendered as an <code>ElevatedButton</code> .
ChatResponse	<code>text</code> , <code>actions</code> , <code>data_fetched</code> , <code>thinking</code>	Outbound contract. <code>text</code> is the human-readable reply; <code>actions</code> holds up to 4 redirect actions; <code>thinking</code> carries the classification trace for debugging.
ToolResult	<code>tool_name</code> , <code>success</code> , <code>data</code> , <code>error</code>	Internal result wrapper returned by every tool's <code>execute()</code> method.

Intent Classification

The intent classification layer comprises two components working in tandem. `llm_intent.py` is the primary classifier: it sends a terse system prompt listing all 17 available tools to the configured LLM and expects a JSON response containing the intent label, a steps array, and an optional reasoning field. This plan is validated by `_validate_plan()`, which checks that the intent label is known, all tool names exist in `TOOL_MAP`, and no `result_key` is duplicated across steps.

If validation fails for any reason — malformed JSON, unknown tool, missing required field — control passes to `intent.py`, the deterministic regex classifier. Table 7.6 lists the 12 intent classes and their primary trigger patterns.

Table 7.6. Intent Classes — `agent/intent.py`

Intent	Primary Trigger Patterns
BOM	<i>bom, component, material, consumption, scan</i>
PRODUCTION_CYCLES	<i>cycle, declaration, declared, production history</i>
OPERATION_LIVE	<i>live, current operation, running, in progress</i>
OPERATION_HISTORY	<i>history, past operation, finished, cancelled</i>

Continued on next page

Intent	Primary Trigger Patterns
ORDERS	<i>order, production order, PO-, scheduled</i>
MACHINE_LIVE	<i>machine status, what is MC-, ongoing</i>
DASHBOARD	<i>dashboard, summary, overview, KPI</i>
MACHINE_STATUS	<i>idle, working, which machines are</i>
SCRAP_ANALYSIS	<i>scrap, reject, waste, defect</i>
ACTIVITY	<i>activity, event log, what happened, recent actions</i>
DEPARTMENT	<i>work center, department, sector, operator summary</i>
GENERAL	Catch-all when no higher-priority rule matches.

Tool Catalogue

Table 7.7 lists all 17 tool classes registered in TOOL_MAP, together with the Business Central endpoint each tool calls and its primary arguments.

Table 7.7. MES Tool Catalogue — tools/mes_tools.py

Tool Class	BC Endpoint	Key Arguments
ListMachinesTool	FetchMachines	work_center_no
GetMachineOrdersTool	getMachineOrders	machine_no
GetOngoingOperationsTool	fetchOngoing- OperationsState	machine_no
GetOperationLiveDataTool	fetchOperationLiveData	machine_no, prod_order_no, operation_no
GetProductionCyclesTool	fetchProductionCycles	machine_no, prod_order_no, operation_no
GetOperationsHistoryTool	fetchOperationsHistory	machine_no
GetActivityLogTool	fetchActivityLog	hours_back
GetMachineDashboardTool	fetchMachineDashboard	hours_back, work_center_nos

Continued on next page

Tool Class	BC Endpoint	Key Arguments
GetProductionOrdersTool	fetchProductionOrders	status_filter, work_center_no, machine_no
GetWorkCenterSummaryTool	fetchWorkCenterSummary	work_center_nos, hours_back
GetOperatorSummaryTool	fetchOperatorSummary	work_center_nos, hours_back
GetMyDataTool	fetchMyData	hours_back (token-scoped)
GetScrapSummaryTool	fetchScrapSummary	hours_back, multi-filter
GetDelayReportTool	fetchDelayReport	work_center_nos, pause_threshold_minutes
GetConsumptionSummaryTool	fetchConsumptionSummary	prod_order_no, operation_no, machine_no, hours_back
GetSupervisorOverviewTool	fetchSupervisorOverview	work_center_nos, hours_back, pause_threshold_minutes
GetBomTool	fetchBom	prod_order_no, operation_no

Orchestration Pipeline

MESAgentOrchestrator.run() processes each ChatRequest through an eight-step pipeline. Step 1 calls LLMIntentIdentifier.classify() (with regex fallback) to produce a ToolChain. Step 2 routes to the composite path (if is_composite = true) or the normal sequential path. Step 3 resolves any unresolved_machine_ref using resolver.py. Step 4 iterates the tool steps: slot arguments prefixed with \$ are resolved against prior step results, access-control is verified against allowed_machines, and the tool is executed. Step 5 applies data_analysis.py enrichment per result key. Step 6 assembles the LLM context block and truncates it to LLM_MAX_CONTEXT_CHARS. Step 7 calls the LLM once for synthesis. Step 8 extracts any ```actions``` block from the reply and separates it from the displayed text.

Multi-Provider LLM Client

The build_llm_client() factory reads the LLM_PROVIDER environment variable and instantiates the corresponding client class. All clients expose the same async complete(messages:

list) → str interface so the rest of the agent is provider-agnostic. Table 7.8 summarises the supported providers.

Table 7.8. Supported LLM Providers — agent/llm_client.py

Client Class	Provider	Notes
OllamaClient	Local Ollama	No external credential required; model served locally.
OpenAIClient	OpenAI	Requires OPENAI_API_KEY.
HuggingFaceClient	HF Serverless Inference	Requires HF_API_TOKEN.
OpenAICompatibleClient	Groq, Together, Mistral, vLLM, LM Studio	Uses the OpenAI Chat Completions schema with a configurable base URL.
AnthropicClient	Anthropic	Extracts the system prompt from the messages list before calling the Messages API.

7.4.2 Node.js Middleware

Endpoint Routing

We configured the middleware to map all calls coming from the frontend using the structure /api/ai to the ai proxy handler which forwards them to the /chat endpoint of the Python agent. The agents tool calls are treated through the same path as the normal business central function calls by the flutter frontend.

7.4.3 Flutter Frontend

AI Data Layer

The Flutter data layer for the AI feature comprises three classes. AiChatService constructs the full request payload — including user context assembled from SessionStorage at call time — and POSTs it to /api/ai/chat. It deserialises the response into an AiChatResponse object. ConversationTurn is a minimal data class with role and content fields and a toJson() serialiser. AiChatResponse holds the reply text, the list of AiRedirectAction objects, and an optional error string.

AI Domain Layer

AiChatProvider extends ChangeNotifier and owns the conversation history as a List<ConversationTurn>. Its sendMessage() method appends the user turn before calling the service, and on success appends the assistant turn and stores lastResponse. On failure it

removes the failed user turn and sets `errorMessage`. `clearHistory()` resets both the history and the last response, enabling the user to start a fresh conversation.

User Interface

Figure 7.6 illustrate the *AI Chat* interface in its two display modes. On desktop (width ≥ 600 px), the chat panel opens as a 400 px fixed-width side panel anchored to the right of the *Machine List Page*, and the floating action button (FAB) is hidden while the panel is open. On mobile, the chat opens as a full Dialog widget (max 800×800 px). In both modes the interface comprises the same four sub-widgets: `_EmptyState` (shown before the first message), `_MessageBubble` (dark background for user turns, light grey for assistant turns, capped at 78% of available width), `_ActionButtons` (a `Wrap` of `ElevatedButtons` for each `RedirectAction`), and `_InputBar` (a `TextField` with a send icon button that switches to a `CircularProgressIndicator` while a request is in flight).



Figure 7.5. *AI Chat* dialog — mobile view.

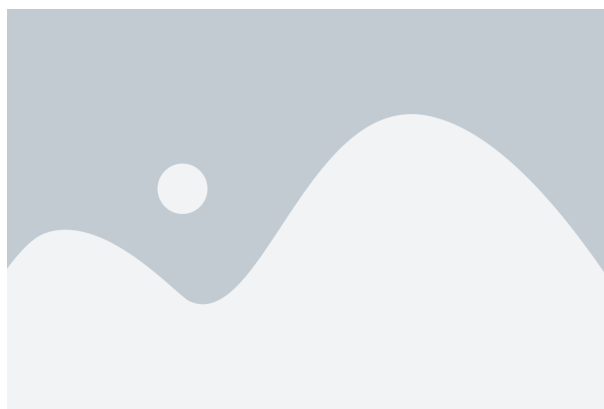


Figure 7.6. *AI Chat* dialog — mobile view.

Conclusion

This sprint delivered the complete AI intelligence layer of the MES system. Operators and supervisors can now query production data, machine status, scrap rates, BOM consumption, operator

performance, and shift overviews using natural language through the embedded chat assistant. The two-tier intent classification architecture — LLM-based planning with deterministic regex fallback — ensures reliable and responsive behaviour regardless of LLM availability. Work-centre-scoped access control, implemented entirely in Python and independent of LLM output, guarantees that users can only query data within their authorised scope. The multi-provider LLM client makes the system adaptable to different deployment environments, from a locally-hosted Ollama model to cloud providers such as Anthropic or OpenAI. The Node.js middleware resolves the SSPI authentication challenge inherent to Business Central's OData layer while keeping the Flutter client and Python agent free of Windows-specific dependencies. Together, these components form a production-ready conversational intelligence layer that integrates seamlessly with the shop-floor execution features delivered in the preceding sprints.

General Conclusion

This report has presented the full lifecycle of the MES (*Manufacturing Execution System*) project developed at **Mavision** over the course of four sprints. Starting from a blank slate, the team designed and implemented a complete shop-floor execution system natively integrated with Microsoft Dynamics 365 Business Central.

Sprint 1 established the security and identity foundation: a token-based authentication system, role-based access control, and a full administrator interface for user and account management. Sprint 2 delivered real-time machine monitoring, production order management, and a complete operation history trail. Sprint 3 completed the production execution loop with cycle declarations, pause, resume, and close flows, Bill of Materials visibility, component scanning, scrap tracking, and label printing. Sprint 4 added the administration layer with a machine performance dashboard, a paginated activity log, multi-language support covering English, French, and Arabic, and in-app onboarding tutorials.

Across all four sprints, the append-only data model ensures a tamper-evident audit trail from every machine state change through to every production cycle declaration. The reactive stream architecture in the Flutter frontend provides operators and supervisors with a consistent, real-time view of the production floor without manual refreshes.

The project demonstrates that a tightly integrated MES solution can be built on top of Microsoft Dynamics 365 Business Central using OData V4 unbound actions, without requiring additional third-party middleware. The extensible layered architecture positions the system for future enhancements such as statistical reporting, alert management, and additional language support.

Bibliography

- [1] Mavision, *Official Website*, <https://www.mavision.tn>, visited April 2025.